

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

[ПОТРЕБУЄ УТОЧНЕННЯ: інститут]

[ПОТРЕБУЄ УТОЧНЕННЯ: кафедра]

[ПОТРЕБУЄ УТОЧНЕННЯ: Ім'я ПРИЗВИЩЕ]

(група [ПОТРЕБУЄ УТОЧНЕННЯ: група])

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА
ВЕБСЕРВІС ДЛЯ ПОТОКОВОЇ ПЕРЕДАЧІ МЕДІАКОНТЕНТУ З
ФУНКЦІЄЮ ЗАХОПЛЕННЯ ОКРЕМИХ ЗАСТОСУНКІВ [ПОТРЕБУЄ
УТОЧНЕННЯ: за наказом]

Спеціальність:

[ПОТРЕБУЄ УТОЧНЕННЯ: спеціальність]

Освітньо-професійна програма:

[ПОТРЕБУЄ УТОЧНЕННЯ: освітня програма]

Керівник:

[ПОТРЕБУЄ УТОЧНЕННЯ: керівник]

Одеса – [ПОТРЕБУЄ УТОЧНЕННЯ: рік]

АНОТАЦІЯ

[ПОТРЕБУЄ УТОЧНЕННЯ: Прізвище І. Б.] Вебсервіс для потокової передачі медіаконтенту з функцією захоплення окремих застосунків [ПОТРЕБУЄ УТОЧНЕННЯ: тема за наказом ректора] : кваліфікаційна робота бакалавра за спеціальністю «[ПОТРЕБУЄ УТОЧНЕННЯ: код і найменування спеціальності]» / [ПОТРЕБУЄ УТОЧНЕННЯ: Ім'я По батькові Прізвище] ; керівник [ПОТРЕБУЄ УТОЧНЕННЯ: Ім'я По батькові Прізвище]. – Одеса : Нац. ун-т «Одес. політехніка», [ПОТРЕБУЄ УТОЧНЕННЯ: рік]. – [ПОТРЕБУЄ УТОЧНЕННЯ: загальна кількість сторінок після верстки] с.

Кваліфікаційна робота містить основну текстову частину на [ПОТРЕБУЄ УТОЧНЕННЯ: кількість після верстки] сторінках, список використаних джерел з 83 найменувань на [ПОТРЕБУЄ УТОЧНЕННЯ: кількість після верстки] сторінках, додатки на [ПОТРЕБУЄ УТОЧНЕННЯ: кількість після верстки] сторінках.

Метою кваліфікаційної роботи є забезпечення трансляції окремого застосунку разом із його власним звуком глядачам у браузері з низькою затримкою доставки шляхом розроблення вебсервісу stream-share та настільного застосунку для захоплення.

У роботі проаналізовано технології доставки медіапотоків, топології багатосторонньої передачі та обмеження захоплення екрана у браузері, порівняно шість наявних рішень і визначено нішу сервісу. Сформульовано функціональні та нефункціональні вимоги, оцінено трудомісткість методом точок варіантів використання, складено календарний план і реєстр ризиків. Спроектовано архітектуру з вебзастосунку, сервера сигналізації з медіасервером SFU і настільного застосунку, протокол сигналізації, модель даних і класи сервера.

У результаті створено програмну систему, у якій настільний застосунок Electron захоплює обране вікно разом зі звуком лише процесу-власника цього вікна та автоматично перемикає трансляцію на нові вікна того самого застосунку, сервер на основі mediasoup пересилає потік глядачам через WebRTC, а глядач

переглядає трансляцію у браузері, зокрема як гість без облікового запису. Статична перевірка пройшла для всіх пакетів, 27 із 30 автоматизованих функціональних прикладів дали очікуваний результат, а виявлені дефекти авторизації сервера сигналізації визначено як напрям доопрацювання. Сервіс розгорнуто й доступно в мережі Інтернет.

Ключові слова: вебсервіс, потокова передача медіаконтенту, пряма трансляція, захоплення застосунку, захоплення звуку процесу, WebRTC, SFU, mediasoup, Electron, Next.js, TypeScript, PostgreSQL.

ABSTRACT

[ПОТРЕБУЄ УТОЧНЕННЯ: Surname N. P.] Web service for streaming media content with the function of capturing individual applications : bachelor's qualification work in the specialty «[ПОТРЕБУЄ УТОЧНЕННЯ: specialty code and name in English]» / [ПОТРЕБУЄ УТОЧНЕННЯ: Name Patronymic Surname] ; supervisor [ПОТРЕБУЄ УТОЧНЕННЯ: Name Patronymic Surname]. – Odesa : Odesa Polytech. Nat. Univ., [ПОТРЕБУЄ УТОЧНЕННЯ: year]. – [ПОТРЕБУЄ УТОЧНЕННЯ: total number of pages after layout] p.

The qualification work contains the main text part on [ПОТРЕБУЄ УТОЧНЕННЯ: number after layout] pages, a list of used sources with 83 titles on [ПОТРЕБУЄ УТОЧНЕННЯ: number after layout] pages, appendices on [ПОТРЕБУЄ УТОЧНЕННЯ: number after layout] pages.

The purpose of the qualification work is to enable streaming of an individual application together with its own audio to viewers in a web browser with low delivery latency by developing the stream-share web service and a desktop application for capture.

The work analyses media delivery technologies, multiparty transmission topologies and the limitations of screen capture in the browser, compares six existing solutions and identifies the niche of the service. Functional and non-functional requirements are formulated, the development effort is estimated with the use case points method, and a schedule and a risk register are drawn up. The architecture consisting of a web application, a signaling server with an SFU media server and a desktop application, the signaling protocol, the data model and the server classes are designed.

As a result, a software system was created in which the Electron desktop application captures the selected window together with the audio of only the process that owns it and automatically switches the stream to new windows of the same application, the mediasoup-based server forwards the stream to viewers over WebRTC, and a viewer watches the stream in the browser, including as a guest without an

account. Static verification passed for all packages, 27 of 30 automated functional test cases produced the expected result, and the detected authorization defects of the signaling server are identified as a direction for further work. The service is deployed and available on the Internet.

Keywords: web service, media streaming, live streaming, application capture, process audio capture, WebRTC, SFU, mediasoup, Electron, Next.js, TypeScript, PostgreSQL.

ЗМІСТ

Перелік умовних позначень.....	8
Вступ.....	10
1 Аналіз предметної області та специфікація вимог.....	13
1.1 Аналіз предметної області.....	13
1.2 Аналіз наявних програмних рішень.....	17
1.3 Функціональні вимоги до програмної системи.....	19
1.4 Нефункціональні вимоги.....	26
Висновки до розділу 1.....	28
2 Планування програмного проєкту.....	29
2.1 Оцінювання тривалості розробки.....	29
2.2 Розробка плану виконання проєкту.....	32
2.3 Аналіз ризиків та планування реакцій.....	34
Висновки до розділу 2.....	35
3 Проєктування програмної системи.....	36
3.1 Архітектура програмної системи.....	36
3.2 Проєктування процесів у системі.....	39
3.3 Проєктування структури бази даних.....	48
3.4 Проєктування класів.....	49
3.5 Проєктування інтерфейсу користувача.....	52
Висновки до розділу 3.....	53
4 Програмна реалізація системи.....	54
4.1 Вибір інструментів розробки.....	54
4.2 Реалізація захоплення окремих застосунків.....	56
4.3 Реалізація потокової передачі на основі WebRTC та SFU.....	61
4.4 Реалізація автентифікації та авторизації.....	65
4.5 Реалізація рівня даних і програмного інтерфейсу сервера.....	67
4.6 Реалізація інтерфейсу користувача та розгортання.....	69
Висновки до розділу 4.....	70

5 Тестування програмної системи.....	71
5.1 Принципи вибору тестових прикладів.....	71
5.2 Функціональне тестування.....	73
5.3 Нефункціональне тестування.....	74
5.4 Виправлені дефекти та аналіз результатів.....	76
Висновки до розділу 5.....	78
6 Експлуатація програмної системи.....	79
6.1 Контрольний приклад застосування програмної системи.....	79
6.2 Експлуатаційні випробування програмної системи.....	82
Висновки до розділу 6.....	83
Загальні висновки.....	84
Список використаних джерел.....	86
Додаток А Лістинг програми.....	94
Додаток Б Структура таблиць бази даних.....	111
Додаток В Тестові приклади функціонального тестування.....	113

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

люд.-год – людино-година

ОС – операційна система

СКБД – система керування базами даних

ФВ – функціональна вимога

API – Application Programming Interface, прикладний програмний інтерфейс

CI – Continuous Integration, безперервна інтеграція

CSS – Cascading Style Sheets, каскадні таблиці стилів

DTLS – Datagram Transport Layer Security, протокол захисту транспортного рівня для датаграм

EF – Environmental Factor, коефіцієнт середовища

FK – Foreign Key, зовнішній ключ

HLS – HTTP Live Streaming, протокол потокової передачі через HTTP

HTTP – HyperText Transfer Protocol, протокол передачі гіпертексту

HTTPS – HyperText Transfer Protocol Secure, захищений протокол передачі гіпертексту

ICE – Interactive Connectivity Establishment, протокол встановлення інтерактивного з'єднання

ID – Identifier, ідентифікатор

IPC – Inter-Process Communication, міжпроцесна взаємодія

ISO/IEC – International Organization for Standardization / International Electrotechnical Commission, Міжнародна організація зі стандартизації / Міжнародна електротехнічна комісія

JSON – JavaScript Object Notation, текстовий формат обміну даними

JWT – JSON Web Token, вебтокен у форматі JSON

OAuth – Open Authorization, відкритий протокол авторизації

OBS – Open Broadcaster Software, програма для запису та трансляцій

ORM – Object-Relational Mapping, об'єктно-реляційне відображення

PF – Productivity Factor, коефіцієнт продуктивності

PID – Process Identifier, ідентифікатор процесу

PK – Primary Key, первинний ключ

PKCE – Proof Key for Code Exchange, ключ підтвердження для обміну кодом

REST – Representational State Transfer, передача стану представлення

RFC – Request for Comments, серія документів IETF

RTMP – Real-Time Messaging Protocol, протокол обміну повідомленнями в реальному часі

RTP – Real-time Transport Protocol, транспортний протокол реального часу

SFU – Selective Forwarding Unit, вузол селективного пересилання

SRTP – Secure Real-time Transport Protocol, захищений транспортний протокол реального часу

TCF – Technical Complexity Factor, технічний коефіцієнт

TCP – Transmission Control Protocol, протокол керування передачею

UAW – Unadjusted Actor Weight, нескоригована вага дійових осіб

UCP – Use Case Points, точки варіантів використання

UDP – User Datagram Protocol, протокол користувацьких датаграм

UML – Unified Modeling Language, уніфікована мова моделювання

UUCW – Unadjusted Use Case Weight, нескоригована вага варіантів використання

UUID – Universally Unique Identifier, універсальний унікальний ідентифікатор

WebRTC – Web Real-Time Communication, технологія обміну медіаданими в реальному часі у браузері

ВСТУП

Актуальність теми роботи. Пряма трансляція вмісту екрана стала звичним способом показати іншим людям роботу з програмою, ігровий процес чи навчальний матеріал. Більшість наявних рішень розв'язує це завдання одним із двох способів. Платформи трансляцій доставляють потік великій аудиторії через HTTP-технології із затримкою в секунди [1, 2] і покладають захоплення на сторонній кодувальник. Сервіси відеоконференцій та месенджери, навпаки, прив'язують глядача до зустрічі чи голосового каналу [3, 4]. Водночас трансляція саме окремого застосунку, а не всього екрана, має на практиці кілька особливостей: потрібно передавати лише звук цього застосунку, а не весь системний звук із повідомленнями та голосами інших програм, і не втрачати зображення, коли застосунок відкриває нове вікно.

Вебсторінка таких можливостей не має. Специфікація Screen Capture вимагає, щоб користувач щоразу сам обирав поверхню захоплення, забороняє застосунку обмежувати цей вибір, а звук розглядає як необов'язковий [5]. Звук окремого процесу доступний лише через системні інтерфейси операційної системи [6]. Отже, актуальним є створення вебсервісу, у якому перегляд відбувається у браузері з низькою затримкою, а захоплення окремого застосунку разом із його власним звуком виконує допоміжний настільний застосунок стрімера.

Технологічну основу для такого сервісу становить стек WebRTC, стандартизований W3C та IETF [7, 8]. Він забезпечує обов'язкове шифрування медіаданих [9] і відтворення потоку у браузері без додаткових програм. Для доставки одного потоку багатьом глядачам доцільна топологія з вузлом селективного пересилання (SFU), який не декодує медіадані, вирізняється надзвичайно низькою затримкою і масштабується на велику кількість користувачів [10]. Серед розглянутих у розділі 1 рішень жодне не поєднує захоплення окремого застосунку з його власним звуком, автоматичний перехід до

нових вікон застосунку та перегляд у браузері гостем без облікового запису. Саме це поєднання визначає новизну практичного завдання, яке розв'язується в роботі.

Мета і задачі роботи. Метою роботи є забезпечення трансляції окремого застосунку разом із його власним звуком глядачам у браузері з низькою затримкою доставки шляхом розроблення вебсервісу stream-share та настільного застосунку для захоплення. Кількісним показником досягнення мети є затримка доставки зображення від вікна застосунку стрімера до глядача, яка не повинна перевищувати [ПОТРЕБУЄ УТОЧНЕННЯ: граничне значення затримки, мс]; її перевірено в експлуатаційних випробуваннях (підрозділ 6.2).

Для досягнення мети необхідно розв'язати такі задачі:

- проаналізувати предметну область і наявні програмні рішення та сформулювати функціональні й нефункціональні вимоги до програмної системи;
- спланувати програмний проєкт: оцінити його трудомісткість, скласти календарний план і проаналізувати ризики;
- спроектувати архітектуру програмної системи, взаємодію її компонентів, структуру даних і користувацький інтерфейс;
- реалізувати вебзастосунок, сервер сигналізації з медіасервером SFU та настільний застосунок із захопленням окремих застосунків і їхнього звуку;
- протестувати програмну систему на відповідність сформульованим вимогам;
- описати порядок експлуатації програмної системи та провести її експлуатаційні випробування.

Об'єкт роботи. Об'єктом роботи є процес розробки вебсервісу для потокової передачі медіаконтенту.

Предмет роботи. Предметом роботи є вебсервіс stream-share з функцією захоплення окремих застосунків, його архітектура, програмні засоби захоплення зображення та звуку і механізми доставки медіапотoku глядачам через WebRTC.

Методи та засоби розроблення. Під час роботи застосовано порівняльний аналіз наявних програмних рішень за їхньою офіційною документацією, моделювання вимог і проєктних рішень мовою UML (діаграми варіантів використання, компонентів, розгортання, послідовності, діяльності, станів і класів), оцінювання трудомісткості методом точок варіантів використання (UCP), статичний аналіз і перевірку типів вихідного коду, функціональне тестування за

сценаріями використання та експлуатаційні випробування. Програмну систему розроблено мовою TypeScript із використанням фреймворку Next.js для вебзастосунку, сервера Fastify з бібліотекою mediasoup для сигналізації та пересилання медіапотоків, платформи Electron для настільного застосунку, СКБД PostgreSQL з бібліотекою Drizzle ORM для зберігання даних і Docker Compose для розгортання. Обґрунтування вибору цих засобів наведено в підрозділі 4.1.

Практична значущість. Розроблений вебсервіс розгорнуто й доступно за адресою <https://streamshare.space>. Стрімер може розпочати трансляцію безпосередньо у браузері або в настільному застосунку, який дозволяє обрати окреме вікно, передає звук лише процесу, якому це вікно належить, і автоматично переходить до нових вікон того самого застосунку. Глядач переглядає трансляцію у браузері без встановлення програм, зокрема як гість без облікового запису, а приватну трансляцію можна передати за посиланням. Вхід до сервісу виконується через обліковий запис Google, причому в настільному застосунку вибір облікового запису відбувається в системному браузері користувача. Інсталятори настільного застосунку для Windows, macOS і Linux публікуються в розділі випусків репозиторію GitHub; автоматичне оновлення підтримується у Windows і Linux.

Апробація матеріалів роботи. [ПОТРЕБУЄ УТОЧНЕННЯ: доповіді на конференціях або публікації за темою роботи з повними бібліографічними даними; якщо їх немає, абзац вилучається]

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА СПЕЦИФІКАЦІЯ ВИМОГ

1.1 Аналіз предметної області

1.1.1 Потокова передача медіаконтенту. Потокова передача передбачає доставлення аудіо- та відеоданих отримувачу в міру їх надходження, коли відтворення починається до завершення передавання. Розрізняють передачу за запитом, за якої матеріал існує заздалегідь, і пряму трансляцію, за якої контент формується в момент передавання. Для прямої трансляції визначальною є затримка доставки — проміжок часу між подією на стороні джерела та її відтворенням у глядача, від якого залежить можливість взаємодії в реальному часі. Затримка визначається насамперед технологією доставки (табл. 1.1).

Протокол HTTP Live Streaming (HLS) передає потік через HTTP як список відтворення з послідовних медіасегментів, і вже сама модель доставки закладає затримку в секунди [1, 11]; розширення Low-Latency HLS знижує її до рівня телевізійного мовлення [12], а YouTube у режимах низької та наднизької затримки обіцяє менше 10 с і менше 5 с відповідно [2]. Протокол RTMP поверх TCP нині слугує переважно для приймання потоку від кодувальника платформою [13], як-от на Twitch з OBS Studio [14]. Технологія WebRTC, натомість, призначена для двобічного обміну медіаданими в реальному часі безпосередньо з браузера [7].

Таблиця 1.1 – Технології доставки медіапотоків

Технологія	Транспорт	Роль у ланцюгу трансляції	Затримка (за джерелами)
RTMP	TCP	приймання потоку від кодувальника платформою [13, 14]	не нормується, залежить від подальшої доставки
HLS	HTTP, сегменти	доставка глядачам через HTTP-інфраструктуру [1]	секунди: старт не ближче трьох цільових тривалостей (рекомендовано 6 с) від кінця списку [1, 11]
Low-Latency	HTTP,	доставка глядачам зі	на рівні

Технологія	Транспорт	Роль у ланцюгу трансляції	Затримка (за джерелами)
HLS	часткові сегменти	зниженою затримкою [12]	телевізійного мовлення [12]
WebRTC	UDP (RTP/SRTP), ICE	двобічна передача між браузерами та серверами [7, 8]	реальний час; у топології SFU — «надзвичайно низька» [10]

З табл. 1.1 видно, що технології на основі HTTP оптимізовано для масштабованої доставки ціною затримки в секунди, тоді як WebRTC розрахована на інтерактивну взаємодію. Оскільки глядач спостерігає за роботою застосунку стрімера і реагує на неї, у роботі як транспорт обрано WebRTC.

1.1.2 Стек технологій WebRTC. WebRTC поєднує програмний інтерфейс браузера, стандартизований W3C [7], і набір протоколів IETF, огляд яких наведено в RFC 8825 [8]. Медіадані передаються у форматі RTP [15], причому реалізація захищеного профілю SRTP є обов’язковою [8]. Архітектура безпеки WebRTC забороняє передавання медіа через незашифрований RTP і вимагає пропонувати для кожного медіаканалу узгодження ключів DTLS-SRTP [9]. Встановлення з’єднання крізь трансляцію мережевих адрес (NAT) забезпечує протокол ICE, який перевіряє пари кандидатних адрес учасників [16]. Опис сеансу узгоджується за моделлю «пропозиція/відповідь» протоколу JSEP, який навмисно не визначає сигнальний протокол і залишає спосіб обміну описами застосунку [17].

Тому в stream-share сигналізацію реалізовано власним протоколом повідомлень поверх WebSocket (типи повідомлень — у пакеті packages/shared), а медіасервер оголошує в ICE-кандидатах публічну адресу зі змінної MEDIASOUP_ANNOUNCED_IP; протокол розглянуто в розділі 3.

1.1.3 Топології багатосторонньої передачі. Передавання від одного стрімера кільком глядачам можливе в трьох основних топологіях RFC 7667 [10] (рис. 1.1). У топології mesh відправник надсилає окрему копію потоку кожному отримувачу, тож вихідний канал стрімера зростає пропорційно кількості глядачів. Мікшер (MCU) декодує, змішує та повторно кодує потоки, що потребує значних обчислювальних ресурсів і погіршує якість [10]. Вузол селективного пересилання

(SFU) лише пересилає RTP-потоки без обробки сигналу, тому має надзвичайно низьку затримку і масштабується на велику кількість користувачів [10].

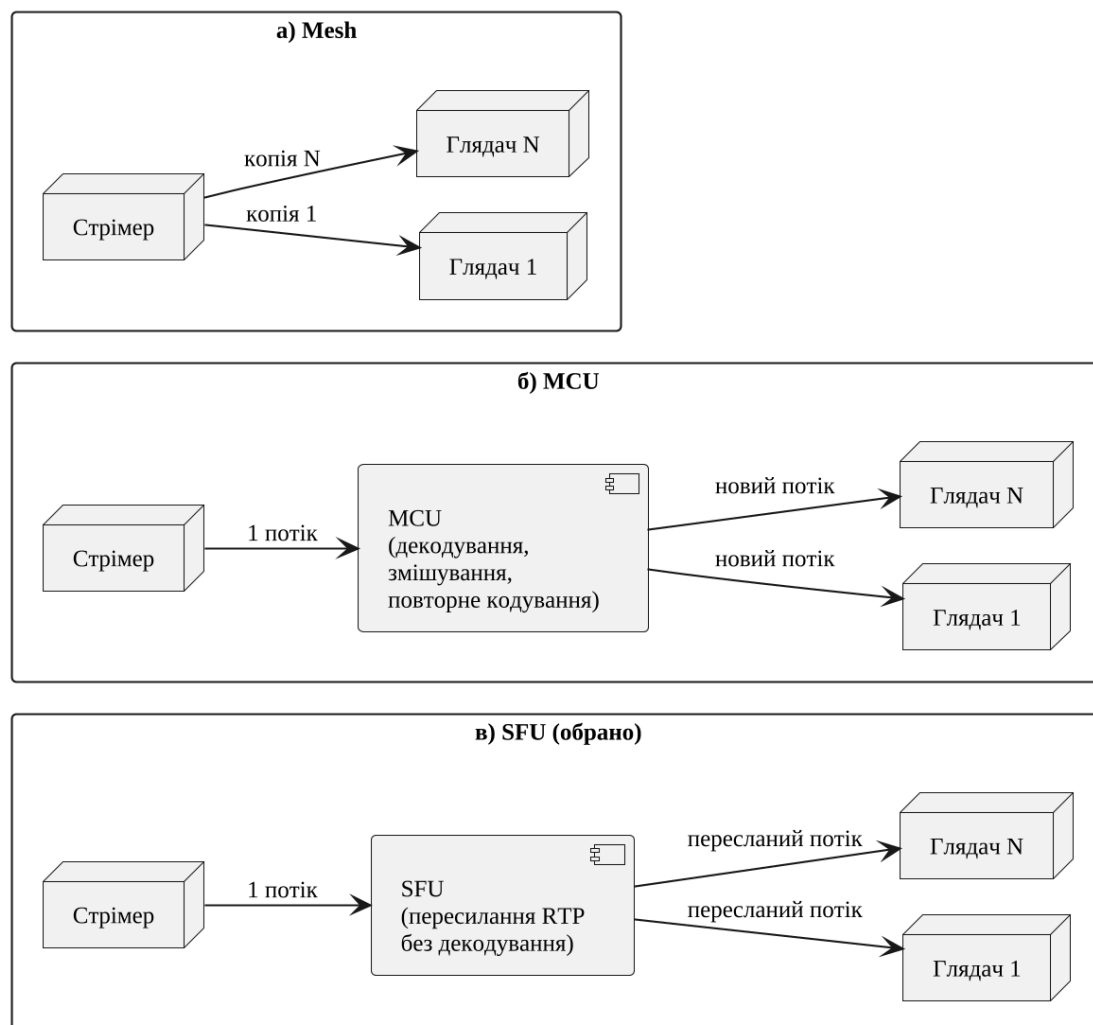


Рисунок 1.1 – Топології передачі медіапотоку від одного стрімера кільком глядачам

На рис. 1.1 показано, що в топології SFU стрімер відправляє рівно один потік незалежно від кількості глядачів, а сервер розмножує його без декодування. Відеоконференції та трансляції є двома найпоширенішими сценаріями застосування SFU [18]. Бібліотека mediasoup реалізує SFU як модуль Node.js і не нав'язує сигнальний протокол [19]; у stream-share вона працює на сигнальному сервері (apps/signaling/src/services/mediasoup.service.ts), а її вибір обґрунтовано в підрозділі 4.1.

1.1.4 Захоплення екрана, вікна та окремого застосунку. У браузері захоплення вмісту дисплея виконує метод `getDisplayMedia()` специфікації Screen

Capture, яка визначає три типи поверхонь: монітор, вікно окремого застосунку та вкладку браузера [5]. Специфікація вимагає, щоб користувач щоразу сам обирав поверхню з усіх доступних, і забороняє застосунку обмежувати цей вибір; джерела захоплення не перелічуються сторінці, оскільки це розкрило б забагато інформації про систему [5]. Виклик можливий лише у відповідь на дію користувача, а дозвіл не зберігається [20]. Звук необов'язковий: браузер може не повернути аудіодоріжку, а параметри `systemAudio` і `windowAudio` є лише підказками [5, 20]. Chrome пропонує звук вкладки, а системний звук, за поясненням розробників Chrome, містить і сторонні джерела, зокрема голоси учасників відеоконференції [21].

Отже, вебсторінка не може програмно обрати потрібне вікно, гарантовано отримати звук лише цього застосунку чи самостійно перейти до його нового, наприклад дочірнього, вікна, адже кожна нова поверхня потребує нового вибору користувачем.

Ці обмеження долаються в настільному застосунку. Модуль `desktopCapturer` платформи Electron перелічує джерела типів «екран» і «вікно», а обробник `setDisplayMediaRequestHandler` дозволяє самостійно відповісти на запит `getDisplayMedia()` обраним джерелом [22]. Його аудіорежим `loopback` захоплює лише весь системний звук і підтримується тільки у Windows [23]. Звук окремого процесу дають системні інтерфейси WASAPI Process Loopback (Windows 10 2004+), ScreenCaptureKit (macOS 13+) і PipeWire (Linux), які об'єднують нативний модуль `electron-native-screenshare`; він також визначає ідентифікатор процесу за дескриптором вікна [6].

Настільний застосунок `stream-share` використовує саме цей підхід: обране через `desktopCapturer` вікно передається вебсторінці, звук процесу-власника вікна захоплюється в окремому службовому процесі, а компонент `SourceFollower` перемикає джерело на нові вікна тієї самої родини процесів і повертається до початкового вікна після їх закриття (підрозділ 4.2).

1.2 Аналіз наявних програмних рішень

Для порівняння обрано рішення різних класів, що дозволяють показати іншим вміст екрана: Twitch разом із кодувальником OBS Studio, YouTube Live, Discord Go Live, Google Meet, Zoom і засіб віддаленого доступу Parsec. Аналіз ґрунтується лише на офіційній документації; не описані в ній характеристики позначено «не зазначено» (табл. 1.2).

1.2.1 Платформи трансляцій. OBS Studio — вільна програма для запису та прямих трансляцій для Windows, macOS і Linux [24]. Джерело «Window Capture» захоплює вміст конкретного вікна навіть тоді, коли воно перекрите іншими, а параметр «Window Match Priority» визначає, як шукати вікно, якщо попереднє не знайдено: за збігом заголовка або вікно того самого типу [25]. Починаючи з версії 28, OBS Studio може захоплювати звук окремих застосунків, але лише у Windows 10 версії 2004 і новіших та Windows 11 [26]. YouTube Live приймає трансляції з вебкамери, мобільного пристрою, ігрової консолі та програмного кодувальника, тож трансляція окремого застосунку також потребує стороннього кодувальника й верифікованого каналу [27]; відео з доступом за посиланням доступне без облікового запису Google [28].

1.2.2 Месенджери та сервіси відеоконференцій. Discord Go Live дозволяє транслювати окреме вікно застосунку або весь екран у голосовому каналі сервера чи у виклику. Звук застосунку захоплюється лише в настільних клієнтах для Windows і macOS, у браузері Chrome та в мобільних клієнтах, а в Linux недоступний. Глядач має приєднатися до голосового каналу, а без підписки Nitro діють обмеження якості й кількості глядачів [4]. Google Meet дозволяє показати вкладку, окреме вікно або весь екран; звук передається або разом із вкладкою Chrome, або як системний звук під час показу вікна чи екрана [3]. Zoom також дозволяє показати весь екран чи окремі вікна застосунків, проте функція спільного доступу до звуку передає весь звук, який відтворює комп'ютер; у вебверсії доступний звук вкладки лише в Chrome, а в сеансах Linux на Wayland можна показати тільки весь робочий стіл [29].

1.2.3 Засоби віддаленого доступу. Parsec — пропрієтарний застосунок для ігор і віддаленої роботи з власним протоколом BUD поверх UDP, який, за вимірюваннями розробника, додає в локальній мережі лише 7 мс затримки [30]. Хост надає доступ до всього екрана і підтверджує підключення запрошених користувачів [31]; хостом може бути лише комп'ютер з Windows або macOS [32]. Вебклієнт працює тільки в Chrome чи Chromium і поступається настільному за продуктивністю [33].

Таблиця 1.2 – Порівняння наявних програмних рішень

Критерій	Twitch + OBS Studio	YouTube Live	Discord Go Live	Google Meet	Zoom	Parsec
Клас рішення	платформа трансляцій + кодувальник	платформа трансляцій	месенджер	відеоконференції	відеоконференції	віддалений доступ
Захоплення окремого вікна	так [25]	через сторонній кодувальник [27]	так [4]	так [3]	так [29]	ні, весь екран [31]
Звук лише обраного застосунку	так, лише Windows 10 2004+ [26]	через сторонній кодувальник	так, крім Linux [4]	ні: вкладка або системний звук [3]	ні: весь звук комп'ютера [29]	не зазначено
Перехід до нових вікон застосунку	частково: пошук вікна за заголовком чи типом [25]	не зазначено	не зазначено	не зазначено	не зазначено	не застосовно
Умови перегляду	[ПОТРЕБУЄ УТОЧНЕННЯ: умови перегляду за офіційною довідкою Twitch]	за посиланням без облікового запису Google [28]	приєднання до голосового каналу або виклику [4]	участь у зустрічі [3]	участь у зустрічі [29]	підключення з підтвердженням хоста [31]
Задокументовано	не	без 4K у	50	до 10	не зазначено	хост лише

Критерій	Twitch + OBS Studio	YouTube Live	Discord Go Live	Google Meet	Zoom	Parsec
вані обмеження	зазначено	режимах низької затримки [2]	глядачів ; 720p/30 без Nitro [4]	одночасних показів [3]		Windows і macOS [32]
Затримка заданими розробника	не зазначено	< 10 с або < 5 с [2]	не зазначено	не зазначено	не зазначено	+7 мс у локальній мережі [30]

Табл. 1.2 показує, що жодне з рішень не поєднує всіх властивостей, потрібних для трансляції окремого застосунку. Платформи трансляцій покладають захоплення на сторонній кодувальник, у якому звук окремого застосунку доступний лише у Windows, а затримка доставки YouTube вимірюється секундами. Discord, Google Meet і Zoom прив'язують глядача до голосового каналу або зустрічі, і звук окремого застосунку з них передає лише Discord, причому не в Linux. Parsec розрахований на віддалене керування цілим комп'ютером. Автоматичний перехід до нових вікон застосунку не описано в жодній документації; найближчим є повторний пошук вікна в OBS Studio.

Отже, нішу розроблюваного вебсервісу визначає поєднання захоплення окремого застосунку з його власним звуком, слідування за дочірніми вікнами цього застосунку, доставки з низькою затримкою через WebRTC і SFU та перегляду у браузері без встановлення програм, зокрема гостем без облікового запису. Ці властивості покладено в основу функціональних вимог підрозділу 1.3.

1.3 Функціональні вимоги до програмної системи

1.3.1 Дійові особи та варіанти використання. Функціональні вимоги до вебсервісу сформульовано за результатами аналізу предметної області (підрозділ 1.1) та наявних рішень (підрозділ 1.2) і зафіксовано у вигляді моделі варіантів використання мови UML [34]. Перелік варіантів використання відповідає маршрутам вебзастосунку, правилам доступу до них і програмному інтерфейсу настільного застосунку. [ПОТРЕБУЄ УТОЧНЕННЯ: звірити перелік вимог із формулюваннями індивідуального завдання на кваліфікаційну роботу]

Система взаємодіє з двома категоріями людей і однією зовнішньою системою. Гість – відвідувач без облікового запису Google, який переглядає трансляції за посиланням у гостьовому сеансі, може завантажити настільний застосунок і увійти до системи. Користувач, що увійшов через Google, успадковує можливості гостя й додатково проводить трансляції та переглядає перелік активних публічних трансляцій; ролі стрімера та глядача за обліковим записом не закріплено. Зовнішньою дійовою особою є сервіс автентифікації Google, що підтверджує особу користувача за протоколом OAuth 2.0. Діаграму варіантів використання наведено на рис. 1.2.

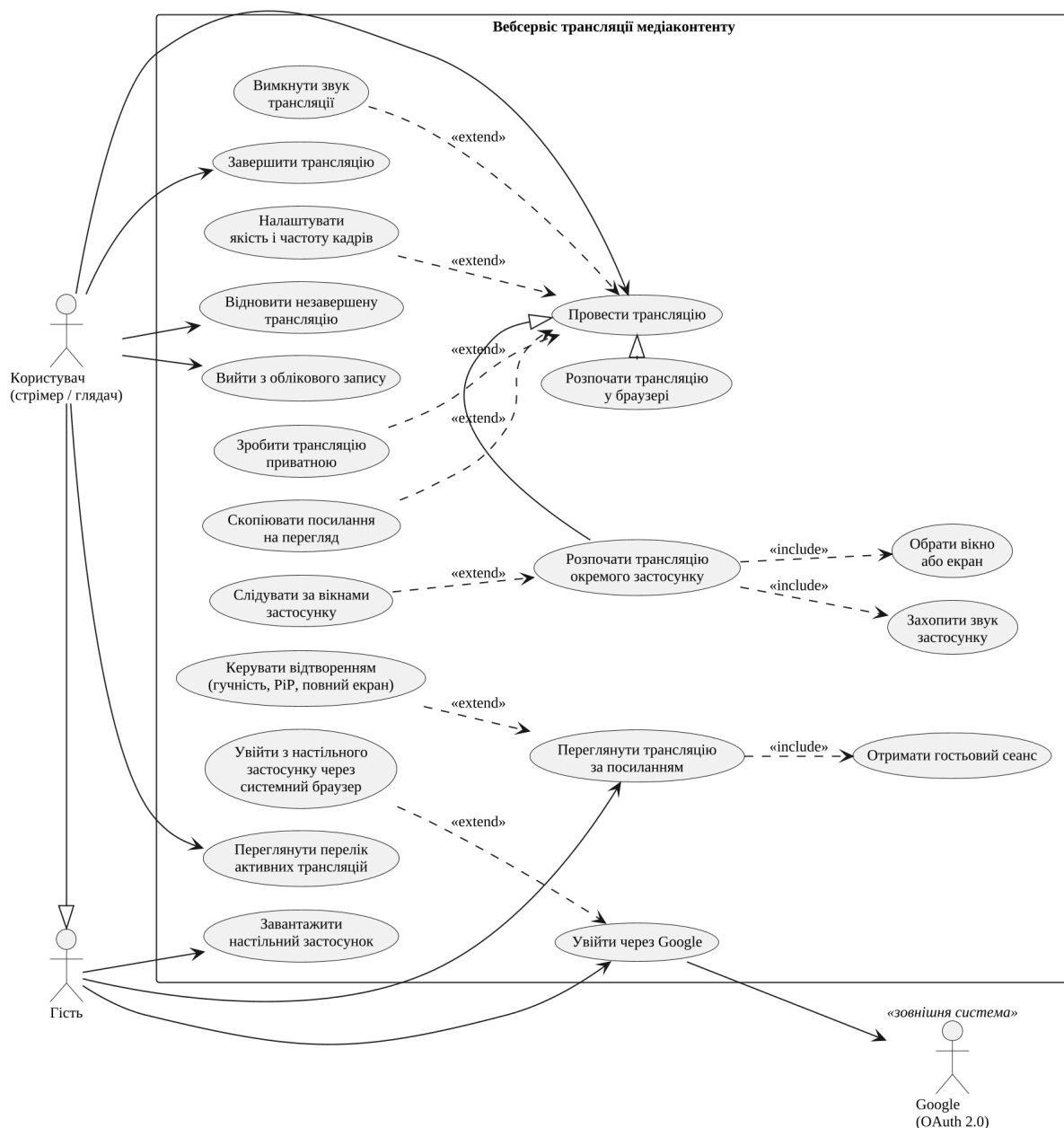


Рисунок 1.2 – Діаграма варіантів використання вебсервісу

Центральним на діаграмі є абстрактний варіант використання «Провести трансляцію», який має дві спеціалізації. Трансляція у браузері використовує стандартне захоплення екрана, вікна чи вкладки, а трансляція окремого застосунку доступна лише в настільному застосунку й включає вибір вікна та захоплення звуку цього застосунку; її розширює слідування за вікнами. Налаштування якості, приватність, копіювання посилання та вимкнення звуку розширюють будь-яку трансляцію. Вхід із настільного застосунку розширює вхід через Google: автентифікація відбувається в системному браузері, а сеанс передається в застосунок.

1.3.2 Перелік функціональних вимог. Узагальнений перелік функціональних вимог наведено в табл. 1.3. Кожну вимогу позначено кодом, на який посилаються подальші розділи, і модулем, у якому її реалізовано: вебзастосунком (apps/web), сервером сигналізації та пересилання медіапотоків (apps/signaling) або настільним застосунком (apps/desktop).

Таблиця 1.3 – Функціональні вимоги до програмної системи

Код	Вимога	Модуль
ФВ-01	Вхід через обліковий запис Google з вибором облікового запису під час кожного входу; вихід із системи	web
ФВ-02	Вхід у настільному застосунку через системний браузер із передаванням сеансу в застосунок за схемою РКСЕ	desktop, web
ФВ-03	Автоматичне створення гостьового сеансу з випадковим іменем та згенерованим аватаром під час відкриття сторінки перегляду; повторне використання гостьового запису в тому самому браузері	web
ФВ-04	Обмеження доступу: сторінки, крім головної, сторінки входу та сторінки перегляду, доступні лише користувачам, які увійшли через Google	web
ФВ-05	Перелік активних публічних трансляцій із мініатюрами та посторінковим переглядом по 12 елементів	web, signaling
ФВ-06	Трансляція у браузері з вибором екрана, вікна чи вкладки та звуку стандартними засобами браузера	web
ФВ-07	Перелік екранів і вікон із мініатюрами та піктограмами застосунків у настільному застосунку	desktop
ФВ-08	Захоплення звуку обраного застосунку; за неможливості захоплення – трансляція лише відео з повідомленням користувачу	desktop
ФВ-0	Режим слідування: автоматичне перемикання трансляції	desktop, web

Код	Вимога	Модуль
9	на нові вікна того самого застосунку та повернення до початкового вікна	
ФВ-10	Вибір якості (360р, 480р, 720р, 1080р, 1440р, вихідна) і частоти кадрів (5, 30, 60 кадр/с) до початку та під час трансляції	web
ФВ-11	Позначення трансляції як приватної: вона не потрапляє до переліку, але доступна за посиланням	web, signaling
ФВ-12	Формування та копіювання посилання на перегляд активної трансляції	web
ФВ-13	Вимкнення та ввімкнення звуку трансляції без її зупинення	web
ФВ-14	Перегляд трансляції з керуванням гучністю, режимом «картинка в картинці» та повноекранним режимом	web, signaling
ФВ-15	Завершення трансляції стрімером зі сповіщенням глядачів	web, signaling
ФВ-16	Збереження трансляції під час втрати з'єднання стрімера (до 5 спроб з інтервалом 6 с) і відновлення її з того самого або іншого вікна браузера	signaling, web
ФВ-17	Періодичне оновлення мініатюри трансляції (кожні 5 хв); завантаження мініатюри дозволено лише власнику трансляції	web, signaling
ФВ-18	Завантаження настільного застосунку для операційної системи відвідувача; перевірка й установлення оновлень для Windows і Linux	web, desktop
ФВ-19	Щотижневе видалення неактивних гостей записів, не пов'язаних із жодною трансляцією	signaling

Вимоги ФВ-01–ФВ-05 визначають доступ до системи, ФВ-06–ФВ-13 – підготовку й ведення трансляції, ФВ-14–ФВ-17 – доставлення потоку глядачам і життєвий цикл трансляції, ФВ-18 і ФВ-19 є службовими. Вимоги ФВ-07–ФВ-09 реалізують захоплення окремих застосунків і доступні лише в настільному застосунку, оскільки вебсторінка не може ні обрати звук окремого процесу, ні відстежувати появу його вікон.

1.3.3 Сценарії використання. Основні варіанти використання деталізовано у сценаріях 1.1–1.5; альтернативні кроки позначено номером основного кроку з літерою.

Сценарій 1.1 – Увійти через Google

Основна дійова особа: гість.

Мета/Результат: користувач отримує сеанс.

Передумови: відвідувач не увійшов до системи; має обліковий запис Google.

Тригер: відвідувач відкриває сторінку входу або сторінку, що потребує входу.

Основний успішний сценарій:

1. Відвідувач натискає кнопку входу через Google на сторінці входу.
2. Система перенаправляє його до Google із запитом на вибір облікового запису.
3. Відвідувач обирає обліковий запис і підтверджує вхід.
4. Система створює або оновлює запис користувача та встановлює сеансовий файл cookie.
5. Система перенаправляє користувача на головну сторінку з переліком активних трансляцій.

Альтернативні сценарії:

2а. Вхід розпочато в настільному застосунку: кроки 2–4 виконуються в системному браузері, після чого вебзастосунок передає в застосунок через локальну адресу 127.0.0.1 одноразовий код, який застосунок обмінює на сеанс.

2б. Користувач скасовує вхід у настільному застосунку або не завершує його протягом 5 хв: очікування припиняється, сеанс не створюється.

4а. Google відхиляє автентифікацію: система повертає відвідувача на сторінку входу.

Сценарій 1.2 – Розпочати трансляцію у браузері

Основна дійова особа: користувач (стрімер).

Мета/Результат: трансляція має статус «наживо», глядачі можуть її переглядати.

Передумови: користувач увійшов через Google.

Тригер: користувач відкриває сторінку трансляції.

Основний успішний сценарій:

1. Користувач обирає якість, частоту кадрів і за потреби вмикає приватний режим.
2. Користувач натискає кнопку вибору джерела; браузер показує стандартне вікно вибору екрана, вікна або вкладки.
3. Користувач обирає джерело; система показує його попередній перегляд.
4. Користувач натискає кнопку початку трансляції.

5. Система створює трансляцію, з'єднується із сервером і передає відео- та аудіопотоки.
6. Трансляція отримує статус «наживо»; система показує посилання на перегляд.

Альтернативні сценарії:

2а. Користувач закриває вікно вибору без вибору джерела: трансляцію не розпочато.

6а. Під час трансляції користувач змінює якість або частоту кадрів: система застосовує нові параметри кодування без перезапуску трансляції.

6б. Користувач припиняє показ засобами браузера: трансляцію завершено.

Сценарій 1.3 – Розпочати трансляцію окремого застосунку

Основна дійова особа: користувач (стрімер).

Мета/Результат: глядачі отримують зображення та звук лише обраного застосунку.

Передумови: користувач увійшов у настільному застосунку.

Тригер: користувач відкриває сторінку трансляції в настільному застосунку.

Основний успішний сценарій:

1. Користувач обирає вікно застосунку в переліку екранів і вікон із мініатюрами та за потреби вмикає режим слідування.
2. Система запускає захоплення звуку процесу, якому належить вікно, і показує попередній перегляд.
3. Користувач обирає параметри якості й починає трансляцію; система виконує кроки 5–6 сценарію 1.2.
4. Застосунок відкриває нове вікно; у режимі слідування система перемикає трансляцію на це вікно та показує його назву.
5. Нове вікно закривається; система повертає трансляцію до початкового вікна.

Альтернативні сценарії:

1а. Користувач обирає екран: режим слідування недоступний.

2а. Захопити звук застосунку не вдалося: система повідомляє, що звук недоступний, і транслює лише відео.

5а. Закрито вікно, з якого розпочато трансляцію: система повідомляє про закриття застосунку та завершує трансляцію.

Сценарій 1.4 – Переглянути трансляцію

Основна дійова особа: гість або користувач (глядач).

Мета/Результат: глядач бачить і чує трансляцію в реальному часі.

Передумови: глядач має посилання на трансляцію.

Тригер: глядач відкриває сторінку перегляду.

Основний успішний сценарій:

1. Система перевіряє, що трансляція існує та не завершена.
2. За відсутності сеансу система створює гостьовий сеанс або відновлює створений раніше в цьому браузері.
3. Система реєструє глядача, встановлює з'єднання з сервером і отримує відео- та аудіопотоки.
4. Система відтворює трансляцію; глядач керує гучністю, режимом «картинка в картинці» та повноекранним режимом.

Альтернативні сценарії:

1а. Трансляцію не знайдено або її завершено: система показує відповідне повідомлення.

4а. Браузер заблокував автоматичне відтворення: система показує кнопку запуску відтворення.

4б. Стрімер втратив з'єднання: сторінка перегляду залишається відкритою, а після відновлення трансляції система автоматично отримує новий потік.

Сценарій 1.5 – Завершити або відновити трансляцію

Основна дійова особа: користувач (стрімер).

Мета/Результат: трансляцію коректно завершено або відновлено після втрати з'єднання.

Передумови: трансляція користувача має статус «наживо».

Тригер: стрімер натискає кнопку завершення або з'єднання стрімера з сервером розривається.

Основний успішний сценарій:

1. Стрімер натискає кнопку завершення трансляції.
2. Сервер встановлює статус «завершено» з причиною «зупинено стрімером» і звільняє ресурси.
3. Сервер надсилає глядачам подію завершення, і сторінка перегляду показує відповідне повідомлення.

Альтернативні сценарії:

1а. З'єднання стрімера розірвано: сервер сповіщає глядачів і очікує повторного підключення до 5 спроб з інтервалом 6 с.

1б. Стрімер протягом цього часу відкриває сторінку трансляції й підтверджує відновлення: потік відновлюється для тих самих глядачів.

1в. Стрімер не підключився: сервер завершує трансляцію з причиною «тайм-аут» і виконує крок 3.

1г. Сервер перезапущено: під час запуску всі незавершені трансляції отримують статус «завершено».

Сценарії охоплюють усі варіанти використання рис. 1.2, крім тих, що зводяться до однієї дії (вихід, перегляд переліку, завантаження застосунку). Кроки 2 і 4 сценарію 1.3 відрізняють систему від аналогів (табл. 1.2). Сценарії є основою для оцінювання трудомісткості (розділ 2) і тестових прикладів (розділ 5).

1.4 Нефункціональні вимоги

Нефункціональні вимоги згруповано за характеристиками якості моделі ISO/IEC 25010 [35]. Числові значення в табл. 1.4 взято з конфігурації та вихідного коду системи; величини, які потребують вимірювань, позначено як такі, що потребують уточнення.

Таблиця 1.4 – Нефункціональні вимоги до програмної системи

Характеристика	Вимога	Значення в системі
Ефективність функціонування	Роздільна здатність і частота кадрів	захоплення до 3840×2160; вихідні профілі від 360р до вихідної роздільності; 5, 30 або 60 кадр/с, понад 1440р – не більше 30 кадр/с
Ефективність функціонування	Бітрейт відео	300 кбіт/с – 18 Мбіт/с залежно від роздільності й частоти кадрів; початковий вихідний бітрейт сервера не менше 4 Мбіт/с, пропорційно бюджету бітрейту трансляції; вхідний – до 21,6 Мбіт/с
Ефективність функціонування	Використання ресурсів сервера	процесів пересилання медіапотоків стільки, скільки ядер процесора; трансляція потрапляє до найменш

Характеристика	Вимога	Значення в системі
		завантаженого
Ефективність функціонування	Затримка доставки	[ПОТРЕБУЄ УТОЧНЕННЯ: цільове значення затримки та результати її вимірювання]
Сумісність	Кодеки	Opus для звуку; H.264 як основний відеокодек, VP8 – резервний
Сумісність	Платформи настільного застосунку	інсталятори для Windows (x64), macOS (універсальний) і Linux (AppImage); автоматичне оновлення – Windows і Linux
Надійність	Відновлення трансляції	очікування стрімера до 5 спроб з інтервалом 6 с; закриття незавершених трансляцій під час запуску сервера
Надійність	Обслуговування даних	щотижневе (понеділок, 00:00) видалення гостей записів, неактивних понад 7 діб
Безпека	Сеанси	JWT у файлі cookie з атрибутами HttpOnly і SameSite=Lax, у виробничому середовищі – Secure і префікс __Secure-; той самий сеанс перевіряє сервер WebSocket
Безпека	Вхід із настільного застосунку	PKCE (S256), перевірка параметра state, локальна адреса 127.0.0.1; запит дійсний 15 хв, одноразовий код – 120 с
Безпека	Ізоляція настільного застосунку	ізоляція контексту, відсутність Node.js у сторінці, інструменти розробника лише в режимі розробки; перевірка даних IPC схемами
Супроводжуваність	Конфігурація	перевірка змінних середовища схемами під час запуску; спільні пакети протоколу, схем даних і конфігурації

Профілі якості дозволяють передавати дрібний текст екранного вмісту у вихідній роздільності, а обмеження частоти кадрів понад 1440р стримує бітрейт, що зростає з кількістю пікселів у кадрі. Цільову затримку доставки в коді не задано; її має бути визначено як вимірюваний показник мети роботи та перевірено експериментально (розділ 6).

Вибір кодеків обґрунтовано в підрозділі 4.3. Автоматичне оновлення підключено лише для Windows і Linux: для macOS механізм оновлення electron-builder вимагає підпису застосунку [36], а збірку не підписано, тому нова версія встановлюється вручну з образу DMG. [ПОТРЕБУЄ УТОЧНЕННЯ: чи випробувано настільний застосунок на macOS і які операційні системи заявляти як підтримувані]

Вхід із настільного застосунку виконується в системному браузері із захистом РКСЕ, як рекомендують RFC 8252 і Google [37, 38, 39]. Зарезервовані в схемі бази даних поля пароля та журналу аудиту не використовуються (підрозділ 3.3), тому до вимог не включені.

Вимоги до взаємодії з користувачем є якісними: світла й темна теми за налаштуваннями системи, попередній перегляд джерела, спливні повідомлення про помилки, приховування елементів керування плеєра через 2,5 с бездіяльності; мова інтерфейсу – англійська.

Висновки до розділу 1

Аналіз предметної області показав доцільність технології WebRTC у топології SFU для інтерактивних трансляцій з одним джерелом і багатьма глядачами, а порівняння аналогів – відсутність рішення, що поєднує захоплення окремого застосунку з його звуком, слідування за вікнами та перегляд за посиланням без облікового запису.

На цій основі визначено дійових осіб і варіанти використання, сформульовано 19 функціональних вимог, деталізовано п'ять сценаріїв і згруповано нефункціональні вимоги за моделлю ISO/IEC 25010. Отримана специфікація є вихідними даними для планування проєкту, вибору інструментів розробки та проєктування архітектури.

2 ПЛАНУВАННЯ ПРОГРАМНОГО ПРОЄКТУ

2.1 Оцінювання тривалості розробки

Для оцінювання трудомісткості обрано метод точок варіантів використання (Use Case Points, UCP), запропонований Г. Карнером у 1993 р. [40]. Вихідними даними методу є модель варіантів використання з підрозділу 1.3 (рис. 1.2, сценарії 1.1–1.5). Вагові коефіцієнти та формули взято з опису методу [40]; значення впливу факторів є експертними судженнями автора, обґрунтованими нижче.

2.1.1 Вагові коефіцієнти дійових осіб. Дійових осіб (табл. 2.1) поділяють на прості (зовнішня система з програмним інтерфейсом), середні (система, що взаємодіє через протокол, наприклад HTTP, або сховище даних) і складні (людина, що працює через графічний або вебінтерфейс). Нескоригована вага дійових осіб обчислюється за формулою (2.1):

$$UAW = 1 \cdot a_s + 2 \cdot a_a + 3 \cdot a_c, \quad (2.1)$$

де a_s , a_a , a_c – кількість простих, середніх і складних дійових осіб відповідно.

Таблиця 2.1 – Класифікація дійових осіб

Дійова особа	Спосіб взаємодії із системою	Тип	Вага
Гість	вебінтерфейс у браузері	складний	3
Користувач (стрімер / глядач)	вебінтерфейс і настільний застосунок	складний	3
Google (OAuth 2.0)	протокол OAuth 2.0 поверх HTTPS	середній	2

Отже, $UAW = 2 \cdot 3 + 1 \cdot 2 = 8$.

2.1.2 Вагові коефіцієнти варіантів використання. Складність варіанта використання визначають кількістю транзакцій – пар «дія дійової особи – відповідь системи» в основному й альтернативних сценаріях: до трьох транзакцій – простий (вага 5), від чотирьох до семи – середній (10), понад сім – складний (15). Нескоригована вага варіантів використання обчислюється за формулою (2.2):

$$UUCW = 5 \cdot u_s + 10 \cdot u_a + 15 \cdot u_c, \quad (2.2)$$

де u_s , u_a , u_c – кількість простих, середніх і складних варіантів використання.

Для п'яти деталізованих варіантів використання транзакції підраховано за кроками сценаріїв підрозділу 1.3. Варіанти, що включаються в основні або розширюють їх (вибір вікна, звук застосунку, слідування, якість, приватність, гостьовий сеанс), окремо не враховано, бо їхні кроки вже містяться в сценаріях. Решту варіантів оцінено за вимогами табл. 1.3. Результат наведено в табл. 2.2.

Таблиця 2.2 – Класифікація варіантів використання

Варіант використання	Транзакцій	Тип	Вага
Увійти через Google (у т. ч. з настільного застосунку)	$5 + 3 = 8$	складний	15
Розпочати трансляцію у браузері	$6 + 3 = 9$	складний	15
Розпочати трансляцію окремого застосунку	$5 + 3 = 8$	складний	15
Переглянути трансляцію	$4 + 3 = 7$	середній	10
Завершити або відновити трансляцію	$3 + 4 = 7$	середній	10
Переглянути перелік активних трансляцій	3	простий	5
Керувати активною трансляцією (звук, посилання)	2	простий	5
Завантажити та оновити настільний застосунок	2	простий	5
Вийти з облікового запису	2	простий	5
Разом	—	—	85

Нескориговані точки варіантів використання становлять
 $UUCP = UAW + UUCW = 8 + 85 = 93$.

2.1.3 Технічна складність. Технічний коефіцієнт враховує 13 факторів (табл. 2.3), вплив кожного з яких оцінюють від 0 (відсутній) до 5 (суттєвий) і множать на вагу фактора. Коефіцієнт обчислюється за формулою (2.3):

$$TCF = 0,6 + 0,01 \cdot \sum_{i=1}^{13} w_i \cdot f_i, \quad (2.3)$$

де w_i – вага i -го технічного фактора; f_i – оцінка його впливу.

Таблиця 2.3 – Оцінка технічних факторів

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування
T1	Розподілена система	2	5	10	вебзастосунок, сервер SFU, настільний застосунок, СКБД
T2	Вимоги до швидкодії	2	5	10	доставлення відео в реальному часі
T3	Ефективність роботи користувача	1	3	3	запуск трансляції в кілька дій
T4	Складність обробки	1	4	4	параметри кодування, відстеження процесів
T5	Повторне використання	1	3	3	спільні пакети shared, env, db

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування
	коду				
T6	Простота встановлення	0,5	3	1,5	інсталятори, автооновлення
T7	Простота використання	0,5	4	2	гостьовий перегляд без реєстрації
T8	Переносимість	2	4	8	браузери; Windows, macOS, Linux
T9	Простота змін	1	3	3	шари контролерів, сервісів, репозиторіїв
T10	Паралельність	1	4	4	багато глядачів, кілька робочих процесів
T11	Безпека	1	3	3	сеанси JWT, PKCE, обмеження маршрутів
T12	Доступ для третіх сторін	1	1	1	відкритого API немає
T13	Потреба в навчанні	1	1	1	не потрібне
	Разом			53,5	

За формулою (2.3) $T C F = 0,6 + 0,01 \cdot 53,5 = 1,135$.

2.1.4 Фактори середовища. Коефіцієнт середовища враховує вісім факторів, що характеризують виконавця й умови розробки; два останні мають від’ємну вагу, оскільки збільшують трудомісткість. Коефіцієнт обчислюється за формулою (2.4):

$$E F = 1,4 - 0,03 \cdot \sum_{i=1}^8 v_i \cdot e_i, \quad (2.4)$$

де v_i – вага i -го фактора середовища; e_i – оцінка його впливу.

Проект виконувався одним розробником, тому фактори середовища оцінено для нього (табл. 2.4). [ПОТРЕБУЄ УТОЧНЕННЯ: підтвердити оцінки E2, E3 і E7 – попередній досвід розробника та зайнятість протягом проєкту]

Таблиця 2.4 – Оцінка факторів середовища

№	Фактор	Вага	Оцінка	Внесок
E1	Знайомство з процесом RUP	1,5	1	1,5
E2	Досвід розробки подібних застосунків	0,5	2	1
E3	Досвід об’єктно-орієнтованої розробки	1	4	4
E4	Кваліфікація аналітика	0,5	3	1,5
E5	Мотивація	1	5	5
E6	Стабільність вимог	2	3	6
E7	Часткова зайнятість	-1	3	-3
E8	Складність мови програмування	-1	2	-2
	Разом			14

Стабільність вимог оцінено як середню, оскільки за історією репозиторію функції настільного застосунку й режим слідування додано на завершальних

етапах. Мовою всіх модулів є TypeScript, тому складність мови оцінено як невисоку. За формулою (2.4) $EF = 1,4 - 0,03 \cdot 14 = 0,98$.

2.1.5 Скориговані точки та трудомісткість. Скориговані точки варіантів використання обчислюються за формулою (2.5):

$$UCP = (UAW + UUCW) \cdot TCF \cdot EF = 93 \cdot 1,135 \cdot 0,98 \approx 103,4. \quad (2.5)$$

Трудомісткість визначається добутком UCP на коефіцієнт продуктивності за формулою (2.6):

$$E = PF \cdot UCP, \quad (2.6)$$

де PF – кількість людино-годин на одну точку.

Карнер запропонував $PF = 20$ люд.-год, а Шнайдер і Вінтерс – вибір між 20, 28 і 36 люд.-год залежно від кількості несприятливих факторів середовища: оцінок, менших за 3, серед E1–E6 і більших за 3 серед E7–E8 [40]. У табл. 2.4 таких оцінок дві (E1, E2), що відповідає $PF = 20$. Отже, $E = 20 \cdot 103,4 \approx 2069$ люд.-год, або близько 259 людино-днів по 8 год. Якщо часткову зайнятість (E7) буде оцінено вище за 3, PF зросте до 28, а трудомісткість – до 2896 люд.-год.

Фактичний проєкт тривав 122 календарні дні (підрозділ 2.2), що для одного розробника суттєво менше за розрахункову трудомісткість. Розбіжність пояснюється тим, що UCP вимірює функціональний розмір, а значну частину функцій надають готові бібліотеки (mediasoup, Auth.js, Next.js, Electron, electron-native-screenshare): уся кодова база містить близько 9 000 рядків TypeScript у 169 файлах. Крім того, фіксований коефіцієнт продуктивності рекомендують калібрувати за історичними даними [40], яких для цього проєкту немає. [ПОТРЕБУЄ УТОЧНЕННЯ: фактичні трудовитрати в годинах, якщо їх обліковували]

2.2 Розробка плану виконання проєкту

План виконання відновлено за історією Git: гілка main містить 43 коміти одного автора, перший – 27.05.2026 («initialize basic project structure»), останній –

25.09.2026. Коміти згруповано в етапи за змістом; кожен етап закінчується датою свого останнього коміту й починається наступного дня після попереднього. Таке розбиття є реконструкцією: проміжна робота між комітами в репозиторії не фіксується. [ПОТРЕБУЄ УТОЧНЕННЯ: фактичні дати етапів і наявність роботи до 27.05.2026 – аналізу предметної області та вибору технологій] Перелік робіт наведено в табл. 2.5.

Таблиця 2.5 – Перелік робіт проекту

№	Етап	Період	Дн.	Поперед-ник	Комітів
1	Каркас монорепозіторію (web, signaling, desktop, shared)	27.05	1	–	1
2	Базова трансляція через SFU, вхід через Google, база даних	28.05–13.07	47	1	1
3	Оптимізація конфігурації TypeScript, аудит залежностей	14.07–21.07	8	2	2
4	Гостьовий сеанс, приватні трансляції, вимкнення звуку, сповіщення, мініатюри, очищення гостьових записів	22.07–12.08	22	3	5
5	Розгортання: Docker-образи, Caddy, валідація змінних середовища	13.08–29.08	17	4	11
6	Доопрацювання інтерфейсу	30.08–03.09	5	5	2
7	Настільний застосунок: звук застосунку, вибір джерела, пакування, випуск у CI	04.09–14.09	11	5, 6	9
8	Керування трансляцією, автооновлення	15.09–17.09	3	7	1
9	Вхід через системний браузер, виправлення інтерфейсу	18.09–19.09	2	8	8
10	Апаратне кодування H.264, узгодження бітрейту	20.09	1	9	2
11	Режим слідування за дочірніми вікнами	21.09–25.09	5	10	1

Етапи виконувалися послідовно, оскільки виконавець був один. Найтриваліший етап 2 закінчився одним комітом на 77 файлів, у якому з'явилися сервіси сервера сигналізації й пересилання медіапотоків, пакет бази даних і вхід через Google. Етап 7 залежить також від етапу 5, оскільки настільний застосунок завантажує розгорнутий вебзастосунок. Календарний графік робіт наведено на рис. 2.1.

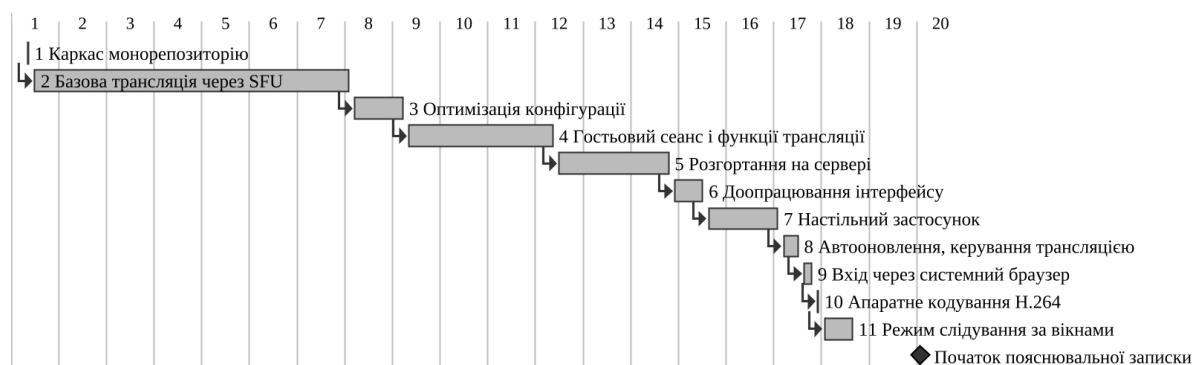


Рисунок 2.1 – Діаграма Ганта виконання проекту (тижні від 25.05.2026)

Шкала діаграми поділена на календарні тижні, перший з яких починається 25.05.2026. Програмну частину розроблено за 18 тижнів, написання пояснювальної записки розпочато на 20-му тижні (05.10.2026). Діаграма показує, що майже 40 % тривалості проекту припало на базову потокову передачу, а функції захоплення окремих застосунків (етапи 7–11) реалізовано протягом останніх чотирьох тижнів. [ПОТРЕБУЄ УТОЧНЕННЯ: дата завершення роботи й етапи календарного плану із завдання на кваліфікаційну роботу]

2.3 Аналіз ризиків та планування реакцій

Ризики виявлено за документацією, конфігурацією збирання та історією змін репозиторію. Імовірність і вплив оцінено якісно за трирівневою шкалою; для кожного ризику визначено стратегію реагування – уникнення, пом'якшення або прийняття (табл. 2.6).

Таблиця 2.6 – Реєстр ризиків проекту

№	Ризик	Імовір-ність	Вплив	Реагування
R 1	Медіаз'єднання не встановлюється через NAT або неправильну публічну адресу сервера	середня	високий	пом'якшення: обов'язкова змінна <code>MEDIASOUP_ANNOUNCED_IP</code> , без якої сервер не запускається; інструкція в README
R 2	Google відхиляє вхід із вбудованого вікна Electron	висока	високий	уникнення: вхід у системному браузері з РКСЕ і локальною адресою повернення
R 3	Нативний модуль захоплення звуку не збирається для версії Electron або ОС	середня	високий	пом'якшення: перекомпіляція під час встановлення, збирання в CI для кожної ОС; трансляція лише відео
R	Немає апаратного	середня	середній	пом'якшення: порядок кодеків

№	Ризик	Імовір-ність	Вплив	Реагування
4	кодувальника H.264, режими 1080p60 і вище деградують			H.264 Baseline, резервний VP8, вибір пріоритету частоти кадрів чи роздільності
R 5	Інсталювальники без цифрового підпису: попередження ОС, немає автооновлення на macOS	висока	середній	прийняття: необов'язковий крок підпису в CI, ручне оновлення на macOS
R 6	Перевантаження одного сервера кількістю глядачів	середня	середній	пом'якшення: кілька робочих процесів mediasoup, вибір найменш завантаженого
R 7	Регресії через відсутність автоматизованих тестів	висока	середній	пом'якшення: перевірки lint, typescheck, build і ручне тестування
R 8	Зміна й розширення вимог під час розробки	середня	середній	пом'якшення: поетапне додавання функцій, модульна структура
R 9	Зрив графіка через єдиного виконавця	середня	високий	пом'якшення: пріоритет основних варіантів використання, короткі етапи

Ризик R1 підтверджено описом розгортання: з приватною адресою в кандидатах ICE з'єднання неможливе, тому шаблон конфігурації залишає змінну порожньою. Ризик R2 зумовлений політикою Google, яка з 30.09.2021 блокує запити авторизації OAuth 2.0 з вбудованих переглядачів [41], і рекомендацією RFC 8252 використовувати для нативних застосунків зовнішній браузер [37]; його усунуто комітом від 19.09.2026. Ризики R3–R5 відображено в конфігурації збирання та коді маршрутизатора.

Висновки до розділу 2

Трудомісткість розробки оцінено методом точок варіантів використання: за трьох дійових осіб і дев'яти варіантів використання $UCP \approx 103,4$, а за коефіцієнта продуктивності 20 люд.-год трудомісткість становить близько 2069 люд.-год. Перевищення фактичної тривалості пояснюється готовими бібліотеками й некаліброваним коефіцієнтом продуктивності.

План виконання відновлено за історією Git як 11 етапів з 27.05.2026 до 25.09.2026. Для ключових технічних ризиків реєстру – медіаз'єднання, входу через Google і збирання нативного модуля – заходи реагування реалізовано в коді.

3 ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

3.1 Архітектура програмної системи

3.1.1 Склад і розподіл відповідальності. Програмну систему організовано як монорепозиторій rnrn [42], у якому три виконувані модулі (apps/*) спираються на три спільні пакети (packages/*). Модулі та зв'язки між ними наведено на діаграмі компонентів (рис. 3.1), побудованій у нотації UML [34]. Вихідний код ключових модулів наведено в додатку А.

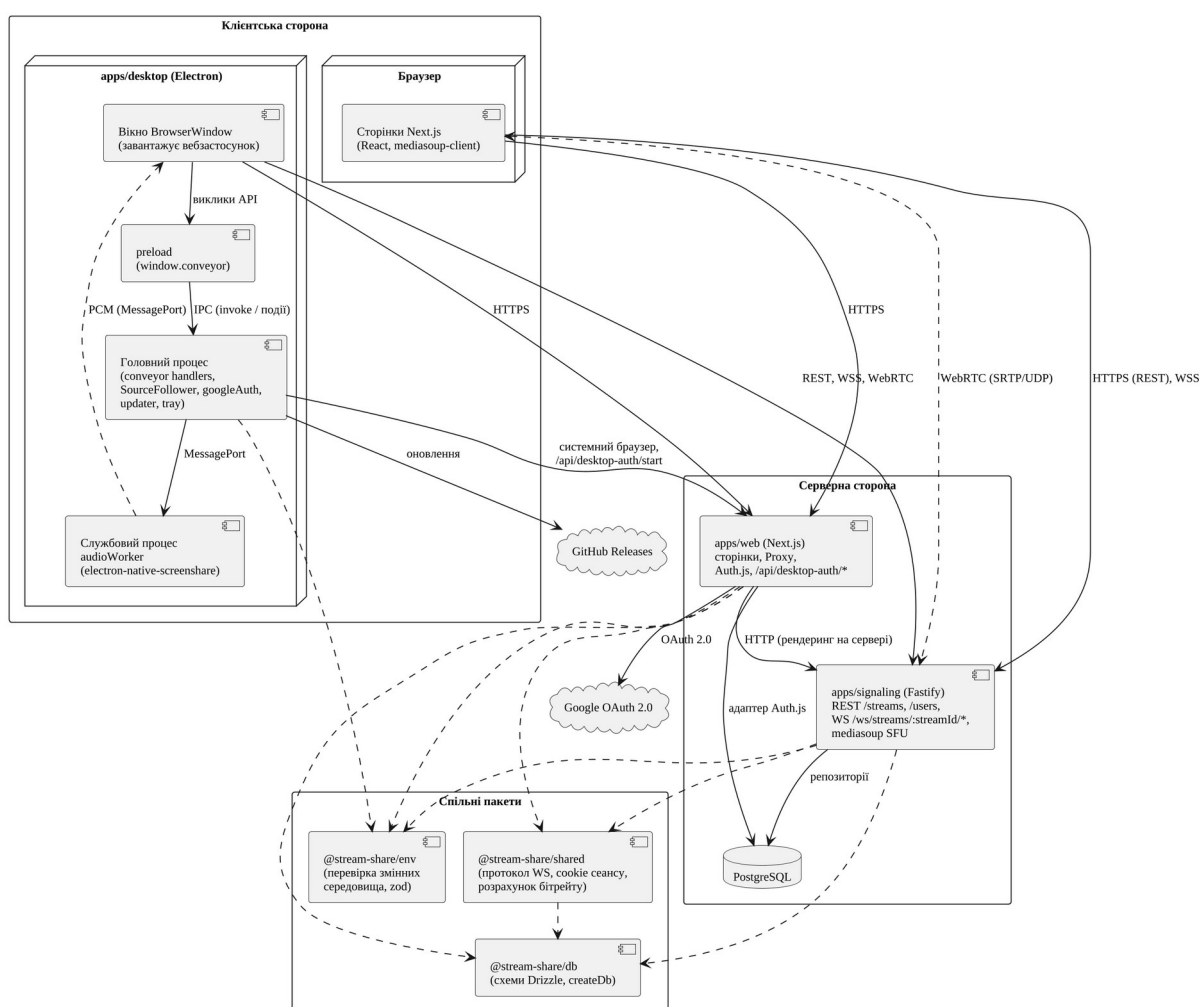


Рисунок 3.1 – Діаграма компонентів програмної системи

Вебзастосунок apps/web (Next.js) формує сторінки, виконує автентифікацію через Auth.js і під час серверного рендерингу отримує дані про трансляції від сервера сигналізації внутрішнім HTTP-запитом. Сервер сигналізації apps/signaling (Fastify) надає REST-інтерфейс і WebSocket-канали [43] для сигнальних

повідомлень, а також пересилає медіапотоки як SFU на основі mediasoup [44]: потік стрімера надходить на сервер один раз і розсилається кожному глядачеві без перекодування.

Настільний застосунок `apps/desktop` (Electron) не дублює інтерфейс: вікно `BrowserWindow` завантажує розгорнутий вебзастосунок, а головний процес [45] надає сторінці через об'єкт `window.conveyor` лише недоступне браузеру – перелік вікон з ідентифікаторами процесів, захоплення звуку застосунку в утилітарному процесі `audioWorker`, слідування за вікнами та вхід через системний браузер.

Пакет `@stream-share/db` містить схеми таблиць Drizzle ORM [46], `@stream-share/shared` – типи повідомлень протоколу, ім'я cookie сеансу та розрахунок бітрейту, `@stream-share/env` – перевірку змінних середовища за схемами `zod` [47]. Тому зміна протоколу чи схеми даних перевіряється компілятором в усіх модулях. Зовнішні системи – Google OAuth 2.0 для підтвердження особи та GitHub Releases для інсталяторів і автоматичного оновлення настільного застосунку [36].

3.1.2 Розгортання. Конфігурацію виробничого середовища задано файлом `infra/docker-compose.prod.yml` [48]. Діаграму розгортання наведено на рис. 3.2.

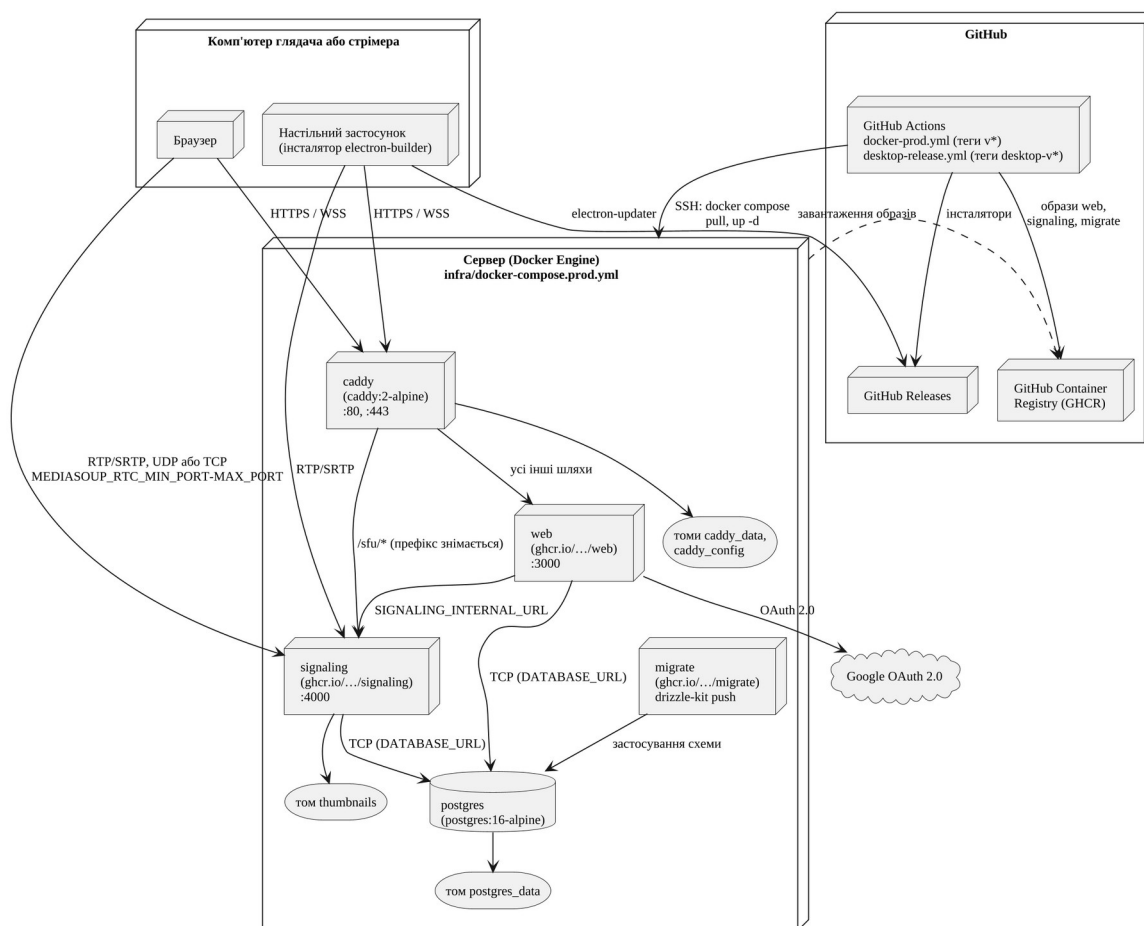


Рисунок 3.2 – Діаграма розгортання програмної системи

На сервері працюють п'ять сервісів:

- caddy (порти 80, 443) автоматично отримує TLS-сертифікат [49] і спрямовує запити `/sfu/*` без префікса до сервера сигналізації, а решту – до web (порт 3000);
- web (внутрішній порт 3000) запускає вебзастосунок Next.js з образу в GitHub Container Registry; змінні `AUTH_URL` (публічна адреса `https://${PUBLIC_HOST}`) і `AUTH_TRUST_HOST` налаштовують Auth.js на роботу за зворотним проксі;
- signaling (порт 4000) додатково публікує діапазон портів UDP і TCP `MEDIASOUP_RTC_MIN_PORT-MEDIASOUP_RTC_MAX_PORT`, через який медіадані йдуть до клієнтів в обхід Caddy;
- postgres (образ `postgres:16-alpine`) зберігає дані [50];
- migrate одноразово застосовує схему командою `drizzle-kit push`.

Мініатюри, дані СУБД і сертифікати зберігаються в томах; доставку змін через GitHub Actions розглянуто в пункті 4.6.3.

3.2 Проектування процесів у системі

3.2.1 Протокол сигналізації. Клієнти та сервер обмінюються JSON-повідомленнями типів req (запит з method, params і requestId), res (відповідь з тим самим requestId, що містить result або помилку {code, msg}) та event (подія сервера). Методи й події з пакета @stream-share/shared наведено в табл. 3.1; оголошений метод setPreferredLayer не обробляє ні сервер, ні клієнт, тому до таблиці він не увійшов.

Таблиця 3.1 – Методи та події протоколу сигналізації

Метод / подія	Канал	Напрямок	Параметри → результат
getRtpCapabilities	broadcast	клієнт → сервер	– → можливості RTP маршрутизатора
createTransport	обидва	клієнт → сервер	direction (send / recv) → id, параметри ICE і DTLS, кандидати ICE
connectTransport	обидва	клієнт → сервер	параметри DTLS клієнта → –
produce	broadcast	клієнт → сервер	kind, rtpParameters, last, maxBitrate → producerId
endStream	broadcast	клієнт → сервер	– → –
joinStream	watch	клієнт → сервер	– → producerIds, можливості RTP
consume	watch	клієнт → сервер	можливості RTP клієнта, producerId → consumerId, kind, rtpParameters
streamEnd	watch	сервер → клієнт	трансляцію завершено
streamerDisconnect	watch	сервер → клієнт	з'єднання зі стрімером втрачено
streamerReconnected	watch	сервер → клієнт	новий producerId після повернення стрімера

Методи відповідають етапам встановлення WebRTC-сеансу [7, 8]: для кожного напрямку створюється окремий транспорт, параметри DTLS клієнт надсилає, коли mediasoup-client [51] генерує подію connect, після чого медіадані йдуть протоколом SRTP через пару кандидатів, обрану ICE [9, 16].

3.2.2 Публікація трансляції. Діаграму послідовності публікації трансляції наведено на рис. 3.3. Вона деталізує крок 5 сценарію 1.2, який використовує й сценарій 1.3.

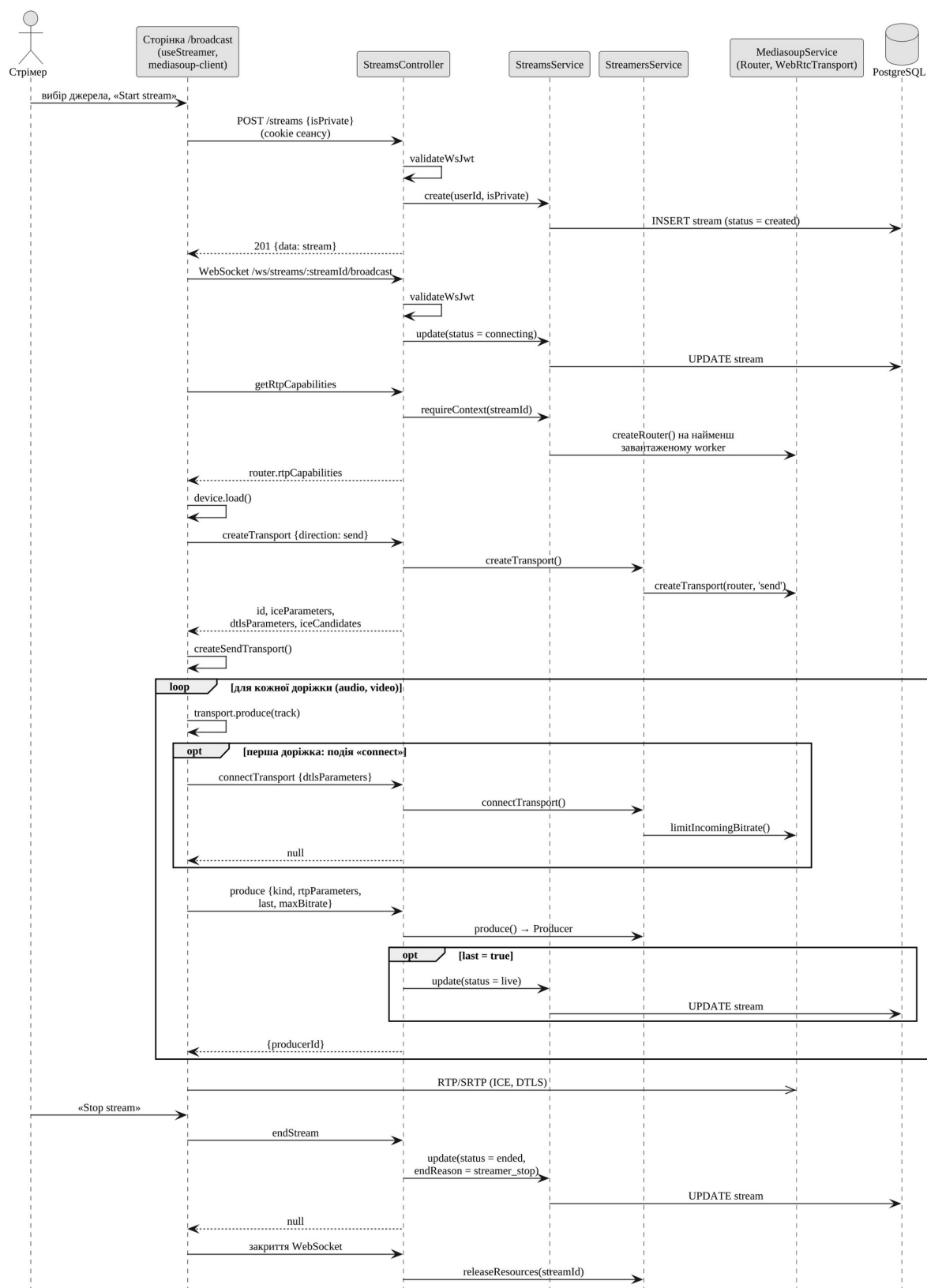


Рисунок 3.3 – Діаграма послідовності публікації трансляції

Сторінка `/broadcast` (хук `useStreamer`) створює запис трансляції запитом `POST /streams` і відкриває з'єднання `/ws/streams/:streamId/broadcast`. Обидва звернення проходять перевірку `validateJwt`, яка розшифровує cookie сеансу `Auth.js` ключем `AUTH_SECRET`, тож сервер сигналізації перевіряє сеанс без звернення до вебзастосунку. Підключення переводить трансляцію в стан `connecting`. Запит `getRtpCapabilities` створює маршрутизатор `mediasoup` на найменш завантаженому процесі-обробнику (`worker`). Далі клієнт викликає `produce` для кожної доріжки, і прапорець `last` останньої доріжки переводить трансляцію в стан `live`. Запит `endStream` фіксує завершення з причиною `streamer_stop`, а закриття сокета звільняє ресурси `mediasoup`.

Контекст активної трансляції (маршрутизатор, транспорти, продюсери й споживачі, таймер очікування повернення) зберігається лише в пам'яті, у реєстрі `StreamsService`; у базі даних – тільки статус і час трансляції.

3.2.3 Перегляд трансляції. Процес перегляду, що відповідає сценарію 1.4, наведено на рис. 3.4.

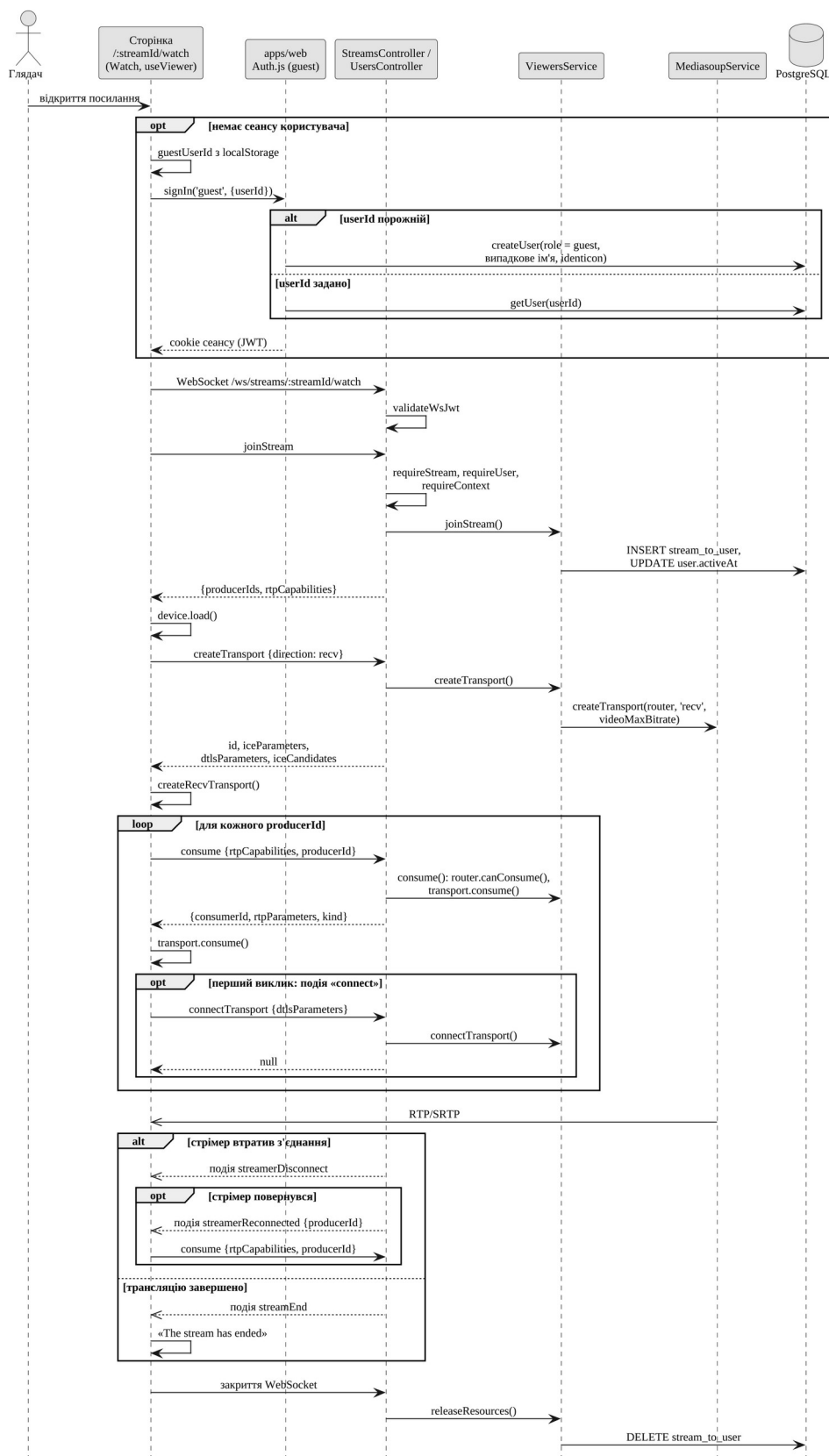


Рисунок 3.4 – Діаграма послідовності перегляду трансляції

Шлях `/<streamId>/watch` виключено з переадресації на сторінку входу у `proxy.ts` [52]. Відвідувачу без сеансу сторінка `watch` виконує вхід через провайдер `guest` (ФВ-03), який повторно використовує запис гостя, збережений у

localStorage, або створює новий з роллю guest, випадковим іменем та аватаром. Запит joinStream додає глядача до stream_to_user і повертає ідентифікатори продюсерів; клієнт створює приймальний транспорт, початковий бітрейт якого сервер обчислює з максимального бітрейту відео трансляції, і виконує consume для кожного продюсера. Подію streamerReconnected клієнт обробляє споживанням нового продюсера, а streamEnd – заміною плеєра повідомленням про завершення; подію streamerDisconnect сервер надсилає, але клієнт її не обробляє. Закриття сокета видаляє запис глядача зі stream_to_user.

3.2.4 Вхід у настільному застосунку. Google блокує OAuth-вхід у вбудованих вебпереглядачах [41], тому нативним застосункам рекомендовано використовувати системний браузер і локальну адресу зворотного виклику [37, 39]. Взаємодію, що реалізує ФВ-02, наведено на рис. 3.5.

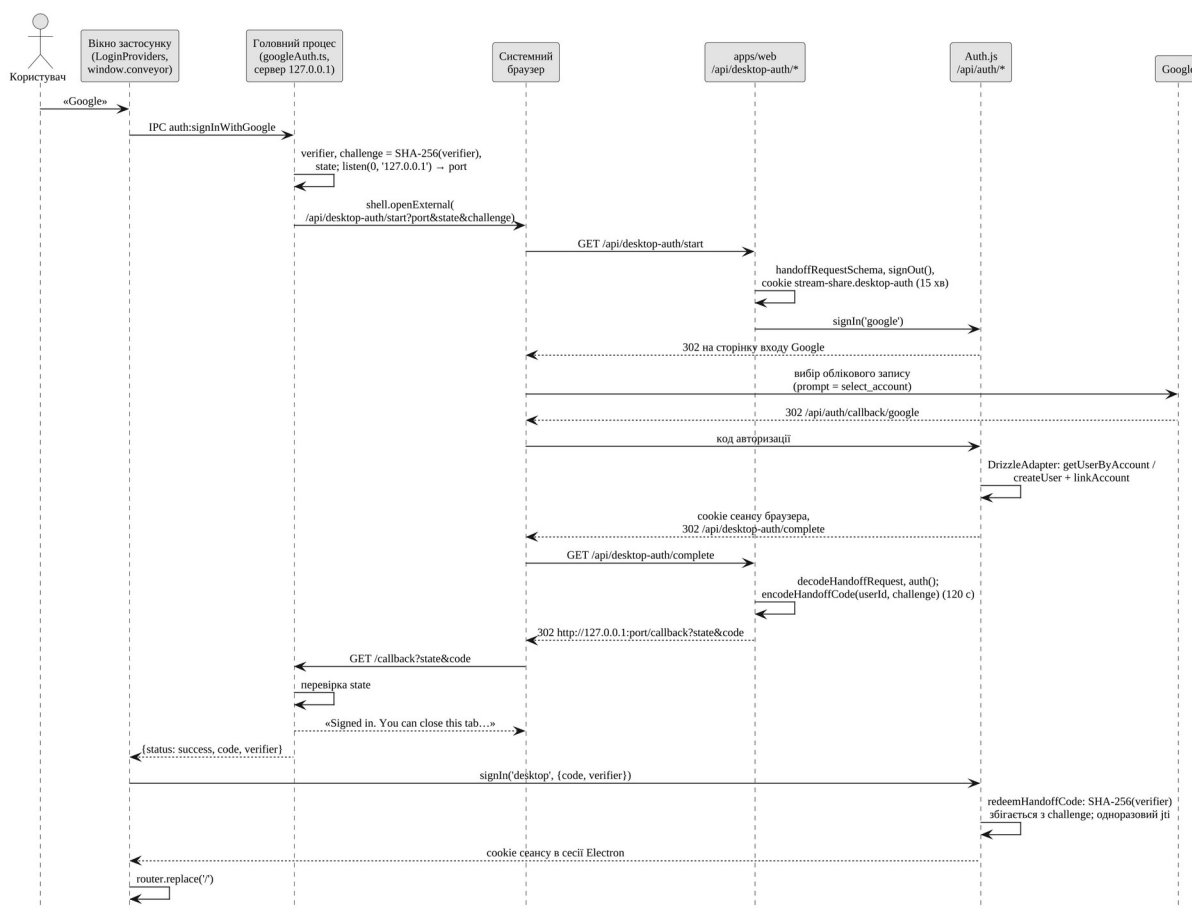


Рисунок 3.5 – Діаграма послідовності входу в настільному застосунку через системний браузер

Головний процес генерує verifier, обчислює $\text{challenge} = \text{SHA-256}(\text{verifier})$ за схемою PKCE [38], запускає HTTP-сервер на 127.0.0.1 з вільним портом і відкриває в системному браузері `/api/desktop-auth/start` з параметрами port, state і challenge. Вебзастосунок зберігає їх у короткочасному cookie та перенаправляє до Google. Після входу маршрут `/complete` перевіряє, що сеанс не гостьовий, видає пов'язаний з challenge одноразовий код (дійсний 120 с) і перенаправляє браузер на локальний сервер. Головний процес звіряє state і передає код і verifier сторінці, яка обмінює їх на сеанс через провайдер desktop. Перехоплений код без verifier непридатний, а повторне використання блокується реєстром ідентифікаторів jti.

3.2.5 Слідування за застосунком. Вимога ФВ-09 передбачає перехід трансляції на нові вікна застосунку, зокрема відкриті його дочірніми процесами, і повернення до початкового вікна. Алгоритм класу SourceFollower наведено на діаграмі діяльності (рис. 3.6).

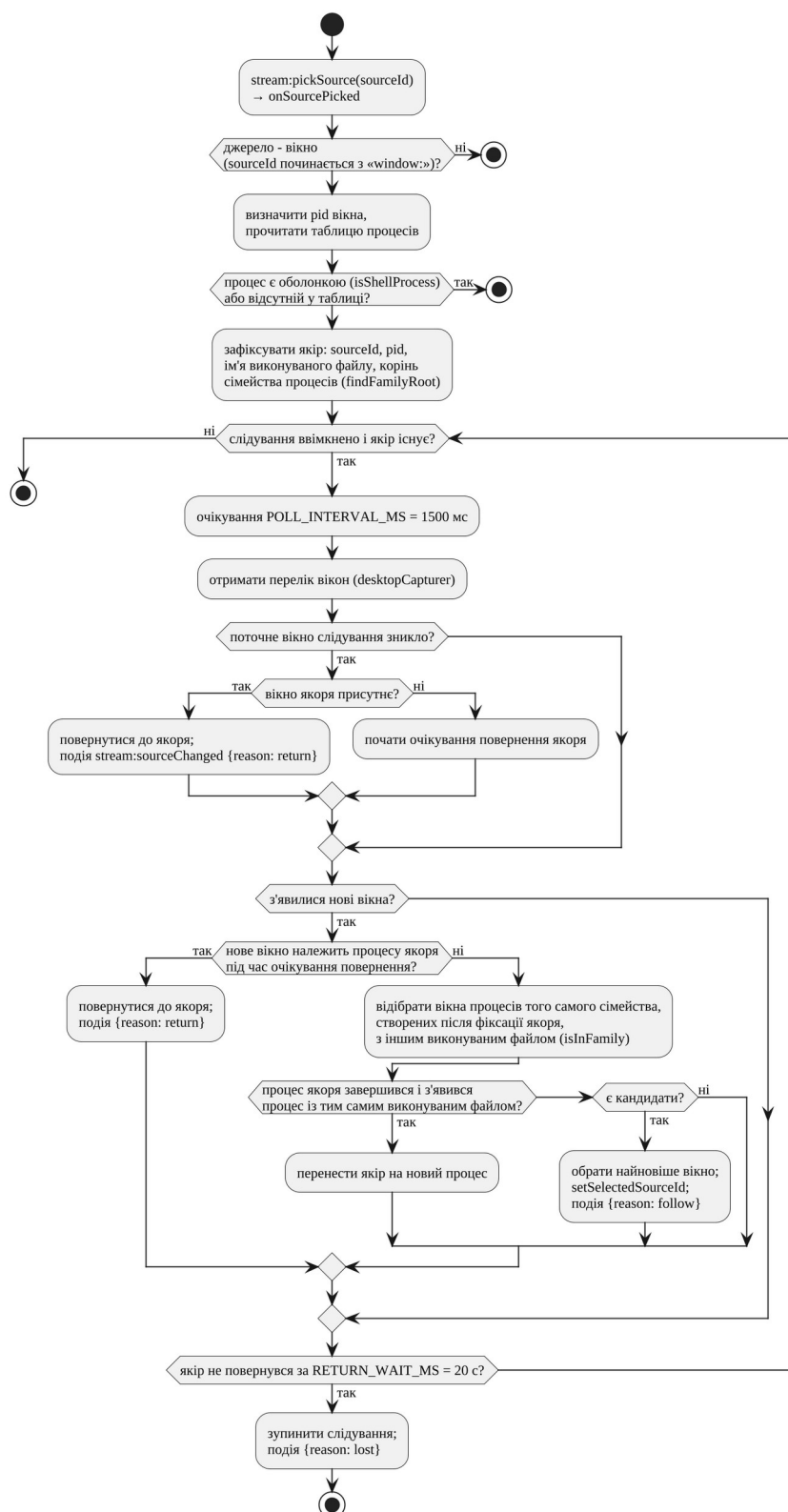


Рисунок 3.6 – Діаграма діяльності режиму слідкування за застосунком

Під час вибору вікна фіксується «якорь»: ідентифікатор джерела, PID, ім'я виконуваного файлу та корінь сімейства процесів (вікна процесів-оболонки якорем не стають). Кожні 1500 мс клас порівнює перелік вікон від desktopCapturer

[22] з попереднім; кандидатом є вікно процесу того самого сімейства, створеного після фіксації якоря, з іншим виконуваним файлом, а з кількох обирається найновіше. Коли вікно слідування зникає, трансляція повертається до якоря; якщо зникло й воно, клас чекає до 20 с і вимикає слідування з подією `lost`. Кожне перемикання сторінки отримує як подію `stream:sourceChanged` і замінює доріжку без перестворення трансляції.

3.2.6 Життєвий цикл трансляції. Стани трансляції (поле `status` таблиці `stream`) і причини завершення (поле `endReason`) наведено на діаграмі станів (рис. 3.7).

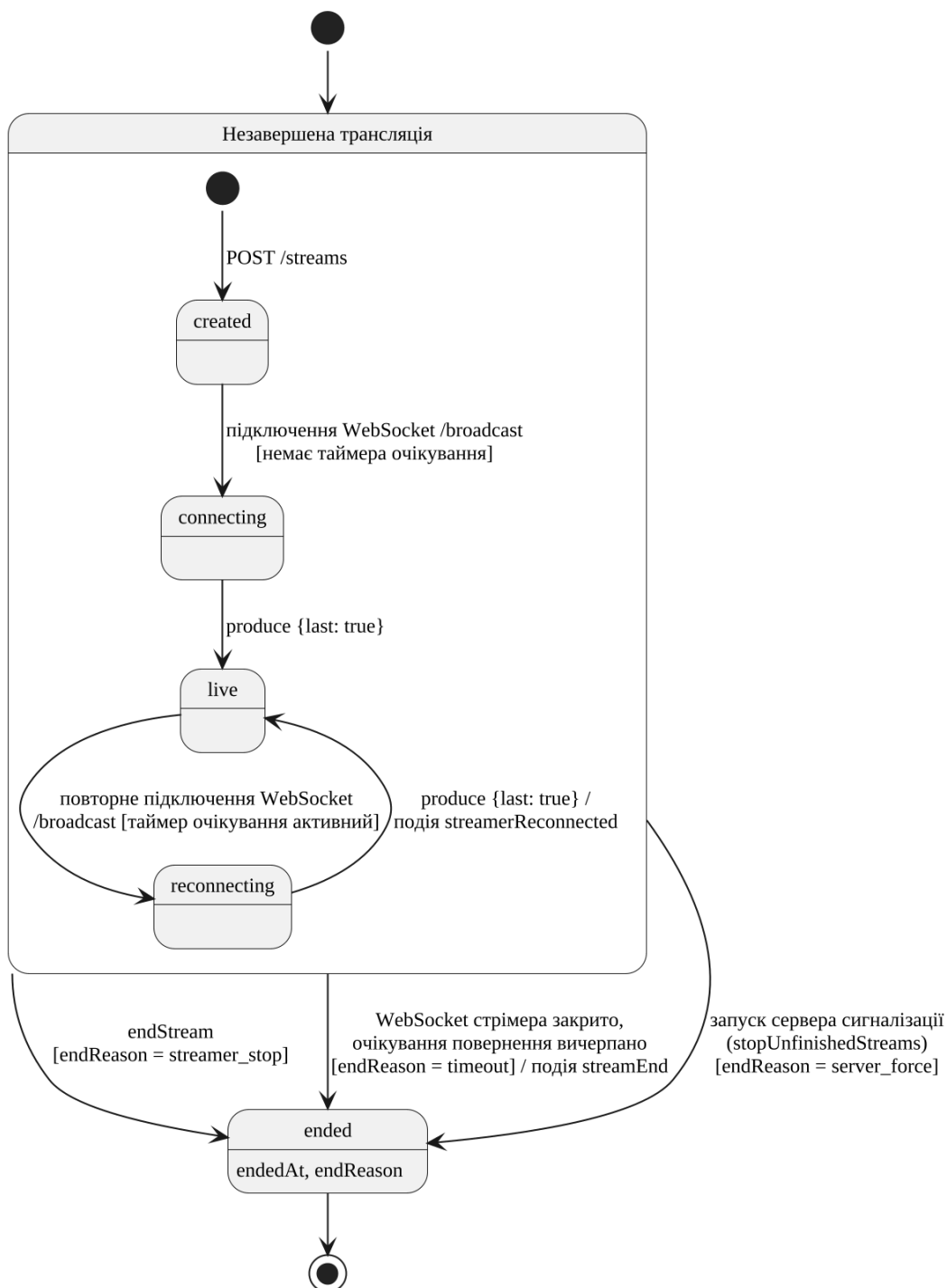


Рисунок 3.7 – Діаграма станів трансляції

Якщо сокет стрімера закривається до **endStream**, сервер надсилає глядачам **streamerDisconnect** і запускає таймер з інтервалом 6 с та обмеженням **MAX_ATTEMPTS = 5** (ФВ-16), не змінюючи статусу в базі даних. Повторне підключення стрімера за активного таймера дає стан **reconnecting**, а **produce** з прапорцем **last** – знову **live**; інакше трансляція завершується з причиною **timeout**. Під час запуску сервер

завершує незавершені трансляції з причиною `server_force`, оскільки їхній контекст у пам'яті втрачено. Значення `admin_force` оголошено, але не використовується.

3.3 Проєктування структури бази даних

3.3.1 Логічна модель даних. Схему бази даних визначено в пакеті `@stream-share/db` і застосовано до PostgreSQL засобами Drizzle. Діаграму «сутність – зв'язок» у нотації ІЕ наведено на рис. 3.8.

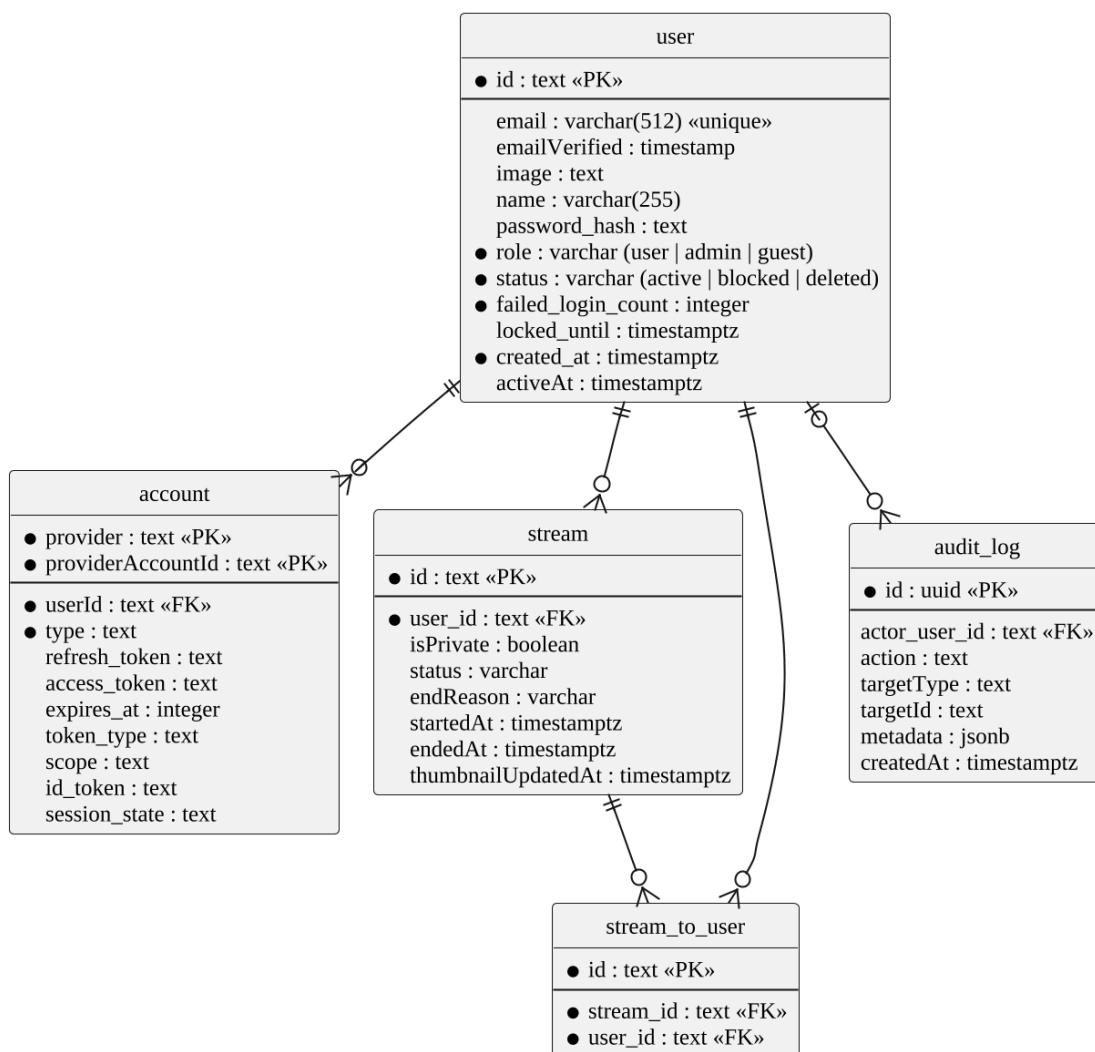


Рисунок 3.8 – Діаграма «сутність – зв'язок» бази даних

Таблиця `stream_to_user` реалізує зв'язок «багато до багатьох» між трансляціями та їхніми поточними глядачами. Таблиці `user` і `account` мають структуру, якої очікує адаптер `Auth.js`: власний `DrizzleAdapter` створює користувача під час першого входу через Google і прив'язує до нього обліковий

запис провайдера. Сеанси зберігаються в зашифрованому JWT у cookie [53], тому таблиць сеансів у схемі немає. Усі зовнішні ключі мають правило `on DELETE CASCADE`, тож щотижневе очищення гостей (ФВ-19), яке видаляє гостей без активності понад 7 днів, що не дивляться жодної трансляції, автоматично видаляє й пов'язані записи.

3.3.2 Опис таблиць. Склад стовпців, типи й обмеження всіх п'яти таблиць наведено в додатку Б (табл. Б.1–Б.5). Стовпці пароля й блокування, стани `blocked / deleted`, роль `admin` і таблицю `audit_log` закладено для входу за паролем і модерації; у поточній версії їх не заповнює жоден модуль.

3.4 Проєктування класів

3.4.1 Класи сервера сигналізації. Код сервера сигналізації поділено на три шари: контролери, сервіси та репозиторії. Діаграму класів наведено на рис. 3.9.

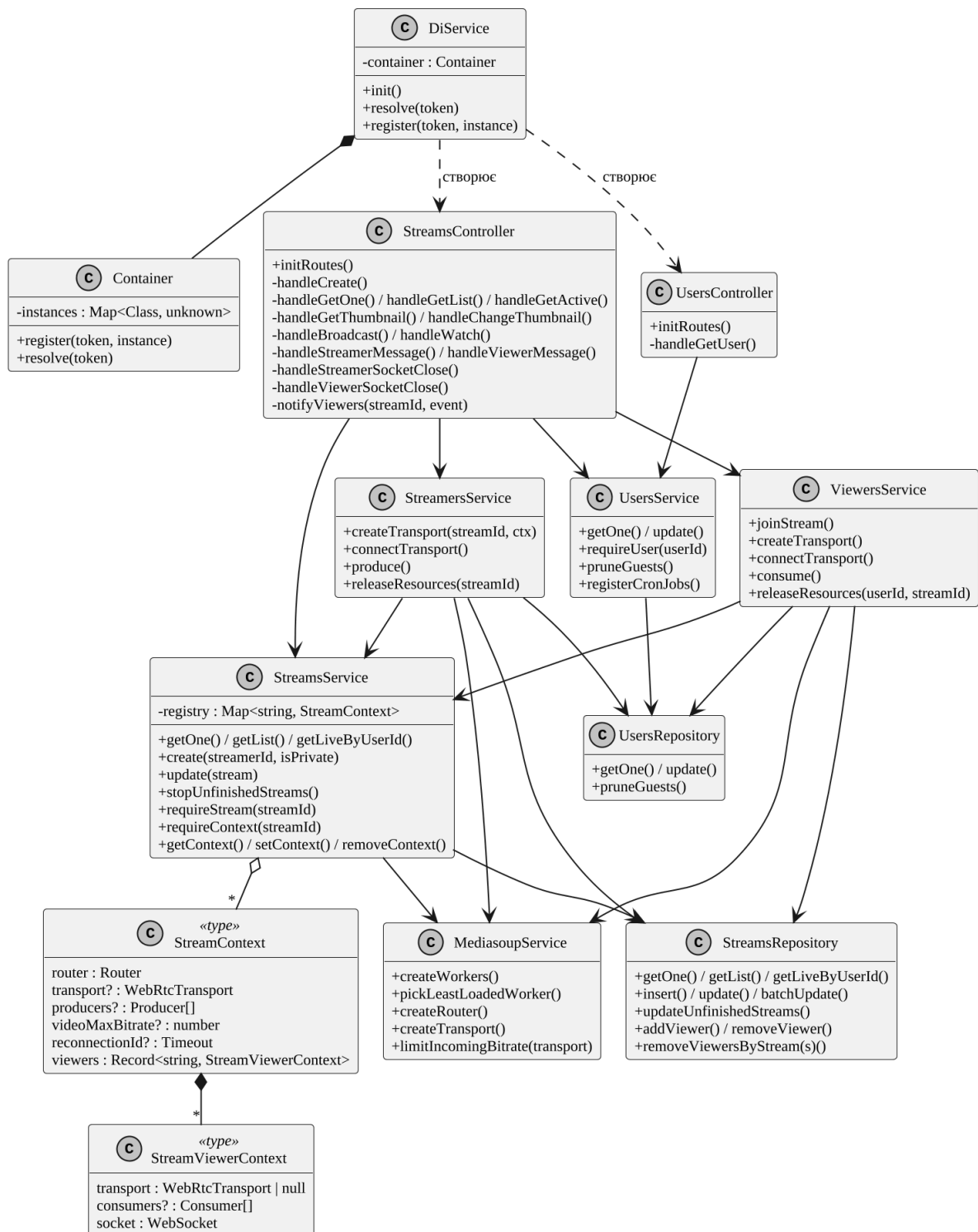


Рисунок 3.9 – Діаграма класів сервера сигналізації

Контролери реєструють маршрути Fastify [54, 55]: `StreamsController` – REST-маршрути трансляцій і обидва WebSocket-канали, `UsersController` – маршрут GET `/users/:userId`. Сервіси містять бізнес-логіку: `StreamsService` – трансляції та реєстр контекстів, `StreamersService` і `ViewersService` – транспорти, продюсери й споживачі стрімера та глядачів, `UsersService` – користувачів і очищення гостей,

MediasoupService – процеси-обробники, маршрутизатори та транспорти. Репозиторії інкапсулюють запити Drizzle. Залежності передаються через конструктори: замість DI-фреймворку клас container зберігає словник «клас → екземпляр», а DiService (декоратор Fastify app.di) під час запуску створює об’єкти в порядку залежностей. Граф об’єктів явний і статично типізований.

3.4.2 Міжпроцесна взаємодія настільного застосунку. Сторінка настільного застосунку працює з увімкненою ізоляцією контексту [56] і не має прямого доступу до API Node.js та Electron. Шар міжпроцесної взаємодії, названий у коді «conveyor», наведено на рис. 3.10.

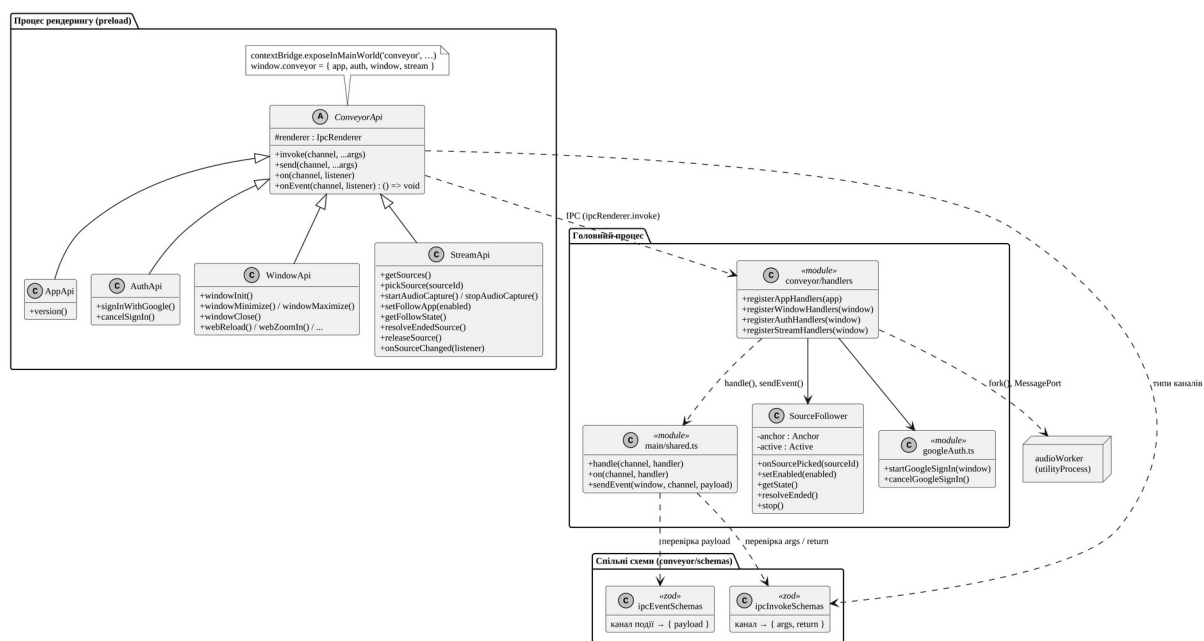


Рисунок 3.10 – Діаграма класів міжпроцесної взаємодії настільного застосунку

Абстрактний клас `conveyorApi` обгортає `ipcRenderer`; його нащадки `AppApi`, `AuthApi`, `WindowApi` і `StreamApi` preload-скрипт публікує як `window.conveyor` через `contextBridge`. У головному процесі функції `register*Handlers` реєструють обробники через `handle()`, а події надсилаються через `sendEvent()`. Обидві сторони спираються на схеми `zod ipcInvokeSchemas` (аргументи й результат каналу) та `ipcEventSchemas` (дані подій): сторона рендерингу отримує з них типи, а головний процес перевіряє дані під час виконання. Захоплення звуку виконує утилітарний процес [57], який передає дані сторінці напряму через пару портів `MessageChannelMain`.

3.5 Проектування інтерфейсу користувача

3.5.1 Навігація. Головна сторінка / відвідувачу без входу показує опис сервісу, кнопку завантаження настільного застосунку та перехід до входу, а користувачу – перелік активних публічних трансляцій із мініатюрами. Кнопка «Start stream» у шапці (для гостей прихована) веде на /broadcast. Сторінка /<streamId>/watch відкривається з картки трансляції або за посиланням і доступна без входу через Google. Написи інтерфейсу англійські, тому на макетах їх наведено мовою оригіналу.

3.5.2 Сторінка трансляції. Макет сторінки трансляції, побудований засобами PlantUML Salt, наведено на рис. 3.11.



Рисунок 3.11 – Макет сторінки трансляції

Під попереднім переглядом з індикатором стану розташовано панель керування: вибір джерела «Pick source» / «Change source», групи якості (360p–1440p, «Source») і частоти кадрів (5, 30, 60) (ФВ-10), перемикачі звуку (ФВ-13) і приватності (ФВ-11), кнопка початку чи зупинки трансляції та поле посилання з кнопкою копіювання (ФВ-12). У настільному застосунку вибір джерела відкриває модальне вікно з вкладками «Applications» і «Entire Screen» (ФВ-07), а панель містить перемикач слідування й напис «Following: ...» (ФВ-09). За наявності незавершеної трансляції замість панелі показано кнопку «Reconnect».

3.5.3 Сторінка перегляду. Макет сторінки перегляду наведено на рис. 3.12.

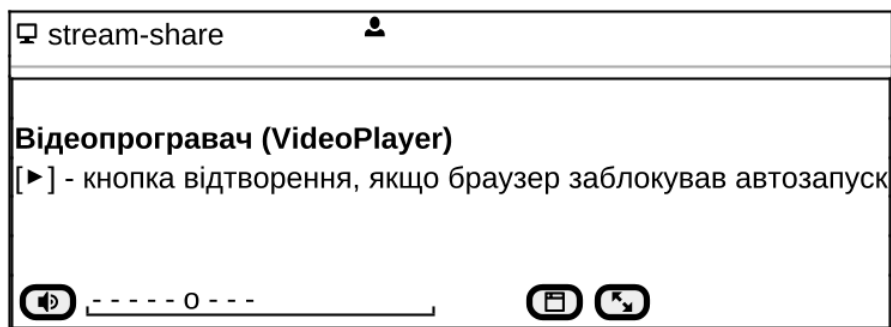


Рисунок 3.12 – Макет сторінки перегляду трансляції

Сторінка містить відеопрогравач з керуванням звуком і гучністю, режимом «картинка в картинці» та повноекранним режимом, доступними й клавішами M, P, F (ФВ-14). Якщо браузер заблокував автоматичне відтворення, поверх відео показано кнопку відтворення. Вибору якості на боці глядача немає. Після завершення трансляції програвач замінюється повідомленням «The stream has ended».

Висновки до розділу 3

Спроектowana система складається з вебзастосунку, сервера сигналізації з SFU-пересиланням медіапотоків і настільного застосунку-оболонки, які спираються на спільні пакети схем даних, протоколу та конфігурації. Систему розгортають як набір контейнерів Docker Compose за зворотним проксі Caddy; медіатрафік WebRTC іде напямую до сервера сигналізації через виділений діапазон портів.

Діаграми процесів формалізують публікацію й перегляд трансляції за протоколом сигналізації поверх WebSocket, вхід у настільному застосунку за схемою PKCE, слідування за вікнами застосунку та відновлення після втрати з'єднання стрімера. Модель даних з п'яти таблиць узгоджено з Auth.js і JWT-сеансами; класи сервера поділено на контролери, сервіси та репозиторії з явним впровадженням залежностей, а міжпроцесну взаємодію типізовано схемами zod. Ці рішення є основою реалізації, описаної в розділі 4.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

4.1 Вибір інструментів розробки

4.1.1 Склад технологічного стеку. Критеріями вибору були єдина мова для всіх модулів, спільний код клієнта й сервера та підтримка WebRTC на кожній стороні. Версії з файлу блокування залежностей наведено в табл. 4.1.

Таблиця 4.1 – Технології розробки програмної системи

Технологія	Версія	Призначення в системі
TypeScript, pnpm (workspaces)	6.0.3 (web), 7.0.2; 12.3.4	мова всіх модулів; монорепозіторій apps/*, packages/*
Next.js, React, Tailwind CSS	16.2.6, 19.2.4, 4.3.0	вебзастосунок
Auth.js (next-auth)	5.0.0-beta.31	вхід через Google, гостьові сеанси
Fastify, @fastify/websocket	5.8.5, 11.2.0	REST- і WebSocket- інтерфейси сервера сигналізації
mediasoup, mediasoup-client	3.20.5, 3.21.0	сервер SFU та його клієнт у браузері
PostgreSQL, Drizzle ORM, drizzle-kit	16, 1.0.0-beta.22	база даних, схеми, міграції
zod	4.4.3	перевірка конфігурації, повідомлень IPC, параметрів запитів
Electron, electron-native-screenshare	35.7.5, 1.2.0	настільний застосунок, звук окремого процесу
electron-vite, electron-builder, electron-updater	5.0.0, 26.8.1, 6.8.9	збирання, інсталятори, автоматичне оновлення
Docker Compose, Caddy, GitHub Actions	–, 2, –	розгортання, HTTPS, збирання й випуск

4.1.2 Мова програмування та організація коду. TypeScript виявляє помилки типів до виконання програми [58], а оскільки всі модулі працюють у середовищах JavaScript, протокол WebSocket, розрахунок бітрейту, схеми бази даних і конфігурацію винесено в спільні пакети робочого простору pnpm [42]. Схеми zod перевіряють дані під час виконання й задають статичні типи [47].

4.1.3 Вебзастосунок і автентифікація. Next.js [59, 60] поєднує сторінки React та API-маршрути, а файл Proху перевіряє доступ до завершення запиту [52] (ФВ-04). Стратегія сеансу JWT в Auth.js зберігає сеанс у зашифрованому файлі cookie без звернень до бази даних, ціною неможливості відкликати його [53], і цей файл cookie розшифровує й сервер сигналізації.

4.1.4 Сервер сигналізації та пересилання медіапотоків. Плагін @fastify/websocket обслуговує маршрути WebSocket [43] тими самими хуками Fastify, що й HTTP [54, 55], тому сеанс перевіряється однаково. Сервери SFU порівняно в табл. 4.2.

Таблиця 4.2 – Порівняння серверів SFU

Критерій	mediasoup	Janus	LiveKit
Форма постачання	модуль Node.js [44]	окремий сервер мовою C [61]	окремий сервер або хмарний сервіс [62]
Взаємодія з прикладним кодом	низькорівневий API в тому самому процесі	JSON-повідомлення до серверних плагінів	кімнати, учасники й треки через SDK

Вбудований у процес сервера сигналізації mediasoup дозволяє керувати трансляцією та повторним підключенням стрімера з одного коду й задати порядок кодеків і межі бітрейту з підрозділу 1.4. Платою є власний сигналізаційний протокол (табл. 3.1) з клієнтом mediasoup-client [51].

4.1.5 Настільний застосунок. Вимоги ФВ-07–ФВ-09 потребують доступу до вікон і процесів операційної системи. Electron і Tauri порівняно в табл. 4.3.

Таблиця 4.3 – Порівняння платформ настільного застосунку

Критерій	Electron	Tauri
Рушій відображення	вбудований Chromium [63]	системний WebView [64]
Основний процес	Node.js [45]	Rust [65]
Захоплення екрана й вікон	desktopCapturer [22, 23]	залежить від WebView [64]

Застосунок відкриває розгорнутий вебзастосунок, і лише Chromium гарантує однакову роботу WebRTC на всіх платформах, а Node.js в основному

процесі дозволяє запустити модуль `electron-native-screenshare` [6] в окремому `UtilityProcess` [57]. Платою є більший розмір дистрибутиву [63].

4.1.6 Зберігання даних. Обрано PostgreSQL – об’єктно-реляційну СКБД з відкритим кодом, що відповідає вимогам ACID [50]. Prisma ORM описує схему окремою мовою й потребує генерації клієнта [66], а схеми обраного Drizzle ORM є модулями TypeScript без такого кроку [46], тому їх зі спільного пакета `packages/db` імпортують і адаптер `Auth.js`, і сервер сигналізації.

4.1.7 Розгортання. Сервіси описано одним файлом `Docker Compose` [48]; Caddy автоматично отримує сертифікати TLS [49], а HTTPS обов’язковий для захоплення екрана [20]. Інсталює `electron-builder` [67], оновлення з випусків GitHub отримує `electron-updater` [36].

4.2 Реалізація захоплення окремих застосунків

4.2.1 Типізований міжпроцесний інтерфейс. Через ізоляцію контексту вебзастосунку в настільному застосунку доступний лише об’єкт `window.conveyor` сценарію `preload` [56]. Методи класу `StreamApi` відповідають каналам `stream:*` основного процесу (рис. 3.10), для кожного з яких у файлі `stream.schema.ts` описано схеми `zod` аргументів і результату [47]. Обгортки `handle()` і `sendEvent()` перевіряють за ними аргументи, результати та події на межі процесів.

4.2.2 Перелік джерел і вибір вікна. Перелік екранів і вікон із мініатюрами та піктограмами застосунків (ФВ-07) формує обробник `stream:getSources`, а підміну системного вибору джерела – обробник запитів захоплення сеансу (лістинг 4.1).

Лістинг 4.1 – Перелік джерел, визначення PID вікна та відповідь на запит захоплення

```
function pidForSource(sourceId: string): number {
  if (!sourceId.startsWith('window:')) return 0;
  const hwnd = Number(sourceId.split(':')[1]);
  return Number.isFinite(hwnd) ? getPidFromWindowHandle(hwnd) : 0;
}
// ...
handle('stream:getSources', async () => {
  const [width, height] = mainWindow.getSize();
  const sources = await desktopCapturer.getSources({
```

```

    types: ['screen', 'window'],
    thumbnailSize: { width: width ?? 320, height: height ?? 180 },
    fetchWindowIcons: true,
  });
  // ...
});
// ...
session.defaultSession.setDisplayMediaRequestHandler(async (request, callback)
=> {
  // ...
  const sources = await desktopCapturer.getSources({ types: ['window', 'screen']
});
  const chosen = sources.find((s) => s.id === selectedSourceId);
  // ...
  callback({ video: chosen ?? sources[0] });
});

```

Джерела повертаються з мініатюрами у вигляді data URL [22]. Функція `pidForSource` виділяє з ідентифікатора `window:...` дескриптор вікна й визначає ідентифікатор процесу (PID) функцією `getPidFromWindowHandle` модуля `electron-native-screenshare` [6], що приймає `HWND` у Windows, `CGWindowID` у macOS або ідентифікатор вікна `X11`; для екранів PID дорівнює 0. Обраний ідентифікатор (`stream:pickSource`) зберігається в основному процесі, а сторінка викликає стандартний `getDisplayMedia` [5] з бажаною роздільністю 3840×2160. Системне вікно вибору не відкривається: обробник `setDisplayMediaRequestHandler` [23] відповідає збереженим джерелом, а за його відсутності – першим у списку.

4.2.3 Захоплення звуку окремого застосунку. Обробник запитів захоплення Electron повертає як звук лише `loopback` – звук усієї системи і лише у Windows [23], тому вимогу ФВ-08 реалізовано модулем `electron-native-screenshare` в утилітному процесі, призначеному, зокрема, для компонентів, схильних до аварійного завершення [45, 57]. Обробник `stream:startAudioCapture` запускає процес методом `utilityProcess.fork` і передає йому PID джерела та один кінець каналу `MessageChannelMain` [68] (лістинг 4.2).

Лістинг 4.2 – Запуск захоплення звуку процесу в утилітному процесі

```

process.parentPort.once('message', (e) => {
  const { processId } = e.data as AudioWorkerInit;
  const port = e.ports[0];
  // ...
  try {
    const startCaptureCallback: AudioDataCallback = (audioData, meta) => {
      const ab = audioData.buffer.slice(
        audioData.byteOffset,
        audioData.byteOffset + audioData.byteLength,
      );
      port.postMessage({ ab, meta });
    };
  }
});

```

```

const args: [number, boolean, AudioDataCallback] = processId
  ? [processId, true, startCaptureCallback]
  : [process.pid, false, startCaptureCallback];

port.start();
const started = startCapture(...args);
if (!started) throw new Error('startCapture() returned false');
report({ type: 'ready' });
} catch (err) {
  report({ type: 'error', error: err instanceof Error ? err.message :
String(err) });
}
});

```

Для відомого PID `startCapture` працює в режимі включення й захоплює звук лише цільового процесу, а для екрана – у режимі виключення самого утилітного процесу, тобто захоплює весь системний звук. Буфери PCM з метаданими формату йдуть портом, другий кінець якого основний процес передає сторінці лише після повідомлення `ready`. На сторінці модуль `audio.bridge.ts` створює вузол `AudioWorkletNode`, процесор якого виконується в окремому потоці Web Audio [69]: він перетворює відліки на дійсні, лінійно передискретизує їх до частоти контексту, тримає в буфері не більше 1 с звуку, а за нестачі даних видає тишу. Отримана звукова доріжка додається до потоку `getDisplayMedia`; якщо будь-який крок не вдається, трансляція продовжується лише з відео з повідомленням користувачу (ФВ-08). [ПОТРЕБУЄ УТОЧНЕННЯ: операційні системи, на яких фактично перевірено захоплення звуку окремого застосунку.]

4.2.4 Слідування за вікнами застосунку. Алгоритм режиму слідування (ФВ-09) наведено на рис. 3.6. Після вибору вікна клас `SourceFollower` фіксує якір: PID, виконуваний файл, корінь сімейства процесів і час вибору. Таблицю процесів отримують командою `Get-CimInstance Win32_Process` у Windows або `ps` в інших ОС. Коренем сімейства є найвищий предок, що не належить до оболонки (`explorer.exe`, `launchd`, `systemd` тощо), а батько, створений більш ніж на 2 с пізніше за нащадка, не враховується, що захищає від повторного використання PID. Кожні 1500 мс клас перевіряє нові вікна (лістинг 4.3).

Лістинг 4.3 – Вибір вікна для слідування в класі `SourceFollower`

```

const candidates: (PidWindow & { createdAt: Date })[] = [];
for (const w of others) {
  const info = table.get(w.pid);
  if (!info) continue;
  if (!isInFamily(table, w.pid, anchor.familyRoot)) continue;
  if (info.createdAt <= anchor.startedAt) continue;
}

```

```

const sameExe = info.name.toLowerCase() === anchor.exe.toLowerCase();
if (sameExe && !anchorAlive) {
  // The anchor process was replaced (League's "close client during game").
  this.anchor = { ...anchor, sourceId: w.id, pid: w.pid };
  // ...
  return;
}
if (sameExe) continue;
candidates.push({ ...w, createdAt: info.createdAt });
}
if (candidates.length === 0) return;

candidates.sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime());
const winner = candidates[0]!;
// ...
this.deps.setSelectedSourceId(winner.id);
// ...
this.deps.emit({ reason: 'follow', sourceId: winner.id, name: winner.name });

```

Обирається найновіше вікно процесу того самого сімейства, запущеного після вибору якоря, з іншим виконуваним файлом; процес з тим самим файлом за завершеного якоря означає перезапуск застосунку, і якір переноситься на нього. Після закриття вікна, за яким слідували, клас повертається до якоря, а якщо той не з'являється за 20 с, надсилає подію `lost`, і трансляція зупиняється. На події `follow` і `return` сторінка послідовно (функція `runSourceSwitch`) зупиняє доріжки й утилітний процес, знову викликає `getDisplayMedia`, запускає захоплення звуку нового PID і замінює доріжки методом `replaceTrack`, що не потребує узгодження із сервером [70], тому глядачі не перепідключаються.

4.2.5 Конвеєр захоплення й публікації. Взаємодію описаних механізмів із сервером показано на рис. 4.1.

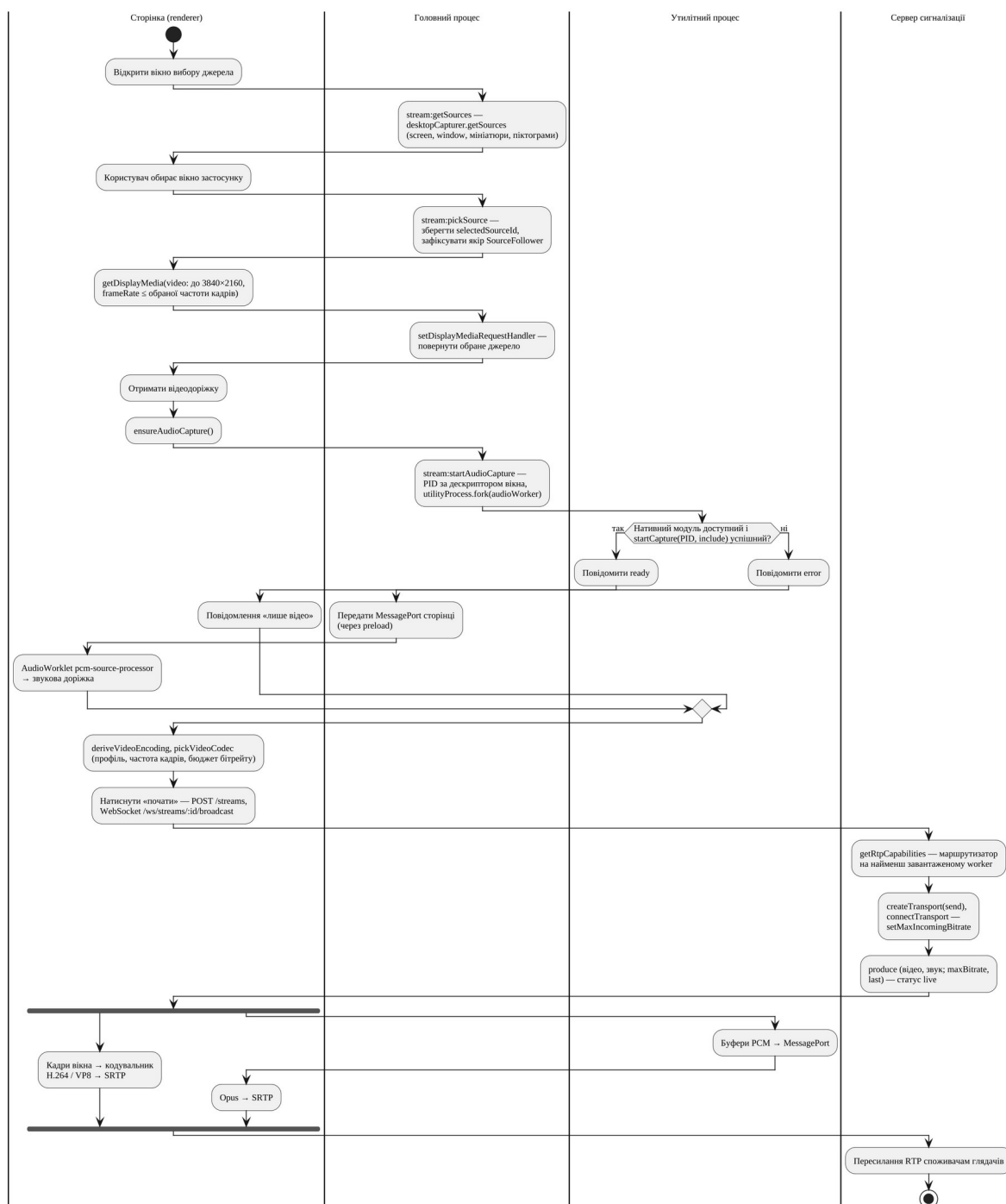


Рисунок 4.1 – Діаграма діяльності конвеєра захоплення й публікації трансляції

Відео й звук ідуть паралельними шляхами: кадри вікна кодує стек WebRTC Chromium, а відліки PCM надходять на сторінку через порт повідомлень і кодуються в Orpus; розгалуження відповідає режиму «лише відео».

4.3 Реалізація потокової передачі на основі WebRTC та SFU

4.3.1 Процеси mediasoup і маршрутизатори. Сервер сигналізації запускає MEDIASOUP_NUM_WORKERS процесів worker (типово – за кількістю ядер процесора) – підпроцесів C++, що обробляють ICE, DTLS і RTP [44], з портами RTC 40000–49999. Кожна трансляція отримує власний маршрутизатор на найменш завантаженому worker (лістинг 4.4).

Лістинг 4.4 – Вибір найменш завантаженого процесу worker

```
async pickLeastLoadedWorker(): Promise<Worker> {
  if (!this.workers.length) {
    throw new Error('Workers is empty');
  }

  const usages = await Promise.all(
    this.workers.map(async (w) => ({
      entry: w,
      usage: await w.getResourceUsage(),
    })),
  );
  usages.sort((a, b) => a.usage.ru_utime - b.usage.ru_utime);

  return usages[0]!.entry;
}
```

Мірою навантаження є час виконання в режимі користувача ru_utime з getResourceUsage [71]; він накопичується від запуску процесу, тож вибір враховує сумарне, а не поточне навантаження.

4.3.2 Кодеки. Маршрутизатор оголошує Opus з частотою 48 кГц і двома каналами, H.264 у двох профілях і VP8 (лістинг 4.5).

Лістинг 4.5 – Кодеки відео маршрутизатора mediasoup

```
const router = await worker.createRouter({
  mediaCodecs: [
    // ...
    {
      kind: 'video',
      mimeType: 'video/H264',
      clockRate: 90000,
      parameters: {
        'packetization-mode': 1,
        'profile-level-id': '42001f',
        'level-asymmetry-allowed': 1,
        'x-google-start-bitrate': START_BITRATE_KBPS,
      },
    },
    // ...
    /** Software fallback for endpoints without H.264. */
    {
      kind: 'video',
```

```

    mimeType: 'video/VP8',
    // ...

```

Браузери зобов'язані підтримувати VP8 і H.264 Constrained Baseline [72], тому кожному клієнту доступний хоча б один кодек. За коментарем у коді H.264 стоїть першим, бо лише його кодують апаратно всі цільові клієнти, що робить стійкими режими 1080p60, 1440p60 і 4K, а Baseline (42001f) передує Constrained Baseline (42e01f), бо останній у Chromium надає лише програмний OpenH264. Коментар посилається на перевірку розробником у Chrome 152 з NVENC; власних вимірювань у роботі не виконувалося. Параметр `packetization-mode=1` задає неперемежований режим пакетизації [73], `level-asymmetry-allowed` дозволяє рівні, вищі за оголошений (1440p60 потребує рівня 5.1). Клієнт обирає кодек функцією `pickVideoCodec` у тому самому порядку.

4.3.3 Параметри кодування. Параметри відео обчислює функція `deriveVideoEncoding` (лістинг 4.6) за розмірами доріжки та обраними профілем і частотою кадрів (ФВ-10; типово 720p і 30 кадр/с).

Лістинг 4.6 – Обчислення параметрів кодування відео

```

export function deriveVideoEncoding(
  track: { width: number; height: number },
  settings: VideoSettings,
): DerivedVideoEncoding {
  const { quality } = settings;
  const fps = clampVideoSettings(settings, track.height).fps;
  const scaleResolutionDownBy = getScaleResolutionDownBy(track.height, quality);
  const targetWidth = track.width / scaleResolutionDownBy;
  const targetHeight = track.height / scaleResolutionDownBy;
  const maxBitrate = videoBitrateBudget(targetWidth * targetHeight, fps);
  const isHighFps = fps > HIGH_FPS_THRESHOLD;

  return {
    encoding: { scaleResolutionDownBy, maxFramerate: fps, maxBitrate },
    degradationPreference: isHighFps ? 'maintain-framerate' : 'maintain-
resolution',
    contentHint: isHighFps ? 'motion' : 'detail',
    frameRate: { ideal: fps, max: fps },
    startBitrateKbps: Math.round(startBitrateFor(maxBitrate) / 1000),
  };
}

```

Коефіцієнт `scaleResolutionDownBy` – відношення висоти доріжки до висоти профілю, не менше 1, тож зображення не збільшується; за вихідної висоти понад 1440 пікселів частоту обмежено 30 кадр/с. Бюджет бітрейту обчислює спільна функція `videoBitrateBudget` за формулою (4.1).

$$B = \min \left(\max \left(N \cdot 30 \cdot 0,07 \cdot \left(\frac{f}{30} \right)^{\log_2 1,5}; 3 \cdot 10^5 \right); 1,8 \cdot 10^7 \right), \quad (4.1)$$

де B – бюджет бітрейту, біт/с; N – кількість пікселів вихідного кадру; f – частота кадрів, кадр/с. Показник $\log_2 1,5$ відображає закладене в коді припущення, що подвоєння частоти кадрів потребує в 1,5 раза більшого бітрейту. Початковий бітрейт кодувальника (`videoGoogleStartBitrate` [70]) становить половину бюджету. За 60 кадр/с обираються `degradationPreference maintain-framerate` і підказка `motion`, інакше – `maintain-resolution` і `detail`, тобто за нестачі пропускнуої здатності зберігається відповідно плавність або чіткість [74]. Обчислені значення для джерела 3840×2160 наведено в табл. 4.4.

Таблиця 4.4 – Параметри кодування відео для джерела 3840×2160, обчислені за кодом

Профіль	Вихідна роздільність	scaleResolutionDown By	maxBitrate, Мбіт/с (5 / 30 / 60 кадр/с)	Початковий бітрейт, Мбіт/с (30 / 60 кадр/с)
360p	640×360	6	0,30 / 0,48 / 0,73	0,24 / 0,36
480p	853×480	4,5	0,30 / 0,86 / 1,29	0,43 / 0,65
720p	1280×720	3	0,68 / 1,94 / 2,90	0,97 / 1,45
1080p	1920×1080	2	1,53 / 4,35 / 6,53	2,18 / 3,27
1440p	2560×1440	1,5	2,71 / 7,74 / 11,61	3,87 / 5,81
Source	3840×2160	1	6,11 / 17,42 / –	8,71 / –

Значення є розрахунковими верхніми межами; фактичний бітрейт визначає оцінювання пропускнуої здатності WebRTC. Зміна профілю чи частоти під час трансляції не потребує повторного узгодження: `applyConstraints` змінює частоту захоплення, а `RTCRtpSender.setParameters` – параметри кодування [7].

4.3.4 Обмеження бітрейту на сервері. Після з'єднання транспорту стрімера сервер викликає `setMaxIncomingBitrate` [71] з типовим значенням 21,6 Мбіт/с – максимальний бюджет 18 Мбіт/с із запасом 1,2 на Orus, повторні передачі й заголовки; менше значення спричиняє попередження під час запуску. Транспорти глядачів створюються з `initialAvailableOutgoingBitrate`, що дорівнює половині бюджету відео, але не менше 4 Мбіт/с, тому для трансляції 4К оцінювання починається з 8,71 Мбіт/с замість типових для mediasoup 600 кбіт/с [71].

4.3.5 Сигналізація. Дії та події (табл. 3.1) передаються через WebSocket [43] як JSON-конверти запиту, відповіді з ознакою `ok` і події. Їхні типи виводяться зі спільного для клієнта й сервера типу `SignalingApi` пакета `packages/shared`, проте на відміну від IPC під час виконання схемами не перевіряються. Клієнтську частину реалізує клас `wsclient` (лістинг 4.7).

Лістинг 4.7 – Типізований запит клієнта сигналізації

```
async request<M extends WsResponse['method']>(
  msg: Extract<WsRequest, { method: M }>,
): Promise<Extract<WsResponse, { method: M }>> {
  if (this.openPromise) await this.openPromise;
  const requestId = this.nextId++;
  return new Promise((resolve, reject) => {
    this.pending.set(requestId, {
      resolve: resolve as (msg: WsResponse) => void,
      reject,
    });
    this.ws.send(constructRequest({ ...msg, requestId }));
  }) as Promise<Extract<WsResponse, { method: M }>>;
}
```

Відповідь знаходить свою обіцянку за `requestId`, а закриття з'єднання відхиляє всі очікувані запити. Послідовність публікації (рис. 3.3) хук `useStreamer` реалізує через `Device.load` і `createSendTransport` [70]: подія `connect` транспорту надсилає `ConnectTransport`, а `produce` — `Produce` з бюджетом `maxBitrate` і ознакою `last`, після якої трансляція переходить у стан `live`.

4.3.6 Відтворення у глядача. Хук `useViewer` реалізує послідовність рис. 3.4: після `JoinStream` створюються приймальний транспорт і споживач для кожного виробника, причому сервер перевіряє `router.canConsume` [71]. Доріжки об'єднуються в `MediaStream` елемента `video`; якщо автовідтворення заблоковано, показується кнопка запуску. Подія `streamerReconnected` (рис. 3.7) замінює споживачів без перезавантаження сторінки. Мініатюру трансляції сторінка стрімера знімає одразу і далі кожні 5 хв як JPEG з якістю 0,7.

Обмеженням реалізації є єдине кодування без `simulcast` і `SVC`: `simulcast` у `mediasoup-client` потребує кількох записів у `encodings` [70], тому всі глядачі отримують однакову якість. Дія `SetPreferredLayer`, оголошена у спільному протоколі, не має обробника ні на сервері, ні в клієнті.

4.4 Реалізація автентифікації та авторизації

4.4.1 Конфігурація Auth.js. Автентифікацію реалізовано у файлі `lib/auth/auth.ts` вебзастосунку трьома провайдерами Auth.js. Провайдер Google (ФВ-01) передає параметр `prompt: 'select_account'`, тож вибір облікового запису пропонується під час кожного входу [75]. Два провайдери типу `Credentials`, у яких рішення ухвалює власна функція `authorize` [76], обслуговують гостьовий сеанс (`guest`, ФВ-03) і завершення входу з настільного застосунку (`desktop`, ФВ-02). Параметри сеансу наведено в лістингу 4.8.

Лістинг 4.8 – Стратегія сеансу та параметри файлу `cookie` в конфігурації Auth.js

```
const config: NextAuthConfig = {
  adapter,
  session: {
    strategy: 'jwt',
  },
  useSecureCookies: useSecureAuthCookies,
  cookies: {
    sessionToken: {
      name: SESSION_COOKIE_NAME,
      options: {
        httpOnly: true,

        sameSite: 'lax',
        path: '/',
        secure: useSecureAuthCookies,
      },
    },
  },
  providers: [
    // ...
  ],
}
```

Сеанс зберігається в зашифрованому токени у файлі `cookie` [53]. Атрибут `HttpOnly` закриває `cookie` від сценаріїв сторінки, `SameSite=Lax` обмежує його надсилання з інших сайтів, а `Secure` дозволяє передавання лише через HTTPS [77]. Ознака `useSecureAuthCookies` спільного пакета `shared` істинна лише у виробничому середовищі, і тоді ім'я `cookie` отримує префікс `__secure-`, з яким браузер приймає його тільки від захищеної сторінки [77]. Функції зворотного виклику `jwt` і `session` переносять у токен і сеанс роль користувача. Гостьовий провайдер створює запис із роллю `guest` і випадковим іменем, ідентифікатор якого сторінка перегляду зберігає в `localStorage` для повторного використання.

4.4.2 Обмеження доступу. Файл `proxy.ts` (ФВ-04) [52] не вважає гостьовий сеанс входом і перенаправляє такого відвідувача на `/login` з усіх сторінок, крім головної, сторінки входу та `/desktop-auth`. Шаблон `matcher` виключає з перевірки API-маршрути, статичні файли й шляхи `<UUID>/watch`, тому перегляд доступний без облікового запису Google.

Сервер сигналізації самостійно розшифровує той самий cookie (лістинг 4.9). Ключ розшифрування `@auth/core` виводить із секрету та «солі» – імені cookie, тому сервер бере це ім'я зі спільного модуля, а `AUTH_SECRET` обох сервісів збігаються.

Лістинг 4.9 – Перевірка сеансового cookie на сервері сигналізації

```
export const NEXT_AUTH_SALT = SESSION_COOKIE_NAME;

export async function decodeAuthToken(rawToken: string): Promise<JWT | null> {
  const payload = await decode({
    salt: NEXT_AUTH_SALT,
    secret: env.AUTH_SECRET,
    token: rawToken,
  });
  if (!payload?.sub) return null;
  return payload;
}
// ...
export const validateWsJwt = async (
  req: FastifyRequest, reply: FastifyReply): Promise<void> => {
  const rawToken = req.cookies[NEXT_AUTH_SALT];
  // ...
  const payload = await decodeAuthToken(rawToken);

  if (!payload) {
    reply.code(401).send({ error: 'Invalid token' });
    return;
  }

  req.context.userId = payload.sub;
};
```

Функція `validateWsJwt` є хуком `prevalidation` групи маршрутів `/streams` і обох маршрутів `WebSocket`; відповідь 401 з хука зупиняє обробку запиту [78]. Належність трансляції перевіряє лише маршрут завантаження мініатюри (відповідь 403), а сокет публікації покладається на непередбачуваність `UUID` трансляції. Приватну трансляцію (ФВ-11) обробник `/streams/search` виключає з переліку примусовим фільтром `isPrivate = false`, а переглянути її може кожен, хто має посилання.

4.4.3 Вхід у настільному застосунку. Послідовність входу наведено на рис. 3.5, а її початок у головному процесі Electron – у лістингу 4.10.

Лістинг 4.10 – Параметри PKCE і запуск входу в системному браузері

```
const verifier = base64url(randomBytes(32));
const challenge = createHash('sha256').update(verifier).digest('base64url');
const state = base64url(randomBytes(16));
// ...
const port = await listen(server);
// ...
const url = new URL('/api/desktop-auth/start', __WEB_URL__);
url.searchParams.set('port', String(port));
url.searchParams.set('state', state);
url.searchParams.set('challenge', challenge);

await shell.openExternal(url.toString());
```

Локальний сервер слухає 127.0.0.1 на порту, призначеному операційною системою [37], і відхиляє запити з чужим state. Вебзастосунок (lib/auth/desktopHandoff.ts) перевіряє параметри схемою zod, адресу повернення будує з фіксованого `http://127.0.0.1:<port>/callback`, що виключає відкрите перенаправлення, а запит і одноразовий код шифруються з різними «солями», тож один тип токена не можна підставити замість іншого. Провайдер desktop порівнює SHA-256 від verifier з challenge коду функцією сталого часу `timingSafeEqual` [38] і відхиляє повторний jti. Реєстр jti зберігається в пам'яті процесу, тому за кількох екземплярів вебзастосунку одноразовість не гарантована.

Вхід за паролем, обмеження спроб входу та журнал аудиту не реалізовано: стовпці `password_hash`, `failed_login_count`, `locked_until` і таблиця `audit_log` лише зарезервовані у схемі. Сеанс JWT неможливо відкликати до завершення строку дії [53].

4.5 Реалізація рівня даних і програмного інтерфейсу сервера

4.5.1 Пакет бази даних. Таблиці (рис. 3.8) описано функцією `pgTable` у пакеті `db`, а зв'язок трансляції зі стрімером і глядачами – функцією `defineRelations`. Типи рядків виводяться зі схем (`$inferSelect`, `$inferInsert`), тому вебзастосунок і сервер сигналізації використовують одні типи без генерації коду [46]. Файлів міграцій немає: контейнер `migrate` виконує `drizzle-kit push --force`, що застосовує відмінності схеми безпосередньо до бази даних [79]. Вебзастосунок звертається до бази даних лише через власний адаптер `Auth.js drizzleAdapter`, а

об'єкт підключення створюється під час першого запиту, тому next build не потребує бази даних.

4.5.2 Шари сервера сигналізації. Репозиторії сервера інкапсують запити Drizzle, сервіси перетворюють відсутність запису на EntityNotFoundError, а контролери – помилки на коди 404, 400 чи 500 (рис. 3.9). Маршрути наведено в табл. 4.5; ззовні вони доступні через Caddy з префіксом /sfu.

Таблиця 4.5 – Маршрути програмного інтерфейсу сервера сигналізації

Метод і шлях	Сеанс	Призначення
POST /streams	так	створення трансляції (ознака isPrivate)
POST /streams/search	так	сторінка публічних трансляцій з фільтрами й сортуванням
GET /streams/:userId/active	так	активна трансляція користувача
GET, POST /streams/:streamId/thumbnail	так (POST – власник)	отримання та збереження мініатюри
GET /streams/:streamId	ні	трансляція зі стрімером і глядачами
GET /users/:userId	ні	відомості про користувача
GET /ws/streams/:streamId/broadcast	так	WebSocket стрімера
GET /ws/streams/:streamId/watch	так, зокрема гостьовий	WebSocket глядача

Метод `userService.registerCronJobs` реєструє в плагіні `@fastify/schedule` завдання з виразом `0 0 * * 1` (понеділок, 00:00), яке одним оператором `DELETE` видаляє гостей без активності понад 7 діб, що не дивляться жодної трансляції (ФВ-19).

4.5.3 Перевірка конфігурації. Функція `createEnv` пакета `env` під час запуску перевіряє змінні середовища схемою `zod`, вважає порожні значення відсутніми й у разі помилок виводить перелік усіх некоректних змінних та зупиняє запуск. Зокрема, `AUTH_SECRET` має містити не менше 32 символів. Секрети вебзастосунку позначено `server-only`, тож їх імпорт у клієнтський код спричиняє помилку збирання, а у виробничому середовищі `AUTH_URL` має бути адресою HTTPS, без якої cookie з атрибутом `Secure` не працюватиме.

4.6 Реалізація інтерфейсу користувача та розгортання

4.6.1 Структура інтерфейсу. Сторінки розміщено у двох групах маршрутів каталогу `app:` (`auth`) для `/login` і `/desktop-auth` та (`main`) зі спільними шапкою й підвалом для головної сторінки, `/broadcast`, `/[streamId]/watch`, `/privacy` і `/terms`. Серверні компоненти отримують сеанс функцією `auth()` і дані з сервера сигналізації через внутрішню мережу Docker, а клієнтські звертаються до нього через префікс `/sfu`. Головна сторінка відвідувачу показує кнопку завантаження настільного застосунку для операційної системи із заголовка User-Agent (ФВ-18), а користувачу – по 12 карток активних трансляцій на сторінці (ФВ-05). Базові елементи (`Button`, `Input`, `Link`, `Typography`) мають варіанти, описані функцією `sva` бібліотеки `class-variance-authority`.

4.6.2 Теми оформлення. Кольори задано змінними CSS для `:root` (світла тема) і класу `.dark` (темна тема), які директива `@theme` перетворює на змінні теми Tailwind CSS з класами `bg-canvas`, `text-stroke` тощо [80]. Компонент `ThemeProvider` бібліотеки `next-themes` з параметрами `attribute="class"` і `defaultTheme="system"` обирає тему за налаштуванням операційної системи [81]; окремого перемикача теми немає. Головну сторінку в обох темах наведено на рис. 4.2.

[ПОТРЕБУЄ УТОЧНЕННЯ: РИСУНОК: скріншот головної сторінки зі списком активних трансляцій у світлій і темній темах – замінити заглушку `thesis/screenshots/scr-home-themes.png`]

[РИСУНОК: скріншот]

Головна сторінка вебзастосунку для користувача, що увійшов, зі списком активних трансляцій (картки з мініатюрами, перемикач сторінок):
а - світла тема операційної системи; б - темна тема (два знімки поруч)

Заглушка: замінити файл `thesis/screenshots/scr-home-themes.png` реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 4.2 – Головна сторінка вебзастосунку: а – світла тема; б – темна тема

Теми різняться лише значеннями змінних кольорів, тому компонент, що використовує кольори теми, підтримує обидві без додаткового коду.

4.6.3 Збирання та розгортання. За тегом `v*` робочий процес GitHub Actions збирає образи `web`, `signaling` і `migrate` у GitHub Container Registry та оновлює їх на сервері через SSH (`docker compose pull, up -d`) згідно з діаграмою розгортання (рис. 3.2), а за тегом `desktop-v*` – інсталятори NSIS, DMG і AppImage засобами `electron-builder` [67] для випуску GitHub. Автоматичне оновлення [36] працює лише у Windows і Linux, бо в macOS воно потребує підписаного застосунку.

Висновки до розділу 4

Систему реалізовано мовою TypeScript зі спільними пакетами протоколу, схем даних і конфігурації. Захоплення окремого застосунку з його звуком і слідування за його вікнами виконує настільний застосунок Electron, а потокову передачу – SFU mediasoup з H.264 як основним кодеком.

Автентифікація поєднує вхід через Google, гостьові сеанси та вхід настільного застосунку через системний браузер за схемою PKCE. Зашифрований сеанс JWT у cookie з атрибутами `HttpOnly`, `SameSite=Lax` і, у виробничому середовищі, `Secure` перевіряє й сервер сигналізації. Конфігурацію перевіряють схеми `zod`, а інтерфейс підтримує світлу й темну теми. Обмеженнями є відсутність перевірки власника для сокета публікації та неможливість відкликати сеанс до завершення строку дії.

5 ТЕСТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

5.1 Принципи вибору тестових прикладів

5.1.1 Об'єкт і рівні перевірки. Об'єктом тестування є програмна система у складі вебзастосунку (apps/web), сервера сигналізації й пересилання медіапотоків (apps/signaling), настільного застосунку (apps/desktop) і спільних пакетів (packages/*) у стані фіксації 7c2f481, програмний код якої на гілці diploma збігається з основною гілкою. Репозиторій продукту не містить автоматизованих модульних, інтеграційних чи наскрізних тестів, а конвеєри безперервної інтеграції (desktop-release.yml, docker-prod.yml) лише збирають інсталятори настільного застосунку та образи контейнерів. Тому перевірку проведено на трьох рівнях: статична перевірка всього вихідного коду засобами, налаштованими в репозиторії; функціональні перевірки серверних компонентів і логіки настільного застосунку сценаріями, підготовленими для цього розділу; ручні перевірки сценаріїв, які потребують реального захоплення екрана, браузера та операційної системи.

5.1.2 Вибір тестових прикладів. Тестові приклади виведено з функціональних вимог ФВ-01...ФВ-19 (табл. 1.3) і сценаріїв використання 1.1, 1.2, 1.3, 1.4 та 1.5. Для кожного сценарію обрано позитивний випадок основного потоку та негативні випадки, що відповідають альтернативним потокам і межам доступу: анонімний і гостьовий користувач, відсутній і підроблений токен сеансу, неіснуюча та приватна трансляція, сторонній користувач щодо чужої трансляції. Для логіки з часовими параметрами (повторні спроби відновлення, інтервал опитування вікон) перевірено фактичні інтервали спрацювання. Нефункціональні перевірки відповідають характеристикам якості табл. 1.4 за моделлю ISO/IEC 25010 [35].

5.1.3 Стенд і засоби. Автоматизовані перевірки виконано 05.10.2026 на стенді з Debian 13 (aarch64), Node.js 24.21.0 і PostgreSQL 18.4; використано mediasoup 3.20.5 і Next.js 16.2.6 із файлу блокування залежностей. Схему бази

даних створено командою `drizzle-kit push`, сервери запущено у виробничому режимі (`next start, node dist/index.js`). Сценарій `thesis/tests/smoke.mjs` звертається до запущених серверів через HTTP і WebSocket тими самими повідомленнями протоколу, що й клієнтський код; сеанс користувача Google, для якого потрібен зовнішній постачальник, сформовано функцією `encode()` бібліотеки `Auth.js` з тим самим секретом і сіллю, якими користується система [53], а гостьовий сеанс отримано через справжню кінцеву точку входу. Сценарій `thesis/tests/follower.mjs` збирає модуль `SourceFollower` настільного застосунку без `Electron` і подає йому змодельований перелік вікон і таблицю процесів. Обидва сценарії не захоплюють, не кодують і не відтворюють медіадані, тому відповідні перевірки віднесено до ручних. Порядок відтворення описано у файлі `thesis/tests/README.md`.

Результати статичної перевірки наведено в табл. 5.1.

Таблиця 5.1 – Результати статичної перевірки та збирання

Перевірка	Команда	Фактичний результат
Встановлення залежностей	<code>pnpm install --frozen-lockfile</code>	успішно, 2 хв 12 с
Статичний аналіз ESLint	<code>pnpm lint</code>	6 із 6 пакетів без помилок, 1 хв 22 с
Перевірка типів TypeScript	<code>pnpm typecheck</code>	6 із 6 пакетів без помилок, 35 с
Збирання спільних пакетів і сервера сигналізації	<code>pnpm -r build (tsup)</code>	успішно
Збирання вебзастосунку без змінних середовища	<code>next build</code>	зупинка з переліком відсутніх змінних <code>DATABASE_URL</code> , <code>AUTH_SECRET</code> , <code>GOOGLE_CLIENT_ID</code> , <code>GOOGLE_CLIENT_SECRET</code>
Збирання вебзастосунку, як у <code>Dockerfile</code>	<code>SKIP_ENV_VALIDATION=1 next build</code>	успішно: компіляція 70 с, перевірка типів 27,6 с, 10 статичних сторінок
Збирання настільного застосунку	<code>electron-vite build</code>	успішно за 3,4 с: <code>main.js</code> 33,6 кБ, <code>preload.js</code> 3,4 кБ, <code>audioworker.js</code> 1,2 кБ
Пакування інсталяторів	<code>electron-builder</code>	не виконувалося: пакування виконує CI під час публікації тегу

Перевірка типів виконується в суворому режимі (strict, noUncheckedIndexedAccess), а конфігурація ESLint окрім рекомендованих правил TypeScript забороняє необроблені проміси (no-floating-promises, no-misused-promises) і пряме звертання до process.env поза модулями конфігурації. Зупинка збирання без змінних середовища є очікуваною поведінкою: схеми createEnv перевіряють конфігурацію до запуску, а образ контейнера свідомо вимикає цю перевірку на етапі збирання й виконує її під час запуску.

5.2 Функціональне тестування

Тестові приклади згруповано за сценаріями використання в табл. В.1–В.5 додатка В. Ідентифікатор прикладу збігається з позначкою у виводі сценаріїв smoke.mjs і follower.mjs; фактичні результати автоматизованих прикладів отримано під час прогону 05.10.2026, а для ручних прикладів наведено, що саме має виконати автор роботи.

5.2.1 Автентифікація та обмеження доступу. Приклади табл. В.1 перевіряють вимоги ФВ-01 – ФВ-04 і перевірку сеансу сервером сигналізації.

Обмеження доступу до сторінок (ФВ-04) і гостьовий вхід (ФВ-03) відповідають вимогам, cookie сеансу має атрибути з табл. 1.4. Приклад А7 виявив дефект: перевірка токена сервером сигналізації не перехоплює виняток JWEInvalid, тому довільне значення cookie дає відповідь 500 замість 401, хоча доступу до даних не надає. Під час перевірки маршрутів також зафіксовано, що сторінка /privacy для анонімного відвідувача переспрямовує на /login, хоча сторінка входу містить посилання на неї; це відповідає формулюванню ФВ-04, але робить правові сторінки недоступними до входу.

5.2.2 Трансляція з браузера. Табл. В.2 охоплює створення трансляції та встановлення медіаз'єднання стрімера (ФВ-06, ФВ-10 – ФВ-13, ФВ-17).

Сигнальна частина потоку 1.2 виконується повністю: маршрутизатор пропонує кодеки в порядку, визначеному в розділі 4, публікація останнього потоку переводить трансляцію в стан live, мініатюру змінює лише власник

(ФВ-17). Приклад Б5 показує, що заборону трансляцій для гостей реалізовано лише маршрутами вебзастосунку: кінцева точка створення трансляції не перевіряє роль сеансу.

5.2.3 Захоплення застосунку в настільному застосунку. Перевірки табл. В.3 стосуються режиму слідування (ФВ-09) і вибору джерела (ФВ-07, ФВ-08). Сценарій `follower.mjs` використовує таблицю процесів із процесом оболонки `explorer.exe`, процесом застосунку та стороннім процесом, а в Д1 – справжню таблицю процесів ОС стенда.

Інтервали в Д3, Д5 і Д7 узгоджуються з параметрами модуля: опитування раз на 1,5 с і очікування вихідного вікна 20 с плюс цикл опитування. Ці перевірки підтверджують логіку вибору вікна, але не саме захоплення, тому Д8 – Д10 виконуються вручну у Windows.

5.2.4 Перегляд трансляції. Приклади табл. В.4 перевіряють перелік трансляцій, приватні трансляції та підключення глядача (ФВ-05, ФВ-11, ФВ-14).

Сервер приховує приватну трансляцію з переліку й водночас надає її за ідентифікатором, що відповідає ФВ-11, а гість отримує медіапотік без входу через Google. Відтворення на боці браузера потребує ручної перевірки П6.

5.2.5 Завершення та відновлення трансляції. Табл. В.5 перевіряє вимоги ФВ-15 і ФВ-16, а також право керувати трансляцією.

Тривалість у 32 складається з п'яти інтервалів по 6 с і ще одного, на якому таймер фіксує вичерпання спроб, тобто 36 с, а не 30 с. Подію `streamEnd` сервер надсилає до запису причини завершення в базу даних. Приклад 34 виявив найсерйозніший дефект: обробник `WebSocket` стрімера перевіряє наявність сеансу, але не належність трансляції, тому будь-який сеанс може завершити чужу трансляцію.

5.3 Нефункціональне тестування

Нефункціональні характеристики з табл. 1.4 та результати їх перевірки наведено в табл. 5.2. Величини продуктивності залежать від обладнання й мережі

і не можуть бути отримані на стенді без графічного середовища, тому їх позначено як дані, що має надати автор.

Таблиця 5.2 – Результати нефункціонального тестування

Характеристика	Метод перевірки	Очікуваний результат	Фактичний результат
Затримка доставки	екранний секундомір у вікні, що транслюється; знімок екрана з вікнами стрімера й глядача; 10 вимірювань	[ПОТРЕБУЄ УТОЧНЕННЯ: цільове значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: середнє та максимальне значення, мережа]
Бітрейт і частота кадрів	chrome://webrtc-internals, вихідний потік: targetBitrate, framesPerSecond, frameWidth, qualityLimitationReason	для 1080p60 – бюджет 6,53 Мбіт/с, старт 3,27 Мбіт/с (табл. 4.4)	[ПОТРЕБУЄ УТОЧНЕННЯ: значення для 720p30, 1080p60, 1440p60, вихідної]
Завантаження клієнта	диспетчер завдань Windows, процеси настільного застосунку під час трансляції 1080p60	[ПОТРЕБУЄ УТОЧНЕННЯ: допустиме значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: CPU, GPU, модель процесора й відеокарти, кодер encoderImplementation]
Кількість глядачів	N вкладок або пристроїв на одній трансляції; навантаження процесу сервера (top)	відтворення без зупинок	[ПОТРЕБУЄ УТОЧНЕННЯ: N, CPU сервера, freezeCount у глядачів]
Сумісність	Windows, macOS, Linux × Chrome, Edge, Firefox, Safari: трансляція й перегляд	робота в підтримуваних поєднаннях	[ПОТРЕБУЄ УТОЧНЕННЯ: матриця результатів з версіями]
Узгодженість лімітів бітрейту	запуск сервера з MEDIASOUP_MAX_INCOMING_BITRATE=10000000	попередження про обмеження	попередження з порогом 21 600 000 біт/с
Обмеження доступу	приклади A1, A2, A6, A8	закриті маршрути недоступні без входу	відповідає
Захист сеансу	приклади A3, A4	HttpOnly, Secure, SameSite, префікс __Secure-	відповідає
Авторизація сервера сигналізації	приклади A7, B5, 34	відмова сторонньому чи недійсному сеансу	не відповідає: 500, 201, завершення чужої трансляції

Показники потоку визначено специфікацією статистики WebRTC [82]: `targetBitrate` – цільовий бітрейт кодера, `framesPerSecond` і `frameWidth` – фактичні частота кадрів і ширина кадру, `qualityLimitationReason` – причина зниження якості (`bandwidth`, `cpu` або `none`), `freezeCount` – кількість зупинок відтворення в глядача. Значення слід записувати після 60 с трансляції динамічного вмісту, оскільки кодер починає з половини бюджету бітрейту; затримку – як різницю показів секундоміра на одному знімку з вікнами стрімера й глядача. Перевірка узгодженості лімітів підтверджує, що сервер, запущений з вхідним лімітом, нижчим за стелю відправника 18 Мбіт/с із запасом 1,2, попереджає про обмеження найвищих профілів якості.

5.4 Виправлені дефекти та аналіз результатів

5.4.1 Дефекти, виявлені під час розроблення. Частину дефектів виявлено й виправлено до цього тестування; їхній перелік за історією фіксацій і завдань наведено в табл. 5.3 разом із дефектами, виявленими в підрозділах 5.2 – 5.3.

Таблиця 5.3 – Виявлені дефекти та їх стан

Дефект	Причина	Виправлення	Стан
Блочність зображення 1080p60 у динамічних сценах	транспорт стартував з вихідного бітрейту 1 Мбіт/с; ліміт сервера 10 Мбіт/с був нижчим за стелю відправника, тож найвищі профілі мовчки обмежувалися	спільний модуль <code>bitrate.ts</code> : стеля 18 Мбіт/с, ліміт сервера $1,2 \times$ стеля, старт – половина бюджету, підказка вмісту <code>motion</code>	виправлено (5b3b414)
Невідповідність гідратації React	ОС відвідувача і фокус вікна обчислювалися в браузері під час рендерингу	ОС визначається на сервері із заголовка <code>User-Agent</code> ; фокус – через <code>useSyncExternalStore</code> ; <code>suppressHydrationWarning</code> лише для <code><body></code>	виправлено (55ad31b)
Кожен другий вхід у настільному застосунку зависав	<code>@auth/core</code> оновлює <code>cookie</code> адреси повернення лише за зміни значення, а виклики виходу та входу передавали різні адреси	однакова адреса повернення в обох викликах, обробка відмови, сторожовий таймер, відсутність витoku обробника	виправлено (387cbc2)
Низький контраст	токени поверхонь	світла палітра, <code>var (- -</code>	виправлено

Дефект	Причина	Виправлення	Стан
світлої теми	світлої теми мали темні значення; <code>body</code> посилався на неіснуючу змінну	<code>stroke</code>) для тексту, колір небезпеки темної теми 3,6:1 → рівень AA	(0e82b2a)
Відсутній аватар	файл <code>/default-avatar.png</code> не існував, оптимізатор зображень повертав 400	компонент <code>Avatar</code> : ініціали або піктограма, заміна при помилці завантаження	виправлено (d8310fd)
Помилки типів після чистого встановлення	збій перезбирання нативного модуля без заголовків <code>PipeWire/X11</code> переривав <code>pnpm install</code>	перезбирання обов'язкове лише в CI, явний тип конфігурації, збирання залежностей перед <code>next build</code>	виправлено (af87ddf)
Підроблений токен → 500	неперехоплений <code>JWEInvalid</code>	перехоплення з відповіддю 401	відкрито (A7)
Гість створює трансляцію	REST не перевіряє роль	відмова для ролі <code>guest</code>	відкрито (B5)
Керування чужою трансляцією	WebSocket стрімера не перевіряє власника	перевірка <code>stream.userId</code> під час підключення	відкрито (34)

Виправлення вхідного потоку в настільному застосунку перевірялося під час розроблення допоміжним стендом із вісьмома послідовними спробами входу, скасуванням і повторним запуском; цей стенд до репозиторію не входить, тому повторна перевірка передбачена прикладом A10. Три відкриті дефекти мають спільну причину: сервер сигналізації автентифікує сеанс, але не авторизує дії, покладаючись на обмеження маршрутів вебзастосунку, які не діють при прямих зверненнях до його API. Крім того, під час зупинки стенда процес сервера сигналізації не завершився протягом 10 с після сигналу `SIGTERM`, а обробників сигналів у його коді немає; наслідки для перезапуску контейнера пом'якшує закриття незавершених трансляцій під час запуску сервера.

5.4.2 Аналіз результатів. Статична перевірка всіх шести пакетів пройшла без помилок, усі компоненти збираються. Зі 30 автоматизованих прикладів 27 дали очікуваний результат: 20 із 23 у сценарії `smoke.mjs` і 7 із 7 у `follower.mjs`. Підтверджено основні потоки сценаріїв 1.1, 1.4 і 1.5 на рівні протоколу, логіку режиму слідування та часові параметри відновлення. Невідповідності зосереджено в авторизації сервера сигналізації, а не у функціональності потокової передачі. Ручні приклади (A9, A10, B7 – B9, D8 – D10, П6, П7, 35) і вимірювання

табл. 5.2 потребують реального обладнання та є умовою остаточної оцінки ефективності функціонування й сумісності.

Висновки до розділу 5

Репозиторій програмної системи не містить автоматизованих тестів, тому перевірку побудовано на статичному аналізі, перевірці типів і збиранні, функціональних сценаріях проти запущених серверів і змодельованого оточення настільного застосунку, а також на ручних прикладах для операцій, які потребують захоплення екрана й реальних браузерів.

Статична перевірка і збирання всіх пакетів пройшли успішно. Автоматизовані функціональні приклади підтвердили обмеження доступу до сторінок, гостьовий вхід і захищені атрибути cookie сеансу, створення та публікацію трансляції, приватні трансляції, підключення глядача, сповіщення про втрату з'єднання та завершення трансляції за 36 с, а також логіку режиму слідування за вікнами застосунку.

Тестування виявило три дефекти авторизації сервера сигналізації: відповідь 500 на підроблений токен, створення трансляції гостем і можливість завершити чужу трансляцію. Їх слід усунути перевірками ролі та власника на сервері. Шість дефектів, виявлених під час розроблення, виправлено. Результати ручних прикладів і вимірювання затримки, бітрейту та навантаження позначено як дані, які має надати автор роботи.

6 ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ

6.1 Контрольний приклад застосування програмної системи

Контрольний приклад відтворює сценарії 1.3, 1.4 та 1.5 на розгорнутому сервісі за участю стрімера у Windows і гостя на іншому пристрої [ПОТРЕБУЄ УТОЧНЕННЯ: адреса сервісу, версії ОС, браузера і застосунку].

6.1.1 Встановлення настільного застосунку. Відвідувач, який не увійшов, бачить на головній сторінці кнопку «Download for» з логотипом своєї ОС, що завантажує інсталятор останнього випуску GitHub (рис. 6.1).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-download.png]

[РИСУНОК: скріншот]

Головна сторінка у браузері для відвідувача, який не увійшов: кнопка «Download for» з логотипом ОС і кнопка «Get started»

Заглушка: замінити файл thesis/screenshots/scr-download.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.1 – Головна сторінка для відвідувача з кнопкою завантаження настільного застосунку

Інсталятор NSIS встановлює застосунок для поточного користувача з вибором каталогу. Вікно застосунку відкриває той самий вебінтерфейс без кнопки завантаження, а меню в системному лотку містить пункти «Open», «Check for updates...» і «Quit».

6.1.2 Вхід у настільному застосунку. Кнопка «Log in» відкриває сторінку входу, а «Continue with Google» – вхід у системному браузері; застосунок показує стан очікування з кнопкою «Cancel» (рис. 6.2).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-desktop-sign-in.png]

[РИСУНОК: скріншот]

а - вікно настільного застосунку «Waiting for browser sign-in...» з кнопкою «Cancel»;
 б - вкладка системного браузера «Signed in. You can close this tab...» (два знімки поруч)

*Заглушка: замінити файл thesis/screenshots/scr-desktop-sign-in.png
 реальним скріншотом (PNG, ширина не менше 1600 px)*

Рисунок 6.2 – Вхід у настільному застосунку: а – очікування входу в браузері; б – сторінка завершення входу в браузері

Після вибору облікового запису вкладка браузера пропонує повернутися до застосунку, а він обмінює одноразовий код на сеанс і відкриває перелік трансляцій.

6.1.3 Вибір застосунку для трансляції. Кнопка «Start stream» у шапці відкриває сторінку трансляції, а «Pick source» – вікно вибору джерела з вкладками «Applications» і «Entire Screen» (рис. 6.3).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-source-picker.png]

[РИСУНОК: скріншот]

Вікно вибору джерела в настільному застосунку, вкладка «Applications»: мініатюри вікон з піктограмами застосунків

*Заглушка: замінити файл thesis/screenshots/scr-source-picker.png
 реальним скріншотом (PNG, ширина не менше 1600 px)*

Рисунок 6.3 – Вікно вибору джерела трансляції в настільному застосунку

Вікна показано мініатюрами з піктограмами застосунків. Після вибору вікна починається попередній перегляд зі станом «PREVIEW» і захоплення звуку лише цього застосунку; якщо звук недоступний, з'являється повідомлення про передавання тільки відео. Режим слідування увімкнено за замовчуванням.

6.1.4 Запуск приватної трансляції. Стрімер обирає якість і частоту кадрів (типово 720p і 30 кадр/с), вмикає приватність кнопкою із замком і натискає «Start stream». Стан змінюється на «LIVE», а поле «Stream link» показує посилання <адреса>/<ідентифікатор>/watch з кнопкою «Сору» (рис. 6.4).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-broadcast-live.png]

[РИСУНОК: скріншот]

Сторінка трансляції в настільному застосунку: стан «LIVE», увімкнений замок (приватна), кнопка режиму слідування, поле «Stream link» з посиланням

Заглушка: замінити файл thesis/screenshots/scr-broadcast-live.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.4 – Сторінка активної приватної трансляції в настільному застосунку

Приватна трансляція не потрапляє до переліку на головній сторінці, а змінити приватність після запуску не можна.

6.1.5 Перегляд трансляції гостем. Глядач відкриває посилання без входу, система створює гостьовий сеанс із випадковим іменем і відтворює трансляцію (рис. 6.5, а).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-watch-guest.png]

[РИСУНОК: скріншот]

а - сторінка перегляду в браузері глядача-гостя з елементами керування;
б - сторінка «The stream has ended» (два знімки поруч)

Заглушка: замінити файл thesis/screenshots/scr-watch-guest.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.5 – Сторінка перегляду в браузері глядача: а – під час трансляції; б – після завершення трансляції

Глядач керує гучністю, режимом «картинка в картинці» і повноекранним режимом кнопками або клавішами M, P і F.

6.1.6 Слідування за новим вікном застосунку. Коли застосунок, що транслюється, відкриває нове вікно (наприклад, програма запуску – вікно гри), джерело перемикається без дій стрімера з повідомленням «Switched to ...» і написом «Following: ...» (рис. 6.6).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-follow.png]

[РИСУНОК: скріншот]

Сторінка трансляції з повідомленням «Switched to ...» і написом «Following: ...»; поруч - вікно глядача з новим вікном застосунку

Заглушка: замінити файл thesis/screenshots/scr-follow.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.6 – Перемикання трансляції на нове вікно застосунку в режимі слідування

Глядач бачить нове вікно без перепідключення. Після його закриття трансляція повертається до початкового вікна («Back to ...»), а якщо зникло й воно, завершується з повідомленням «The captured app closed».

6.1.7 Завершення трансляції. Кнопка «Stop stream» завершує трансляцію, а глядач бачить сторінку «The stream has ended» з кнопкою повернення до переліку (рис. 6.5, б).

6.2 Експлуатаційні випробування програмної системи

6.2.1 Показник і методика. Мета роботи передбачає низьку затримку доставки зображення глядачеві, тому критерієм є [ПОТРЕБУЄ УТОЧНЕННЯ: показник мети з вступу та граничне значення]. Як і в підрозділі 5.3, у вікні застосунку працює екранний секундомір, а різницю його показів у вікнах стрімера й глядача фіксують на одному знімку 10 разів для кожних умов.

6.2.2 Результати. Результати випробувань наведено в табл. 6.1.

Таблиця 6.1 – Результати експлуатаційних випробувань

№	Умови	Затримка, мс (сер. / макс.)	Відповідність меті
1	[ПОТРЕБУЄ УТОЧНЕННЯ: мережа, якість, кількість глядачів]	[ПОТРЕБУЄ УТОЧНЕННЯ: значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: так / ні]
2	[ПОТРЕБУЄ УТОЧНЕННЯ: мережа, якість, кількість глядачів]	[ПОТРЕБУЄ УТОЧНЕННЯ: значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: так / ні]
3	[ПОТРЕБУЄ УТОЧНЕННЯ:	[ПОТРЕБУЄ	[ПОТРЕБУЄ

№	Умови	Затримка, мс (сер. / макс.)	Відповідність меті
	мережа, якість, кількість глядачів]	УТОЧНЕННЯ: значення]	УТОЧНЕННЯ: так / ні]

[ПОТРЕБУЄ УТОЧНЕННЯ: висновок за табл. 6.1; факти використання сервісу, лише якщо вони є]

Висновки до розділу 6

Контрольний приклад охоплює повний шлях користувача від встановлення настільного застосунку і входу через системний браузер до приватної трансляції вікна застосунку, перегляду гостем, автоматичного перемикання на нове вікно та завершення трансляції. Усі кроки виконуються засобами інтерфейсу.

Для перевірки мети визначено показник затримки доставки та методику його вимірювання; висновок про досягнення мети [ПОТРЕБУЄ УТОЧНЕННЯ: за результатами табл. 6.1].

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі розроблено вебсервіс stream-share для потокової передачі медіаконтенту з функцією захоплення окремих застосунків. Система складається з вебзастосунку, сервера сигналізації з медіасервером SFU та настільного застосунку для захоплення і розгорнута в мережі Інтернет. Усі поставлені задачі розв'язано.

Аналіз предметної області показав, що для інтерактивної трансляції від одного стрімера багатьом глядачам доцільно використовувати WebRTC у топології SFU, а вебсторінка не може програмно обрати вікно, отримати звук лише одного застосунку чи перейти до його нового вікна. Порівняння шести наявних рішень не виявило жодного, яке поєднує захоплення окремого застосунку з його власним звуком, слідування за вікнами цього застосунку та перегляд у браузері без облікового запису. На цій основі сформульовано 19 функціональних вимог, деталізовано п'ять сценаріїв використання і визначено нефункціональні вимоги за моделлю ISO/IEC 25010.

Під час планування трудомісткість оцінено методом точок варіантів використання: близько 103,4 точки, тобто близько 2069 люд.-год за коефіцієнта продуктивності 20 люд.-год. Календарний план з 11 етапів, виконаних з 27.05.2026 до 25.09.2026, відновлено за історією Git. Реєстр містить дев'ять ризиків, і для ключових технічних ризиків заходи реагування реалізовано в коді.

Спроектовано архітектуру з трьох застосунків і спільних пакетів протоколу, схем даних і конфігурації, що розгортаються контейнерами Docker Compose за зворотним проксі Caddy. Взаємодію описано діаграмами послідовності, діяльності та станів, модель даних – п'ятьма таблицями, узгодженими з Auth.js, а сервер сигналізації поділено на контролери, сервіси та репозиторії.

У реалізації настільний застосунок Electron перелічує вікна засобами desktopCapturer, визначає процес-власник вікна і захоплює звук лише цього процесу в окремому службовому процесі, а компонент sourceFollower кожні 1 500 мс перевіряє появу нових вікон застосунку та перемикає на них трансляцію.

Потокову передачу виконує mediasoup: процеси-обробники за кількістю ядер процесора, H.264 як основний кодек і VP8 як резервний, бюджет бітрейту до 18 Мбіт/с. Автентифікація поєднує вхід через Google, гостьові сеанси та вхід настільного застосунку через системний браузер за схемою PKCE; сеанс JWT перевіряють і вебзастосунок, і сервер сигналізації.

Репозиторій не містить автоматизованих тестів, тому перевірку виконано статичним аналізом, перевіркою типів, збиранням і окремими сценаріями. Усі шість пакетів пройшли статичну перевірку, 27 із 30 автоматизованих функціональних прикладів дали очікуваний результат. Три невідповідності стосуються авторизації дій на сервері сигналізації. Шість дефектів, виявлених під час розроблення, виправлено.

Контрольний приклад показав повний шлях користувача від встановлення настільного застосунку до приватної трансляції вікна застосунку, перегляду гостем і автоматичного перемикання на нове вікно. Функціональну частину мети досягнуто: трансляцію окремого застосунку з його власним звуком переглядають у браузері без встановлення програм. Висновок щодо кількісного показника затримки доставки [ПОТРЕБУЄ УТОЧНЕННЯ: за результатами експлуатаційних випробувань, табл. 6.1].

Практичне значення роботи полягає в тому, що сервіс доступний за адресою <https://streamshare.space>, а інсталятори настільного застосунку для Windows, macOS і Linux публікуються в репозиторії. Подальший розвиток передбачає усунення виявлених дефектів авторизації перевітками ролі та власника трансляції, впровадження автоматизованих тестів у конвеєр CI, підписання збірки для macOS з автоматичним оновленням і розподіл однієї трансляції між кількома маршрутизаторами та серверами mediasoup для збільшення кількості глядачів [83].

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Pantos R., May W. HTTP Live Streaming : RFC 8216. IETF, 2017. URL: <https://www.rfc-editor.org/rfc/rfc8216> (дата звернення: 05.10.2026).
2. Understand live streaming latency. YouTube Help. URL: <https://support.google.com/youtube/answer/7444635> (дата звернення: 05.10.2026).
3. Present during a video meeting. Google Meet Help. URL: <https://support.google.com/meet/answer/9308856> (дата звернення: 05.10.2026).
4. Go Live and Screen Share. Discord Support. URL: <https://support.discord.com/hc/en-us/articles/360040816151-Go-Live-and-Screen-Share> (дата звернення: 05.10.2026).
5. Screen Capture : W3C Working Draft 27 August 2026 / ed. by J.-I. Bruaroey, E. Alon. URL: <https://www.w3.org/TR/screen-capture/> (дата звернення: 05.10.2026).
6. electron-native-screenshare : Cross-platform native audio capture for Electron screen sharing with process-level isolation. GitHub. URL: <https://github.com/CilginSinek/electron-native-screenshare> (дата звернення: 05.10.2026).
7. WebRTC: Real-Time Communication in Browsers : W3C Recommendation 13 March 2025 / ed. by C. Jennings et al. URL: <https://www.w3.org/TR/webrtc/> (дата звернення: 05.10.2026).
8. Alvestrand H. Overview: Real-Time Protocols for Browser-Based Applications : RFC 8825. IETF, 2021. URL: <https://www.rfc-editor.org/rfc/rfc8825> (дата звернення: 05.10.2026).
9. Rescorla E. WebRTC Security Architecture : RFC 8827. IETF, 2021. URL: <https://www.rfc-editor.org/rfc/rfc8827> (дата звернення: 05.10.2026).
10. Westerlund M., Wenger S. RTP Topologies : RFC 7667. IETF, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7667> (дата звернення: 05.10.2026).
11. HTTP Live Streaming (HLS) authoring specification for Apple devices. Apple Developer Documentation. URL:

<https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices> (дата звернення: 05.10.2026).

12. Enabling Low-Latency HTTP Live Streaming (HLS). Apple Developer Documentation. URL:

<https://developer.apple.com/documentation/http-live-streaming/enabling-low-latency-http-live-streaming-hls> (дата звернення: 05.10.2026).

13. Enhanced RTMP (V2) : Document Version v2-2026-01-31-r2 / Veovera Software Organization. URL:

<https://github.com/veovera/enhanced-rtmp/blob/main/docs/enhanced/enhanced-rtmp-v2.md> (дата звернення: 05.10.2026).

14. Video Broadcast. Twitch Developers. URL: <https://dev.twitch.tv/docs/video-broadcast/> (дата звернення: 05.10.2026).

15. RTP: A Transport Protocol for Real-Time Applications : RFC 3550 / H. Schulzrinne et al. IETF, 2003. URL: <https://www.rfc-editor.org/rfc/rfc3550> (дата звернення: 05.10.2026).

16. Keranen A., Holmberg C., Rosenberg J. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal : RFC 8445. IETF, 2018. URL: <https://www.rfc-editor.org/rfc/rfc8445> (дата звернення: 05.10.2026).

17. Uberti J., Jennings C., Rescorla E. JavaScript Session Establishment Protocol (JSEP) : RFC 9429. IETF, 2024. URL: <https://www.rfc-editor.org/rfc/rfc9429> (дата звернення: 05.10.2026).

18. Comparative Study of WebRTC Open Source SFUs for Video Conferencing / E. Andre et al. 2018 Principles, Systems and Applications of IP Telecommunications (IPTComm). IEEE, 2018. P. 1–8. DOI: 10.1109/IPTCOMM.2018.8567642.

19. Overview. mediasoup documentation. URL: <https://mediasoup.org/documentation/overview/> (дата звернення: 05.10.2026).

20. MediaDevices: getDisplayMedia() method. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getDisplayMedia> (дата звернення: 05.10.2026).

21. Privacy-preserving screen sharing controls. Chrome for Developers. URL: <https://developer.chrome.com/docs/web-platform/screen-sharing-controls> (дата звернення: 05.10.2026).

22. desktopCapturer. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/api/desktop-capturer> (дата звернення: 05.10.2026).

23. session. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/api/session> (дата звернення: 05.10.2026).

24. OBS Studio : Free and open source software for video recording and live streaming. OBS Project. URL: <https://obsproject.com/> (дата звернення: 05.10.2026).

25. Window Capture Sources. OBS Knowledge Base. URL: <https://obsproject.com/kb/window-capture-sources> (дата звернення: 05.10.2026).

26. Application Audio Capture Guide. OBS Knowledge Base. URL: <https://obsproject.com/kb/application-audio-capture-guide> (дата звернення: 05.10.2026).

27. Get started with live streaming. YouTube Help. URL: <https://support.google.com/youtube/answer/2474026> (дата звернення: 05.10.2026).

28. Change video privacy settings. YouTube Help. URL: <https://support.google.com/youtube/answer/157177> (дата звернення: 05.10.2026).

29. Freigeben des Bildschirms oder Desktops in Zoom. Zoom Support. URL: https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0060609 (дата звернення: 05.10.2026).

30. Overview. Parsec Support. URL: <https://support.parsec.app/hc/en-us/articles/32361354307348-Overview> (дата звернення: 05.10.2026).

31. Hosting and Permissions. Parsec Support. URL: <https://support.parsec.app/hc/en-us/articles/32381747079572-Hosting-and-Permissions> (дата звернення: 05.10.2026).

32. Hardware and Software Compatibility. Parsec Support. URL: <https://support.parsec.app/hc/en-us/articles/32381568346644-Hardware-and-Software-Compatibility> (дата звернення: 05.10.2026).

33. Use the Web App (browser). Parsec Support. URL: <https://support.parsec.app/hc/en-us/articles/32381650129300-Use-the-Web-App-browser> (дата звернення: 05.10.2026).

34. OMG Unified Modeling Language (OMG UML) : Version 2.5.1. Object Management Group, 2017. URL: <https://www.omg.org/spec/UML/2.5.1> (дата звернення: 05.10.2026).

35. ISO/IEC 25010. ISO 25000 Portal. URL: <https://iso25000.com/en/iso-25000-standards/iso-25010> (дата звернення: 05.10.2026).

36. Auto Update. electron-builder. URL: <https://www.electron.build/docs/features/auto-update> (дата звернення: 05.10.2026).

37. Denniss W., Bradley J. OAuth 2.0 for Native Apps : RFC 8252 (BCP 212). IETF, 2017. URL: <https://www.rfc-editor.org/rfc/rfc8252> (дата звернення: 05.10.2026).

38. Sakimura N., Bradley J., Agarwal N. Proof Key for Code Exchange by OAuth Public Clients : RFC 7636. IETF, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7636> (дата звернення: 05.10.2026).

39. OAuth 2.0 for iOS & Desktop Apps. Google for Developers. URL: <https://developers.google.com/identity/protocols/oauth2/native-app> (дата звернення: 05.10.2026).

40. Azzeh M., Nassif A. B. Analyzing the Relationship between Project Productivity and Environment Factors in the Use Case Points Method : preprint. arXiv:1705.09920. 2017. 24 p. URL: <https://arxiv.org/abs/1705.09920> (дата звернення: 05.10.2026).

41. Azad B. Upcoming security changes to Google's OAuth 2.0 authorization endpoint in embedded webviews. Google Developers Blog. 2021. 29 June. URL: <https://developers.googleblog.com/en/upcoming-security-changes-to-googles-oauth-20-authorization-endpoint-in-embedded-webviews/> (дата звернення: 05.10.2026).

42. Workspace. pnpm Documentation. URL: <https://pnpm.io/workspaces> (дата звернення: 05.10.2026).

43. Fette I., Melnikov A. The WebSocket Protocol : RFC 6455. IETF, 2011. URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата звернення: 05.10.2026).

44. Design. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/mediasoup/design/> (дата звернення: 05.10.2026).

45. Process Model. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/tutorial/process-model> (дата звернення: 05.10.2026).

46. Drizzle ORM : Overview. Drizzle Team. URL: <https://orm.drizzle.team/docs/overview> (дата звернення: 05.10.2026).

47. Zod : TypeScript-first schema validation with static type inference. URL: <https://zod.dev/> (дата звернення: 05.10.2026).

48. How Compose works. Docker Docs. URL: <https://docs.docker.com/compose/intro/compose-application-model/> (дата звернення: 05.10.2026).

49. Automatic HTTPS. Caddy Documentation. URL: <https://caddyserver.com/docs/automatic-https> (дата звернення: 05.10.2026).

50. About PostgreSQL. The PostgreSQL Global Development Group. URL: <https://www.postgresql.org/about/> (дата звернення: 05.10.2026).

51. mediasoup-client. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/mediasoup-client/> (дата звернення: 05.10.2026).

52. File-system conventions: proxy.js. Next.js Docs. URL: <https://nextjs.org/docs/app/api-reference/file-conventions/proxy> (дата звернення: 05.10.2026).

53. Session strategies. Auth.js. URL: <https://authjs.dev/concepts/session-strategies> (дата звернення: 05.10.2026).

54. Fastify Documentation. OpenJS Foundation. URL: <https://fastify.dev/docs/latest/> (дата звернення: 05.10.2026).
55. @fastify/websocket : WebSocket support for Fastify. GitHub. URL: <https://github.com/fastify/fastify-websocket> (дата звернення: 05.10.2026).
56. Context Isolation. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/tutorial/context-isolation> (дата звернення: 05.10.2026).
57. utilityProcess. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/api/utility-process> (дата звернення: 05.10.2026).
58. TypeScript : JavaScript with syntax for types. Microsoft. URL: <https://www.typescriptlang.org/> (дата звернення: 05.10.2026).
59. Next.js Docs. Vercel. URL: <https://nextjs.org/docs> (дата звернення: 05.10.2026).
60. React v19. React Blog. 2024. 5 December. URL: <https://react.dev/blog/2024/12/05/react-19> (дата звернення: 05.10.2026).
61. Janus WebRTC Server : Documentation. Meetecho. URL: <https://janus.conf.meetecho.com/docs/> (дата звернення: 05.10.2026).
62. About LiveKit. LiveKit Docs. URL: <https://docs.livekit.io/intro/about/> (дата звернення: 05.10.2026).
63. Introduction. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/> (дата звернення: 05.10.2026).
64. Webview Versions. Tauri 2 Documentation. URL: <https://v2.tauri.app/reference/webview-versions/> (дата звернення: 05.10.2026).
65. Tauri Architecture. Tauri 2 Documentation. URL: <https://v2.tauri.app/concept/architecture/> (дата звернення: 05.10.2026).
66. What is Prisma ORM? Prisma Documentation. URL: <https://www.prisma.io/docs/orm/overview/introduction/what-is-prisma> (дата звернення: 05.10.2026).

67. electron-builder : A complete solution to package and build a ready for distribution Electron app. URL: <https://www.electron.build/> (дата звернення: 05.10.2026).

68. MessagePorts in Electron. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/tutorial/message-ports> (дата звернення: 05.10.2026).

69. AudioWorklet. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/API/AudioWorklet> (дата звернення: 05.10.2026).

70. mediasoup-client :: API. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/mediasoup-client/api/> (дата звернення: 05.10.2026).

71. mediasoup :: API. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/mediasoup/api/> (дата звернення: 05.10.2026).

72. Roach A. B. WebRTC Video Processing and Codec Requirements : RFC 7742. IETF, 2016. URL: <https://www.rfc-editor.org/rfc/rfc7742> (дата звернення: 05.10.2026).

73. RTP Payload Format for H.264 Video : RFC 6184 / Y.-K. Wang et al. IETF, 2011. URL: <https://www.rfc-editor.org/rfc/rfc6184> (дата звернення: 05.10.2026).

74. MediaStreamTrack Content Hints : W3C Working Draft 19 September 2025. URL: <https://www.w3.org/TR/mst-content-hint/> (дата звернення: 05.10.2026).

75. Google. Auth.js. URL: <https://authjs.dev/getting-started/providers/google> (дата звернення: 05.10.2026).

76. Credentials. Auth.js. URL: <https://authjs.dev/getting-started/authentication/credentials> (дата звернення: 05.10.2026).

77. Set-Cookie header. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie> (дата звернення: 05.10.2026).

78. Hooks. Fastify. URL: <https://fastify.dev/docs/latest/Reference/Hooks/> (дата звернення: 05.10.2026).

79. drizzle-kit push. Drizzle ORM. URL: <https://orm.drizzle.team/docs/drizzle-kit-push> (дата звернення: 05.10.2026).

80. Theme variables. Tailwind CSS. URL: <https://tailwindcss.com/docs/theme> (дата звернення: 05.10.2026).

81. next-themes : Perfect Next.js dark mode in 2 lines of code. GitHub. URL: <https://github.com/pacocoursey/next-themes> (дата звернення: 05.10.2026).

82. Identifiers for WebRTC's Statistics API : W3C Candidate Recommendation Draft 25 September 2025 / ed. by H. Alvestrand, V. Singh, H. Boström. URL: <https://www.w3.org/TR/webrtc-stats/> (дата звернення: 05.10.2026).

83. Scalability. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/scalability/> (дата звернення: 05.10.2026).

ДОДАТОК А

Лістинг програми

У додатку наведено вихідний код ключових модулів програмної системи stream-share у стані фіксації 7c2f481 репозиторію. Пропущені фрагменти (імпорти, допоміжний код, а також код, уже наведений в основній частині роботи) позначено коментарем `// ....`. Вихідні тексти всіх діаграм PlantUML, наведених у роботі, зберігаються в каталозі `thesis/diagrams` репозиторію поруч із відповідними зображеннями.

Далі наведено файл `apps/desktop/src/main/sourceFollower.ts` (лістинг А.1).

Лістинг А.1 – Клас `SourceFollower` настільного застосунку

```
import log from 'electron-log/main';
import type { FollowState, SourceChanged } from
'../conveyor/schemas/stream.schema';
import { findFamilyRoot, isInFamily, isShellProcess } from './processFamily';
import type { ProcessTable } from './processTable';

export const POLL_INTERVAL_MS = 1500;
export const RETURN_GRACE_MS = 4000;
export const RETURN_WAIT_MS = 20_000;

export type WindowSource = { id: string; name: string };

export type FollowerDeps = {
  listWindows: () => Promise<WindowSource[]>;
  listProcesses: () => Promise<ProcessTable>;
  resolvePid: (sourceId: string) => number;
  setSelectedSourceId: (sourceId: string) => void;
  emit: (payload: SourceChanged) => void;
};

type Anchor = {
  sourceId: string;
  pid: number;
  exe: string;
  familyRoot: number;
  startedAt: Date;
};

type Active = { sourceId: string; pid: number; name: string };
type PidWindow = WindowSource & { pid: number };

export class SourceFollower {
  private enabled = true;
  private anchor: Anchor | null = null;
  private active: Active | null = null;
  private knownWindowIds = new Set<string>();
  private needsReseed = false;
  private pendingReturnSince: number | null = null;
  private lastReturnAt = 0;
  private lastError: string | null = null;
  private timer: NodeJS.Timeout | null = null;
  private ticking = false;
```

```

private generation = 0;

constructor(private readonly deps: FollowerDeps) {}

getState(): FollowState {
  return {
    enabled: this.enabled,
    following: this.active !== null,
    activeName: this.active?.name ?? null,
    lastError: this.lastError,
  };
}

setEnabled(enabled: boolean): void {
  this.enabled = enabled;
  if (!enabled) {
    this.stopPolling();
    this.active = null;
    this.pendingReturnSince = null;
    return;
  }
  this.lastError = null;
  if (this.anchor) {
    this.needsReseed = true;
    this.startPolling();
  }
}

async onSourcePicked(sourceId: string): Promise<void> {
  this.stop();
  const gen = this.generation;
  if (!sourceId.startsWith('window:')) return;

  const pid = this.deps.resolvePid(sourceId);
  if (!pid) {
    log.warn('[follower] could not resolve a pid for', sourceId);
    return;
  }

  try {
    const table = await this.deps.listProcesses();
    if (gen !== this.generation) return;
    if (isShellProcess(table, pid)) {
      log.info('[follower] not following a shell process window',
table.get(pid)?.name);
      return;
    }
    const familyRoot = findFamilyRoot(table, pid);
    if (familyRoot === null) {
      log.warn('[follower] pid not in process table', pid);
      return;
    }
    const windows = await this.deps.listWindows();
    if (gen !== this.generation) return;
    this.knownWindowIds = new Set(windows.map((w) => w.id));
    this.anchor = {
      sourceId,
      pid,
      exe: table.get(pid)?.name ?? '',
      familyRoot,
      startedAt: new Date(),
    };
    log.info('[follower] anchored', { sourceId, pid, familyRoot:
table.get(familyRoot)?.name });
  }
}

```

```

        if (this.enabled) this.startPolling();
    } catch (err) {
        if (gen === this.generation) this.fail(err);
    }
}

async resolveEnded(): Promise<SourceChanged> {
    const anchor = this.anchor;
    if (!this.enabled || !anchor) return { reason: 'lost' };
    const gen = this.generation;
    let windows: WindowSource[] = [];
    try {
        windows = await this.deps.listWindows();
    } catch (err) {
        log.warn('[follower] listWindows failed in resolveEnded', err);
    }
    if (gen !== this.generation) return { reason: 'noop' };
    this.knownWindowIds = new Set(windows.map((w) => w.id));

    if (this.active) return this.beginReturn(windows);
    if (this.pendingReturnSince !== null) return { reason: 'noop' };
    const anchorListed = windows.some((w) => w.id === anchor.sourceId);
    if (anchorListed && Date.now() - this.lastReturnAt < RETURN_GRACE_MS) return {
reason: 'noop' };

    log.info('[follower] captured window closed, stopping');
    this.stop();
    return { reason: 'lost' };
}

stop(): void {
    this.generation += 1;
    this.stopPolling();
    this.anchor = null;
    this.active = null;
    this.pendingReturnSince = null;
    this.knownWindowIds.clear();
}

private startPolling(): void {
    if (this.timer) return;
    this.timer = setInterval(() => void this.tick(), POLL_INTERVAL_MS);
}

private stopPolling(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
}

private async tick(): Promise<void> {
    if (this.ticking || !this.anchor) return;
    this.ticking = true;
    const gen = this.generation;
    try {
        const windows = await this.deps.listWindows();
        if (gen !== this.generation) return;
        const ids = new Set(windows.map((w) => w.id));
        if (this.needsReseed) {
            this.knownWindowIds = ids;
            this.needsReseed = false;
            return;
        }
    }
    const appeared = windows.filter((w) => !this.knownWindowIds.has(w.id));
    const gone = [...this.knownWindowIds].filter((id) => !ids.has(id));

```

```

    this.knownWindowIds = ids;

    if (this.active && gone.includes(this.active.sourceId)) {
      const result = this.beginReturn(windows);
      if (result.reason === 'return') this.deps.emit(result);
    }

    if (appeared.length > 0) await this.considerNewWindows(appeared, gen);
    if (gen !== this.generation) return;

    if (
      this.pendingReturnSince !== null &&
      Date.now() - this.pendingReturnSince > RETURN_WAIT_MS
    ) {
      log.info('[follower] anchor did not come back, giving up');
      this.stop();
      this.deps.emit({ reason: 'lost' });
    }
  } catch (err) {
    if (gen === this.generation) this.fail(err);
  } finally {
    this.ticking = false;
  }
}

private beginReturn(windows: WindowSource[]): SourceChanged {
  this.active = null;
  const target = this.findReturnTarget(windows);
  if (target) return this.completeReturn(target);
  if (this.pendingReturnSince === null) {
    this.pendingReturnSince = Date.now();
    log.info('[follower] anchor not visible, waiting for it to come back');
  }
  return { reason: 'noop' };
}

private findReturnTarget(windows: WindowSource[]): WindowSource | undefined {
  const anchor = this.anchor;
  if (!anchor) return undefined;
  return (
    windows.find((w) => w.id === anchor.sourceId) ??
    windows.find((w) => this.deps.resolvePid(w.id) === anchor.pid)
  );
}

private completeReturn(target: WindowSource): SourceChanged {
  this.pendingReturnSince = null;
  this.active = null;
  this.deps.setSelectedSourceId(target.id);
  if (this.anchor) this.anchor = { ...this.anchor, sourceId: target.id };
  this.lastReturnAt = Date.now();
  log.info('[follower] returned to', target.name, target.id);
  return { reason: 'return', sourceId: target.id, name: target.name };
}

private async considerNewWindows(appeared: WindowSource[], gen: number):
Promise<void> {
  const anchor = this.anchor;
  if (!anchor) return;

  const withPid: PidWindow[] = appeared
    .map((w) => ({ ...w, pid: this.deps.resolvePid(w.id) }))
    .filter((w) => w.pid !== 0);

```

```

const ownWindow = withPid.find((w) => w.pid === anchor.pid);
if (ownWindow && this.pendingReturnSince !== null) {
  this.deps.emit(this.completeReturn(ownWindow));
  return;
}

const others = withPid.filter((w) => w.pid !== anchor.pid);
if (others.length === 0) return;

const table = await this.deps.listProcesses();
if (gen !== this.generation) return;
const anchorAlive = table.has(anchor.pid);

const candidates: (PidWindow & { createdAt: Date })[] = [];
for (const w of others) {
  const info = table.get(w.pid);
  if (!info) continue;
  if (!isInFamily(table, w.pid, anchor.familyRoot)) continue;
  if (info.createdAt <= anchor.startedAt) continue;

  const sameExe = info.name.toLowerCase() === anchor.exe.toLowerCase();
  if (sameExe && !anchorAlive) {
    // The anchor process was replaced (League's "close client during game").
    this.anchor = { ...anchor, sourceId: w.id, pid: w.pid };
    log.info('[follower] anchor relaunched as', w.name, w.id);
    if (this.pendingReturnSince !== null)
this.deps.emit(this.completeReturn(w));
    return;
  }
  if (sameExe) continue;
  candidates.push({ ...w, createdAt: info.createdAt });
}
if (candidates.length === 0) return;

candidates.sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime());
const winner = candidates[0]!;
if (this.active?.sourceId === winner.id) return;

this.pendingReturnSince = null;
this.active = { sourceId: winner.id, pid: winner.pid, name: winner.name };
this.deps.setSelectedSourceId(winner.id);
log.info('[follower] following', winner.name, winner.id);
this.deps.emit({ reason: 'follow', sourceId: winner.id, name: winner.name });
}

private fail(err: unknown): void {
  const message = err instanceof Error ? err.message : String(err);
  log.error('[follower] disabled after error:', message);
  this.stop();
  this.enabled = false;
  this.lastError = message;
  this.deps.emit({ reason: 'error', message });
}
}

```

Далі наведено файл apps/signaling/src/services/mediasoup.service.ts

(ЛІСТИНГ А.2).

ЛІСТИНГ А.2 – Клас MediasoupService сервера сигналізації (скорочено)

```

import { env } from '@stream-share/env/signaling';
import {
  MAX_VIDEO_BITRATE,

```

```

    MIN_SERVER_INCOMING_BITRATE,
    startBitrateFor,
  } from '@stream-share/shared';
import mediasoup from 'mediasoup';
import type { Router, WebRtcTransport, Worker } from 'mediasoup/types';

export type TransportRole = 'send' | 'recv';

const RTC_MIN_PORT = env.MEDIASOUP_RTC_MIN_PORT;
const RTC_MAX_PORT = env.MEDIASOUP_RTC_MAX_PORT;
const NUM_WORKERS = env.MEDIASOUP_NUM_WORKERS;
const ANNOUNCED_IP = env.MEDIASOUP_ANNOUNCED_IP;
const INITIAL_OUTGOING_BITRATE = env.MEDIASOUP_INITIAL_OUTGOING_BITRATE;
const MAX_INCOMING_BITRATE = env.MEDIASOUP_MAX_INCOMING_BITRATE;
/** `x-google-start-bitrate` is kbit/s, so the same start as BWE expressed in its
unit. */
const START_BITRATE_KBPS = Math.round(INITIAL_OUTGOING_BITRATE / 1000);

export class MediasoupService {
  private readonly workers: Worker[] = [];
  constructor() {}

  // ...
  async createWorkers(): Promise<Worker[]> {
    for (let i = 0; i < NUM_WORKERS; i++) {
      const worker = await mediasoup.createWorker({
        logLevel: 'warn',
        rtcMinPort: RTC_MIN_PORT,
        rtcMaxPort: RTC_MAX_PORT,
      });
      this.workers.push(worker);
    }

    return this.workers;
  }

  async createRouter(): Promise<Router> {
    const worker = await this.pickLeastLoadedWorker();

    const router = await worker.createRouter({
      mediaCodecs: [
        {
          kind: 'audio',
          mimeType: 'audio/opus',
          clockRate: 48000,
          channels: 2,
        },
      ],
    });
    /**
     * H.264 first: it is the only codec every browser and Electron encode
     * in hardware, and hardware encoding is what makes 1080p60, 1440p60
     * and 4K source streams sustainable. Chrome's software VP8 (libvpx)
     * cannot hold those modes and silently degrades them.
     *
     * Baseline (42001f) before Constrained Baseline (42e01f), deliberately:
     * Chromium's hardware encoder factory only advertises Baseline, Main
     * and High, while Constrained Baseline comes solely from its OpenH264
     * software encoder. Offering 42e01f first therefore forces software
     * encoding in Chrome, Edge and Electron (verified against Chrome 152
     * with NVENC). Baseline is still decodable by the Chromium family and
     * Firefox; 42e01f stays as the fallback for endpoints without it.
     *
     * `level-asymmetry-allowed` lets endpoints encode above level 3.1
     * (1440p60 is level 5.1) without renegotiation.
     */
  }

```

```

    {
      kind: 'video',
      mimeType: 'video/H264',
      clockRate: 90000,
      parameters: {
        'packetization-mode': 1,
        'profile-level-id': '42001f',
        'level-asymmetry-allowed': 1,
        'x-google-start-bitrate': START_BITRATE_KBPS,
      },
    },
    {
      kind: 'video',
      mimeType: 'video/H264',
      clockRate: 90000,
      parameters: {
        'packetization-mode': 1,
        'profile-level-id': '42e01f',
        'level-asymmetry-allowed': 1,
        'x-google-start-bitrate': START_BITRATE_KBPS,
      },
    },
  ],
  /** Software fallback for endpoints without H.264. */
  {
    kind: 'video',
    mimeType: 'video/VP8',
    clockRate: 90000,
    parameters: {
      'x-google-start-bitrate': START_BITRATE_KBPS,
    },
  },
]);
});

return router;
}

async createTransport(
  router: Router,
  direction: TransportRole,
  videoMaxBitrate?: number,
): Promise<WebRtcTransport> {
  const transport = await router.createWebRtcTransport({
    listenIps: [{ ip: '0.0.0.0', announcedIp: ANNOUNCED_IP }],
    enableUdp: true,
    enableTcp: true,
    preferUdp: true,
    initialAvailableOutgoingBitrate:
this.initialOutgoingBitrate(videoMaxBitrate),
    appData: { direction },
  });

  return transport;
}

/**
 * Where bandwidth estimation starts on a transport (bit/s). When the stream's
 * video budget is known – viewer transports, created after the broadcaster
 * produced – it is the same fraction of that budget the sender's encoder opens
 * at, so a 4K stream does not start a viewer at a 1080p estimate. Never below
 * the configured floor, never above what we accept from the broadcaster.
 */
private initialOutgoingBitrate(videoMaxBitrate?: number): number {
  if (!videoMaxBitrate || !Number.isFinite(videoMaxBitrate) || videoMaxBitrate

```

```

<= 0) {
    return INITIAL_OUTGOING_BITRATE;
}
const proportional = startBitrateFor(Math.min(videoMaxBitrate,
MAX_VIDEO_BITRATE));
return Math.max(proportional, INITIAL_OUTGOING_BITRATE);
}

/**
 * Caps what the remote endpoint is told it may send us (REMB / transport-cc).
 * Only meaningful once the transport is connected, so callers apply it there.
 */
async limitIncomingBitrate(transport: WebRtcTransport): Promise<void> {
    await transport.setMaxIncomingBitrate(MAX_INCOMING_BITRATE);
}

/**
 * The invariant of ADR 0001: the cap we advertise to the broadcaster must stay
 * at or above the sender's own ceiling plus overhead, or we silently clamp a
 * mode the app lets people pick. Lowering it is a legitimate operator choice on
 * a thin uplink, so this warns instead of refusing to start.
 */
static bitrateConfigWarning(): string | null {
    if (MAX_INCOMING_BITRATE >= MIN_SERVER_INCOMING_BITRATE) return null;
    return (
        `MEDIASOUP_MAX_INCOMING_BITRATE=${MAX_INCOMING_BITRATE} is below ` +
        `${MIN_SERVER_INCOMING_BITRATE} (the ${MAX_VIDEO_BITRATE} bit/s sender
ceiling plus ` +
        'overhead): the SFU will clamp broadcasters in the highest quality modes.'
    );
}
}

```

Далі наведено файл `apps/signaling/src/controllers/streams.controller.ts` (лістинг А.3).

Лістинг А.3 – Маршрути та обробники WebSocket класу `StreamsController` (фрагмент)

```

// ...

this.app.register(
    (app) => {
        // eslint-disable-next-line @typescript-eslint/no-explicit-any
        app.decorateRequest('context', null as any);
        app.addHook('onRequest', async (req) => {
            req.context = {};
        });

        app.get<{ Params: { streamId: string } }>(
            '/broadcast',
            {
                websocket: true,
                preValidation: validateWsJwt,
            },
            this.handleBroadcast,
        );
        app.get<{ Params: { streamId: string } }>(
            '/watch',
            {
                websocket: true,
                preValidation: async (req, reply) => {

```

```

        await validateJwt(req, reply);
        req.context.userId ??= crypto.randomUUID();
      },
    },
    this.handleWatch,
  );
},
{ prefix: '/ws/streams/:streamId' },
);
}
// ...
private handleBroadcast = (
  socket: WebSocket,
  req: FastifyRequest<{ Params: { streamId: string } }>,
) => {
  const streamId = req.params.streamId;
  let status: StreamStatus.Connecting | StreamStatus.Reconnecting =
StreamStatus.Connecting;
  const streamContext = this.streamsService.getContext(streamId);
  if (streamContext?.reconnectionId) {
    status = StreamStatus.Reconnecting;
    clearInterval(streamContext.reconnectionId);
    this.streamsService.setContext(streamId, { ...streamContext, reconnectionId:
null });
  }
  void this.streamsService.update({ id: streamId, status });

  socket.on('message', (message) => {
    this.app.log.info(message.toString());
    void this.handleStreamerMessage({
      message: message.toString(),
      socket,
      streamId,
    });
  });

  socket.on('close', () => {
    this.app.log.info(`Streamer socket of 'stream:${streamId}' closed`);
    void this.handleStreamerSocketClose(streamId);
  });
};

private handleWatch = (
  socket: WebSocket,
  req: FastifyRequest<{ Params: { streamId: string } }>,
) => {
  const userId = req.context.userId as string;
  const streamId = req.params.streamId;
  const context = { viewerId: userId, streamId };

  socket.on('message', (message) => {
    this.app.log.info(message.toString());
    void this.handleViewerMessage({
      message: message.toString(),
      socket,
      context,
    });
  });

  socket.on('close', () => {
    this.app.log.info(
      `Viewer 'user:${context.viewerId}' socket of 'stream:${context.streamId}'
closed`,
    );
  });
};

```

```

        void this.handleViewerSocketClose(context);
    });
};

private handleStreamerMessage = async (data: {
    message: string;
    socket: WebSocket;
    streamId: string;
}) => {
    const { message, socket, streamId } = data;
    const { method, params, requestId } = parseMessage(message) as
WsRequestEnvelope;

    let streamContext;
    let stream;
    try {
        stream = await this.streamsService.requireStream(streamId);
        streamContext = await this.streamsService.requireContext(streamId);
    } catch (error) {
        if (error instanceof EntityNotFoundError) {
            socket.send(constructErrorResponse(method, { code: 404, msg: error.message
}, requestId));
        } else if (error instanceof Error) {
            socket.send(constructErrorResponse(method, { code: 404, msg: error.message
}, requestId));
        }
        this.app.log.error(error);
        return;
    }

    try {
        switch (method) {
            case StreamerActions.GetRtpCapabilities: {
                return socket.send(
                    constructSuccessResponse(method, streamContext.router.rtpCapabilities,
requestId),
                );
            }
            case CommonActions.CreateTransport: {
                const transport = await this.streamerService.createTransport(streamId,
streamContext);
                return socket.send(
                    constructSuccessResponse(
                        method,
                        {
                            id: transport.id,
                            iceParameters: transport.iceParameters,
                            dtlsParameters: transport.dtlsParameters,
                            iceCandidates: transport.iceCandidates,
                        },
                        requestId,
                    ),
                );
            }
            case CommonActions.ConnectTransport: {
                await this.streamerService.connectTransport(streamContext,
params.dtlsParameters);
                return socket.send(constructSuccessResponse(method, null, requestId));
            }
            case StreamerActions.Produce: {
                const { last, ...restParams } = params;
                const producer = await this.streamerService.produce(streamId,
streamContext, restParams);
                if (last) {

```

```

        await this.streamsService.update({ id: streamId, status:
StreamStatus.Live });
    }

    if (stream.status === 'reconnecting') {
        this.notifyViewers(streamId, {
            type: 'event',
            name: WsEvents.StreamerReconnected,
            data: { producerId: producer.id },
        });
    }

    return socket.send(
        constructSuccessResponse(method, { producerId: producer.id },
requestId),
    );
}
case StreamerActions.EndStream: {
    await this.streamsService.update({
        id: streamId,
        status: StreamStatus.Ended,
        endReason: StreamEndReason.StreamerStop,
        endedAt: new Date(),
    });
    return socket.send(constructSuccessResponse(method, null, requestId));
}
} catch (error) {
    if (error instanceof Error) {
        socket.send(
            constructErrorResponse(method, { code: 'unknown', msg: error.message },
requestId),
        );
        this.app.log.error(error);
    }
}
};

// ...
private handleStreamerSocketClose = async (streamId: string): Promise<void> => {
    let reconnectingAttempts = 0;
    let stream;
    try {
        stream = await this.streamsService.getOne(streamId);
    } catch {
        return this.streamerService.releaseResources(streamId);
    }
    if (stream.status === StreamStatus.Ended) {
        this.notifyViewers(streamId, { type: 'event', name: WsEvents.StreamEnd,
data: null });
        return this.streamerService.releaseResources(streamId);
    }

    const reconnectionId = setInterval(() => {
        if (reconnectingAttempts++ >= StreamsController.MAX_ATTEMPTS) {
            void this.streamsService
                .update({
                    id: streamId,
                    status: StreamStatus.Ended,
                    endReason: StreamEndReason.Timeout,
                    endedAt: new Date(),
                })
                .then(() => this.streamerService.releaseResources(streamId));
            this.notifyViewers(streamId, { type: 'event', name: WsEvents.StreamEnd,

```

```

data: null });

    return clearInterval(reconnectionId);
  }
}, 6000);
const streamContext = this.streamsService.getContext(streamId);
if (streamContext) {
  this.streamsService.setContext(streamId, { ...streamContext,
reconnectionId });
}

this.notifyViewers(streamId, { type: 'event', name:
WsEvents.StreamerDisconnect, data: null });
};

private handleViewerSocketClose = async (data: {
  viewerId: string;
  streamId: string;
}): Promise<void> => {
  const { viewerId, streamId } = data;
  return this.viewerService.releaseResources(viewerId, streamId);
};

private notifyViewers(streamId: string, event: WsEvent) {
  const streamContext = this.streamsService.getContext(streamId);
  if (!streamContext) return;
  Object.values(streamContext.viewers).forEach(({ socket }) => {
    socket.send(constructEvent(event));
  });
}
}

```

Далі наведено файл `apps/web/src/lib/hooks/useStreamer.ts` (лістинг А.4).

Лістинг А.4 – Хук `useStreamer` сторінки трансляції (скорочено)

```

// ...
const DEFAULT_VIDEO_SETTINGS: VideoSettings = { quality: StreamQuality.HD, fps: 30
};

export function useStreamer() {
  // ...
  const applyVideoSettings = useCallback(
    async (track: MediaStreamTrack, producer?: Producer) => {
      const derived = deriveEncodingForTrack(track);
      track.contentHint = derived.contentHint;
      try {
        await track.applyConstraints({ frameRate: derived.frameRate });
      } catch (e) {
        console.warn('[applyVideoSettings] applyConstraints failed:', e);
      }

      if (!producer?.rtpSender) return;
      const parameters = producer.rtpSender.getParameters();
      await producer.rtpSender.setParameters({
        ...parameters,
        degradationPreference: derived.degradationPreference,
        encodings: [{ ...parameters.encodings[0], ...derived.encoding }],
      });
    },
    [deriveEncodingForTrack],
  );

  const createProducer = useCallback(

```

```

async (track: MediaStreamTrack) => {
  let producerOptions: ProducerOptions = { track };
  if (track.kind === 'video') {
    const derived = deriveEncodingForTrack(track);
    producerOptions = {
      ...producerOptions,
      codec: pickVideoCodec(deviceRef.current?.rtpCapabilities.codecs),
      encodings: [derived.encoding],
      codecOptions: { videoGoogleStartBitrate: derived.startBitrateKbps },
    };
  }
  const producer = await transportRef.current!.produce(producerOptions);
  if (track.kind === 'audio' && isMuted) {
    producer.pause();
  }
  if (track.kind === 'video') await applyVideoSettings(track, producer);
  producersRef.current.push(producer);
},
[applyVideoSettings, deriveEncodingForTrack, isMuted],
);

const replaceStream = useCallback(
  async (stream: MediaStream) => {
    return Promise.all(
      stream.getTracks().map(async (track) => {
        const producer = producersRef.current.find((p) => p.kind ===
track.kind);
        if (!producer) return createProducer(track);

        if (!track) return producer.pause();

        await producer.replaceTrack({ track: track });
        if (producer.kind === 'video') await applyVideoSettings(track,
producer);
        if (producer.kind !== 'audio' || !isMuted) {
          producer.resume();
        }
      })
    );
  },
  [applyVideoSettings, createProducer, isMuted],
);
// ...
const connectToStream = useCallback(
  async (stream: Stream) => {
    if (!mediaStreamRef.current) {
      toast.error('Stream is not created yet');
      return;
    }
    const isTrackByKindSent = Object.fromEntries(
      mediaStreamRef.current.getTracks().map((t) => [t.kind, false]),
    );

    wsClientRef.current = new WsClient(signalingWsUrl(`/ws/streams/${
stream.id}/broadcast`));
    wsClientRef.current.ws.addEventListener('close', handleSocketClose);

    const { result: rtpCapabilities } = await wsClientRef.current.request({
      type: 'req',
      method: StreamerActions.GetRtpCapabilities,
      params: null,
    });

    const device = new Device();

```

```

await device.load({ routerRtpCapabilities: rtpCapabilities });
deviceRef.current = device;

const { result: transport } = await wsClientRef.current.request({
  type: 'req',
  method: StreamerActions.CreateTransport,
  params: { direction: 'send' },
});

transportRef.current = device.createSendTransport(transport);

transportRef.current.on('connect', ({ dtlsParameters }, callback,
errorCallback) => {
  wsClientRef
    .current!.request({
      type: 'req',
      method: StreamerActions.ConnectTransport,
      params: { dtlsParameters },
    })
    .then(() => callback())
    .catch(errorCallback);
});

transportRef.current.on('produce', ({ kind, rtpParameters }, callback,
errorCallback) => {
  isTrackByKindSent[kind] = true;
  const videoTrack = kind === 'video' ?
mediaStreamRef.current?.getVideoTracks()[0] : null;
  wsClientRef
    .current!.request({
      type: 'req',
      method: StreamerActions.Produce,
      params: {
        rtpParameters,
        kind,
        last: Object.values(isTrackByKindSent).every((e) => e === true),
        maxBitrate: videoTrack
          ? deriveEncodingForTrack(videoTrack).encoding.maxBitrate
          : undefined,
      },
    })
    .then((res) => callback({ id: res.result.producerId }))
    .catch(errorCallback);
});

await Promise.all(mediaStreamRef.current.getTracks().map(createProducer));
setStatus(StreamStatus.Live);

return stream;
},
[createProducer, deriveEncodingForTrack, handleSocketClose],
);

const broadcast = useCallback(async () => {
  const stream = await createStream(isPrivate);
  if (!stream) return;
  await connectToStream(stream);
  setCurrentStream(stream);
  stopCapturingThumbnailRef.current = startCapturingThumbnail(stream.id);
}, [connectToStream, isPrivate, startCapturingThumbnail]);
// ...

```

Далі наведено файл `apps/desktop/src/main/googleAuth.ts` (лістинг А.5).

Лістинг А.5 – Вхід через Google у системному браузері в настільному

застосунок (скорочено)

```

import { shell, type BrowserWindow } from 'electron';
import { createHash, randomBytes } from 'node:crypto';
import { createServer, type Server } from 'node:http';
import type { AddressInfo } from 'node:net';
import { bringWindowToFront } from './windowFocus';

const SIGN_IN_TIMEOUT_MS = 5 * 60 * 1000;

export type GoogleSignInResult =
  | { status: 'success'; code: string; verifier: string }
  | { status: 'cancelled' }
  | { status: 'timeout' }
  | { status: 'error'; message: string };

type Pending = {
  server: Server;
  settle: (result: GoogleSignInResult) => void;
  /** Handed to any later caller that arrives while this attempt is running. */
  result: Promise<GoogleSignInResult>;
};

let pending: Pending | null = null;

const base64url = (bytes: Buffer) => bytes.toString('base64url');
// ...
export function startGoogleSignIn(window: BrowserWindow):
Promise<GoogleSignInResult> {
  if (pending) return pending.result;

  const verifier = base64url(randomBytes(32));
  const challenge = createHash('sha256').update(verifier).digest('base64url');
  const state = base64url(randomBytes(16));

  const server = createServer();

  let settleAttempt: (result: GoogleSignInResult) => void = () => {};

  const attempt = new Promise<GoogleSignInResult>((resolve) => {
    let done = false;
    const settle = (result: GoogleSignInResult) => {
      if (done) return;
      done = true;
      clearTimeout(timer);
      if (pending?.server === server) pending = null;
      server.close();
      server.closeAllConnections();
      resolve(result);
    };

    const timer = setTimeout(() => settle({ status: 'timeout' })),
    SIGN_IN_TIMEOUT_MS);

    server.on('error', (error) =>
      settle({ status: 'error', message: error.message || 'The sign-in listener
failed' })),
    );

    server.on('request', (req, res) => {
      const url = new URL(req.url ?? '/', 'http://127.0.0.1');

```

```

    if (url.pathname !== '/callback') {
        respond(res, 404, 'Not found.');
```

return;

```
    }

    if (url.searchParams.get('state') !== state) {
        respond(res, 400, 'This sign-in link does not belong to this app.');
```

return;

```
    }

    const error = url.searchParams.get('error');
    const code = url.searchParams.get('code');
    if (error || !code) {
        respond(res, 200, 'Sign-in was not completed. You can close this tab and
try again.', () =>
            settle({
                status: 'error',
                message:
                    error === 'access_denied'
                    ? 'The browser did not finish signing in to a Stream Share
account.'
                    : 'The browser could not complete the sign-in.',
            })),
        );
        return;
    }

    bringWindowToFront(window);

    respond(res, 200, 'Signed in. You can close this tab and return to Stream
Share.', () =>
        settle({ status: 'success', code, verifier })),
    );
});

settleAttempt = settle;

void (async () => {
    try {
        const port = await listen(server);

        if (done) {
            server.close();
            return;
        }

        const url = new URL('/api/desktop-auth/start', __WEB_URL__);
        url.searchParams.set('port', String(port));
        url.searchParams.set('state', state);
        url.searchParams.set('challenge', challenge);

        await shell.openExternal(url.toString());
    } catch (error) {
        settle({
            status: 'error',
            message: error instanceof Error ? error.message : 'Could not open the
browser',
        });
    }
})();
});

pending = { server, settle: settleAttempt, result: attempt };

```

```
    return attempt;
}

export function cancelGoogleSignIn(): void {
  pending?.settle({ status: 'cancelled' });
  pending = null;
}
```

ДОДАТОК Б

Структура таблиць бази даних

У додатку наведено склад стовпців таблиць бази даних, описаних у підрозділі 3.3, за схемами пакета @stream-share/db. Стовпці без обмеження NOT NULL допускають порожнє значення.

Таблиця Б.1 – Структура таблиці user

Стовпець	Тип	Обмеження	Призначення
id	text	PK, UUID за замовчуванням	ідентифікатор користувача
email	varchar(512)	UNIQUE	адреса електронної пошти Google
emailVerified	timestamp	–	час підтвердження адреси (поле Auth.js)
name, image	varchar(255), text	–	ім'я та аватар
role	varchar	NOT NULL, user за замовчуванням	роль: user, admin, guest
status	varchar	NOT NULL, active за замовчуванням	стан: active, blocked, deleted
password_hash, failed_login_count, locked_until	text, integer, timestampz	failed_login_count NOT NULL, 0 за замовчуванням	зарезервовано, функціональність не використовується
created_at	timestampz	NOT NULL, поточний час за замовчуванням	час створення
activeAt	timestampz	поточний час за замовчуванням	час останньої активності гостя

Таблиця Б.2 – Структура таблиці account

Стовпець	Тип	Обмеження	Призначення
provider, providerAccountId	text	складений PK	провайдер і ідентифікатор користувача в ньому
userId	text	NOT NULL, FK → user.id	власник облікового запису
type	text	NOT NULL	тип облікового запису за Auth.js
access_token, refresh_token, id_token, expires_at, token_type, scope, session_state	text, integer	–	дані токенів провайдера OAuth

Таблиця Б.3 – Структура таблиці stream

Стовпець	Тип	Обмеження	Призначення
id	text	PK, UUID за замовчуванням	ідентифікатор трансляції, частина посилання на перегляд
user_id	text	NOT NULL, FK → user.id	стрімер
isPrivate	boolean	false за замовчуванням	приватна трансляція не потрапляє до загального переліку
status	varchar	created за замовчуванням	стан (рис. 3.7)
endReason	varchar	–	streamer_stop, timeout, server_force, admin_force
startedAt, endedAt	timestampz	startedAt – поточний час за замовчуванням	час початку й завершення
thumbnailUpdatedAt	timestampz	–	час останнього оновлення мініатюри

Таблиця Б.4 – Структура таблиці stream_to_user

Стовпець	Тип	Обмеження	Призначення
id	text	PK, UUID за замовчуванням	ідентифікатор запису
stream_id	text	NOT NULL, FK → stream.id	трансляція
user_id	text	NOT NULL, FK → user.id	глядач, підключений зараз

Таблиця Б.5 – Структура таблиці audit_log

Стовпець	Тип	Обмеження	Призначення
id	uuid	PK, випадковий UUID за замовчуванням	ідентифікатор запису
actor_user_id	text	FK → user.id	автор дії
action, targetType, targetId	text	–	дія та її об'єкт
metadata	jsonb	–	додаткові дані
createdAt	timestampz	поточний час за замовчуванням	час дії

ДОДАТОК В

Тестові приклади функціонального тестування

У додатку наведено тестові приклади функціонального тестування, проаналізовані в підрозділі 5.2, згруповані за сценаріями використання. Ідентифікатор прикладу збігається з позначкою у виводі сценаріїв `smoke.mjs` і `follower.mjs`.

Таблиця В.1 – Тестові приклади автентифікації

ID	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
A 1	Анонімний запит закритої сторінки	GET /broadcast без cookie	переспрямування на /login	307 → /login
A 2	Анонімний перегляд	GET /<uuid>/watch без cookie	сторінка доступна	200
A 3	Гостьовий вхід	/api/auth/csrf, POST /api/auth/callback /guest	видано cookie сеансу	__Secure-authjs.session-token, строк дії 30 діб
A 4	Атрибути cookie сеансу	заголовок Set-Cookie з A3	HttpOnly, Secure, SameSite	HttpOnly; Secure; SameSite=Lax
A 5	Запис гостя	запит до таблиці user	роль guest, випадкове ім'я	роль guest, ім'я з двох випадкових слів
A 6	Гість на закритій сторінці	GET /broadcast з cookie гостя	переспрямування на /login	307 → /login
A 7	REST без сеансу та з підробленим токеном	POST /streams без cookie; з cookie forged.token	401 в обох випадках	без cookie – 401; з підробленим токеном – 500 (не відповідає)
A 8	WebSocket без сеансу	підключення до /ws/streams/<id>/broadcast	відмова в ручокостисканні	HTTP 401
A 9	Вхід через Google	кнопка «Google», вибір облікового запису, вихід	сеанс користувача, після виходу – сторінка входу	[ПОТРЕБУЄ УТОЧНЕННЯ: виконати вручну з робочими ключами OAuth; вказати браузер]
A 10	Вхід у настільному застосунку	кнопка входу, вхід у системному браузері, повернення до застосунку; повторити 5 разів поспіль і зі скасуванням	сеанс у застосунку на кожній спробі	[ПОТРЕБУЄ УТОЧНЕННЯ: виконати вручну в зібраному застосунку; вказати ОС і кількість успішних спроб]

Таблиця В.2 – Тестові приклади трансляції з браузера

I D	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
Б 1	Створення трансляції	POST /streams з isPrivate false i true	201, статус created	обидві створено, статус created
Б 2	Можливості маршрутизатора	запит getRtpCapabilities	H.264 першим, VP8 резервним	video/H264, video/H264, video/VP8
Б 3	Транспорт стрімера	createTransport (send)	параметри ICE з оголошеною адресою	кандидати udp i tcp на 127.0.0.1
Б 4	Публікація відео	produce з ознакою last	статус трансляції live	статус live
Б 5	Гість створює трансляцію	POST /streams з cookie гостя	відмова	201, трансляцію створено (не відповідає)
Б 6	Мініатюра трансляції	POST /streams/<id>/thumbnail від власника та від гостя	власнику – 200, іншим – 403	200 i 403
Б 7	Захоплення в браузері	вибір екрана, вікна, вкладки зі звуком у Chrome i Firefox	зображення та звук у глядача	[ПОТРЕБУЄ УТОЧНЕННЯ: результат для кожного браузера і типу джерела]
Б 8	Зміна якості під час трансляції	перемкнути 720p30 → 1080p60	нові frameWidth, framesPerSecond у chrome://webrtc-internals	[ПОТРЕБУЄ УТОЧНЕННЯ: значення до й після перемикання]
Б 9	Звук і посилання	вимкнути та ввімкнути звук; скопіювати посилання	звук зникає без зупинки трансляції; посилання відкриває перегляд	[ПОТРЕБУЄ УТОЧНЕННЯ: результат ручного виконання]

Таблиця В.3 – Тестові приклади настільного застосунку

ID	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
Д1	Таблиця процесів ОС	<code>listProcesses()</code> на Linux (ps)	власний процес із правильним батьківським	40 процесів, батьківський PID збігається
Д2	Вікно оболонки	обрано вікно <code>explorer.exe</code> , з'являється дочірнє вікно	слідування не вмикається	подій немає
Д3	Нове вікно застосунку	у процесу застосунку з'являється дочірній процес із вікном	перемикавання на нове вікно	перемикавання через 1,5 с
Д4	Вікно стороннього процесу	вікно процесу, не пов'язаного з обраним	джерело не змінюється	джерело не змінилося
Д5	Закриття нового вікна	вікно й процес із Д3 зникають	повернення до вихідного вікна	повернення через 1,3 с
Д6	Слідування вимкнено	як у Д3 при <code>setEnabled(false)</code>	джерело не змінюється	подій немає
Д7	Вихідне вікно не повернулося	закрито і вихідне, і нове вікно	подія <code>lost</code> після очікування	<code>lost</code> через 22,3 с
Д8	Перелік джерел	відкрити вибір джерела у Windows	вікна й екрани з мініатюрами та піктограмами	[ПОТРЕБУЄ УТОЧНЕННЯ: результат і версія Windows]
Д9	Звук застосунку	трансляція вікна застосунку зі звуком; застосунок без звуку	звук лише обраного застосунку; повідомлення при неможливості	[ПОТРЕБУЄ УТОЧНЕННЯ: результат ручного виконання]
Д10	Слідування в реальному застосунку	трансляція програми запуску гри, що відкриває вікно гри	глядач бачить вікно гри без дій стрімера	[ПОТРЕБУЄ УТОЧНЕННЯ: назва застосунку і результат]

Таблиця В.4 – Тестові приклади перегляду трансляції

ID	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
П 1	Перелік трансляцій	<code>POST /streams/search</code>	публічна є, приватна відсутня	так
П 2	Приватна трансляція за посиланням	<code>GET /streams/<id></code> приватної	200	200
П 3	Неіснуюча трансляція	<code>GET /streams/<випадковий uuid></code>	404	404
П 4	Гість приєднується	<code>joinStream</code> із cookie гостя	перелік потоків стрімера	один потік
П	Створення	<code>createTransport(recv)</code> ,	споживач відео	споживач

ID	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
5	споживача	consume		video/VP8
П 6	Відтворення	відкрити посилання в іншому браузері; гучність, «картинка в картинці», повний екран	зображення і звук, усі елементи керування працюють	[ПОТРЕБУЄ УТОЧНЕННЯ: результат і браузер]
П 7	Мініатюри й сторінки	понад 12 активних трансляцій	12 карток на сторінці з мініатюрами	[ПОТРЕБУЄ УТОЧНЕННЯ: результат ручного виконання]

Таблиця В.5 – Тестові приклади завершення та відновлення трансляції

ID	Сценарій	Вхідні дані / кроки	Очікуваний результат	Фактичний результат
31	Втрата з'єднання стрімера	розірвати WebSocket стрімера	глядач отримує streamerDisconnect	подію отримано через 2–4 мс
32	Стрімер не повернувся	чекати після 31	завершення після 5 спроб по 6 с, причина timeout	завершено через 36,0–36,1 с, endReason = timeout
33	Завершення стрімером	endStream	глядач отримує streamEnd, причина streamer_stop	так
34	Сторонній користувач	гість відкриває сокет трансляції іншого користувача і надсилає endStream	відмова	сокет відкрито, трансляцію завершено (не відповідає)
35	Відновлення	перезавантажити сторінку стрімера протягом 30 с	streamerReconnected, зображення відновлюється	[ПОТРЕБУЄ УТОЧНЕННЯ: результат і час відновлення]